Made with Goodnotes

Pipelining (3,5)

sidere três máquinas M1, M2 e M3 diferentes com as seguintes características:
M1) não utiliza nenhuma previsão de desvios;
M2) faz uma previsão fixa para todos os desvios com taxa de acerto de 65%;
M3) prevê desvios de laços (loops) com 90% de acerto e de desvios condicionais com 60
de maneira independente;
M4) igual a maquina M3, mas que emprega *data forwarding* e um compilador que reorder
as instruções de tal maneira que elimine todas as dependências de dados.

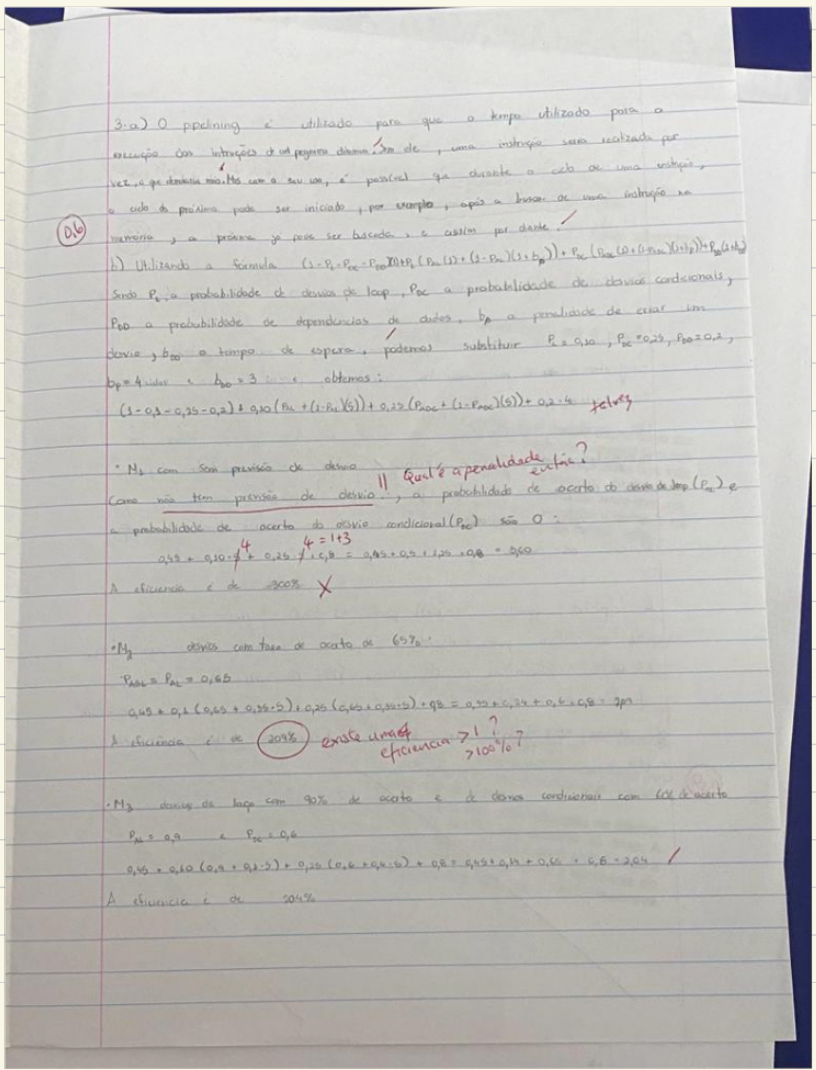
ponha um programa onde 20% das instruções têm dependências de dados, 25% das instruçõe
o desvios condicionais e 10% são desvios de *loop*. A penalidade de esperar (dados) é de
3 ciclos, mas de adivinhar errado é de 4 ciclos. Qual a eficiência do uso de pipeline para cada um
as máquinas M1, M2, M3 e M4? **Descreva seu raciocínio.**

1 - a) o pipelining é utilizado para que o tempo utilizado para a execução das instruções de um programa diminua. sem ele, uma instrucao seria realizada por vez, o que demoraria mais. Mas com o seu uso, é possível que durante o ciclo de uma instrução, o ciclo da próxima possa er iniciado, por exemplo, após buscar uma instrucao na memoria, a próxima ja pode ser buscada e assim por diante.

B) Utilizando a formula sendo p1 a probabilidade de desvios de loop, P? A probabilidade de desvios condicionais, PDD a probabilidade de dependências de dados, B? A probabilidade de criar um desvio, B? O tempo de espera, podemos substituir P1 = 0,10, P? = 0,25, P? = 0,2, B? = 4 e B? = 3 e obtemos :

M1 sem previsão de desvio
Como nao tem previsão de desvio, a probabilidade de acerto do desvio do jump (P?) e a probabilidade de acerto do desvio condicional (P?) são O :
0,45 + 0,10*4 + 0,25*4 + C1B = 0,45 + 0,5 + 1,25 + 0,8 = B1C0
A eficiência é de 300% (?)

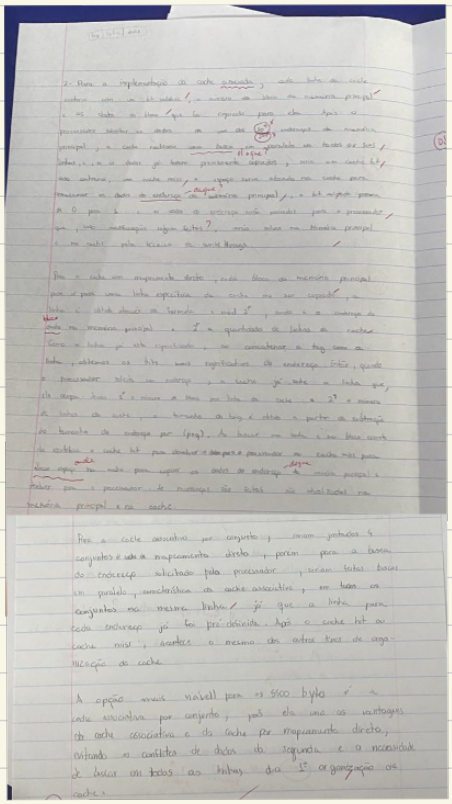
M2 desvios com taxa de acerto de 65%



Made with Goodnotes

Memória Cache (3,5)

cê é um projetista de processadores. Sua tarefa é de decidir qual tipo de cache deve se
plementado com os 5500 bytes disponíveis na pastilha. A memória principal tem 1 Gbyte e
nanho do bloco é de 64 bits. Projete implementações apropriadas dos três tipos de cach
udados durante o curso (cache associativo, cache com mapeamento direto e a cach
sociativo por conjunto) com a técnica de *write-through*. Dica: implementá-los nesta ordem
ondo que o conjunto tem tamanho 4 no caso da cache associativo por conjunto. **Discuta su**
isão.



Informações :

5500 Bytes livres para cache

1GB de memória principal

Barramento de 64 bits

→ 8 bytes / bloco (2^3)

$$\frac{2^{30}}{2^3} = 2^{27}$$

log₂(blocos da MP)

1	27	3
Validade	Endereço	identificador de bloco

nº bytes na cache

$$\frac{2^{12}}{2^3} = 2^9 \text{ linhas}$$

(tamanho do bloco)

bloco na MP

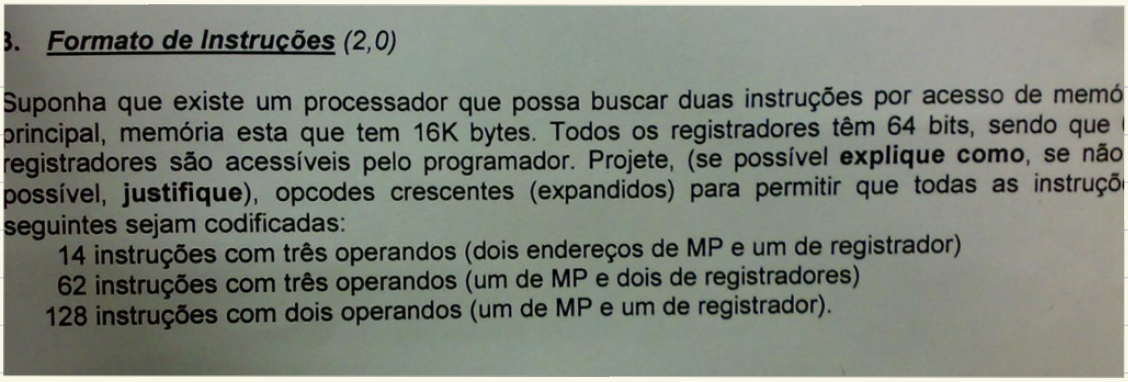
$$\frac{2^{27}}{2^9} = 2^{18} \leftarrow \text{Tags}$$

(18 ← bits de tag; linhas)

Cache = validade + tag + bloco

1	18	3
validade	tag	identifica bloco

Made with Goodnotes

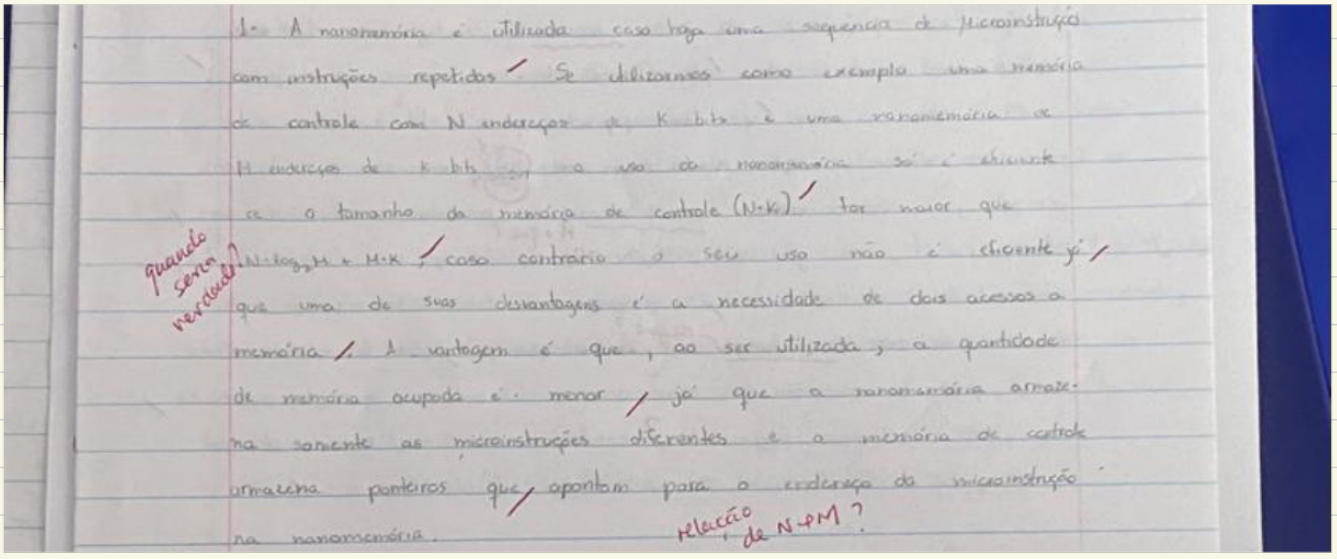


3. ***Formato de Instruções*** *(2,0)*

Suponha que existe um processador que possa buscar duas instruções por acesso de memó principal, memória esta que tem 16K bytes. Todos os registradores têm 64 bits, sendo que registradores são acessíveis pelo programador. Projete, (se possível **explique como**, se não possível, **justifique**), opcodes crescentes (expandidos) para permitir que todas as instruçõ seguintes sejam codificadas:

14 instruções com três operandos (dois endereços de MP e um de registrador)
62 instruções com três operandos (um de MP e dois de registradores)
128 instruções com dois operandos (um de MP e um de registrador).

3 - A nanomemória é utilizada caso haja uma sequencia de microinstruções com instruções repetidas, se utilizarmos como exemplo uma memória de controle com N endereços de K bits e uma nanomemória com M endereços de K bits o uso da nanomemória so é eficiente se o tamanho da memoria de controle (N + K) for maior do que N + log2 M+ M+K, caso contrario seu uso nao é eficiente, ja que uma de suas desvantagens é a necessidade de dois acessos a memoria. A vantagem é que ao ser utilizada, a quantidade de memória ocupada é menor, ja que a nanomemória armazena somente as microinstruções diferentes e a memoria de controle armazena ponteiros que apontam para o endereço da microinstrução na nanomemória.



3- A nanomemória é utilizada caso haja uma sequencia de microinstrução com instruções repetidas. Se utilizarmos como exemplo uma memória de controle com N endereços de K bits e uma nanomemória de M endereços de K bits o uso da nanomemória só é eficiente se o tamanho da memória de controle (N·K) for maior que quando será verdade! N·log2 M + M·K; caso contrário o seu uso não é eficiente já que uma de suas desvantagens é a necessidade de dois acessos a memória. A vantagem é que, ao ser utilizada, a quantidade de memória ocupada é menor, já que a nanomemória armazena somente as microinstruções diferentes e a memória de controle armazena ponteiros que apontam para o endereço da microinstrução na nanomemória.

relação de N+M?

Made with Goodnotes

4. ***Endereçamento*** *(1,0)*

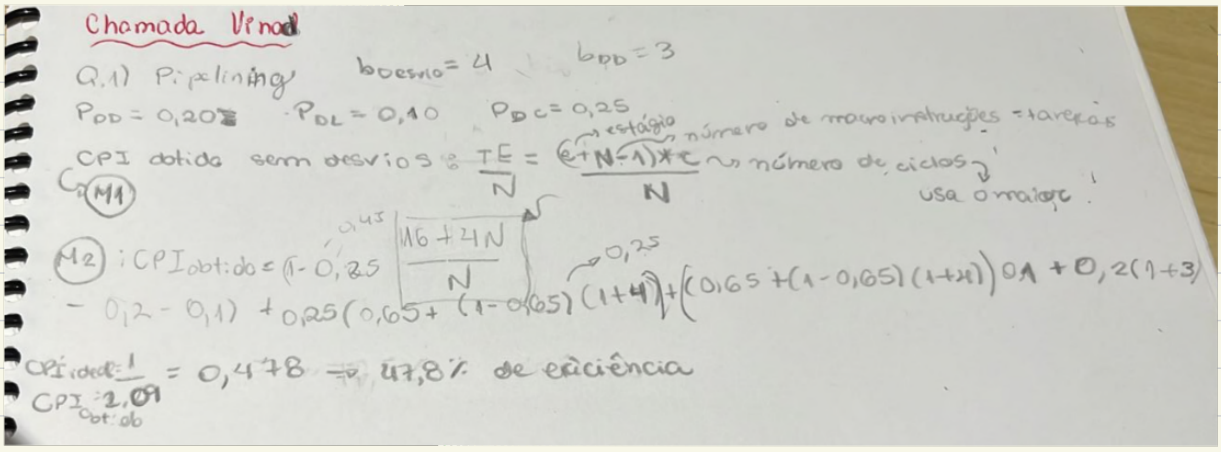
Descreva como acessar um conjunto arbitrário de 2^x endereços, não necessariamente contíguos contidos em um espaço de endereçamento grande (maior que 2^x) e especificados através de um campo de só *x* bits.

Made with Goodnotes

Eficiência = Vazão

$$\text{Vazão} = \frac{N}{(e+N-1)*c}$$

Vazão^{-1} = tempo para executar cada instrução



Made with Goodnotes

Made with Goodnotes

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 2 – MICROARQUITETURA (AULA 18B)

Made with Goodnotes

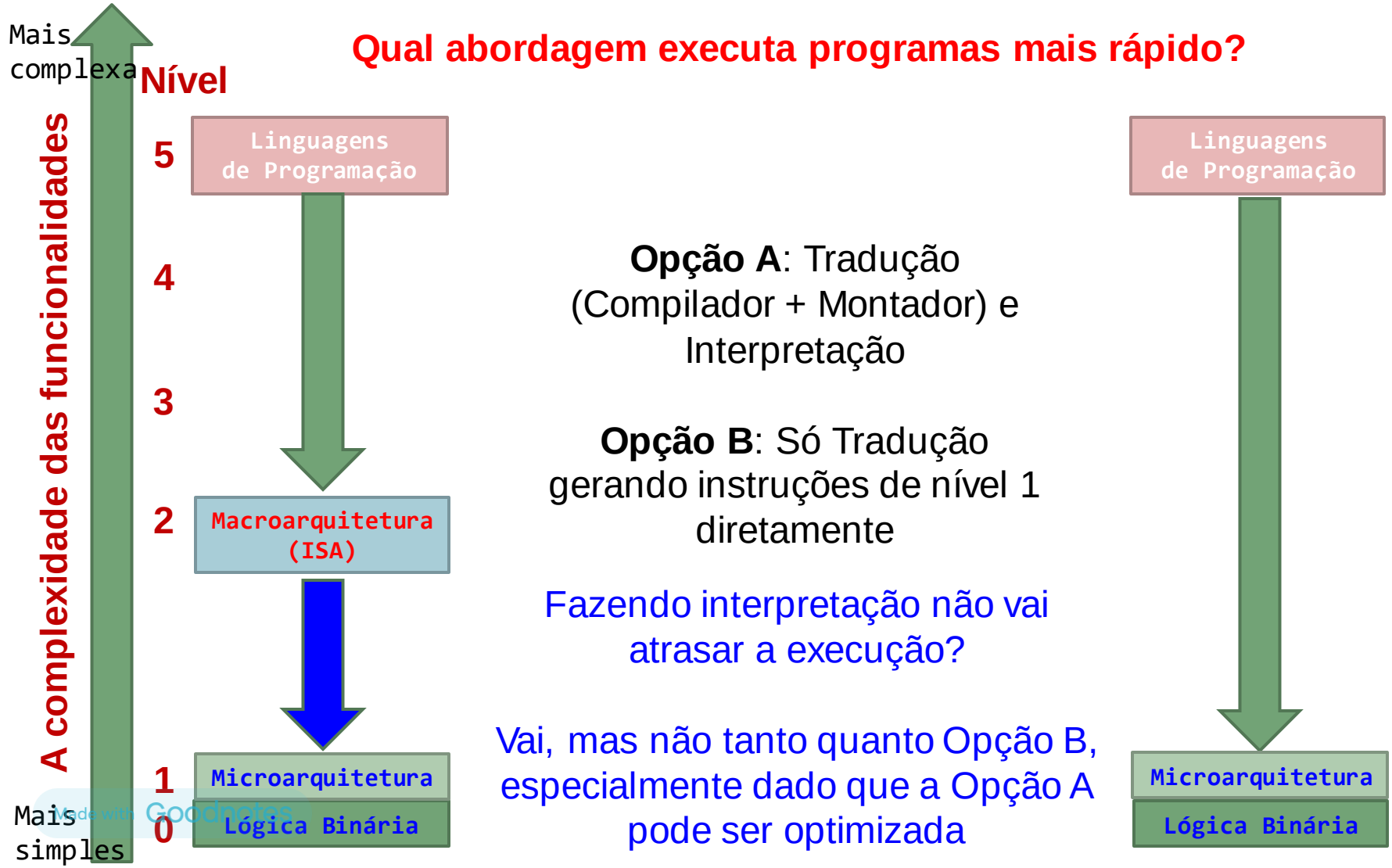
Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

Tópico 2: A grande pergunta

- Nossa questão tornou-se uma de
  - Macro-programação versus Micro-programação ou, em outras palavras, ...
  - Será que o compilador deve gerar uma saída para nível 1 ao invés de nível 2?
    - *"Ser ou não ser? Eis a questão."* O início do solilóquio de Príncipe Hamlet da peça *Hamlet* de William Shakespeare
    - Também é título de um conto escrito por Machado de Assis, em 1876.
    - Então, fazer ou não fazer?

Made with Goodnotes

Tópico 2: A grande pergunta



Macro- x Micro-programação

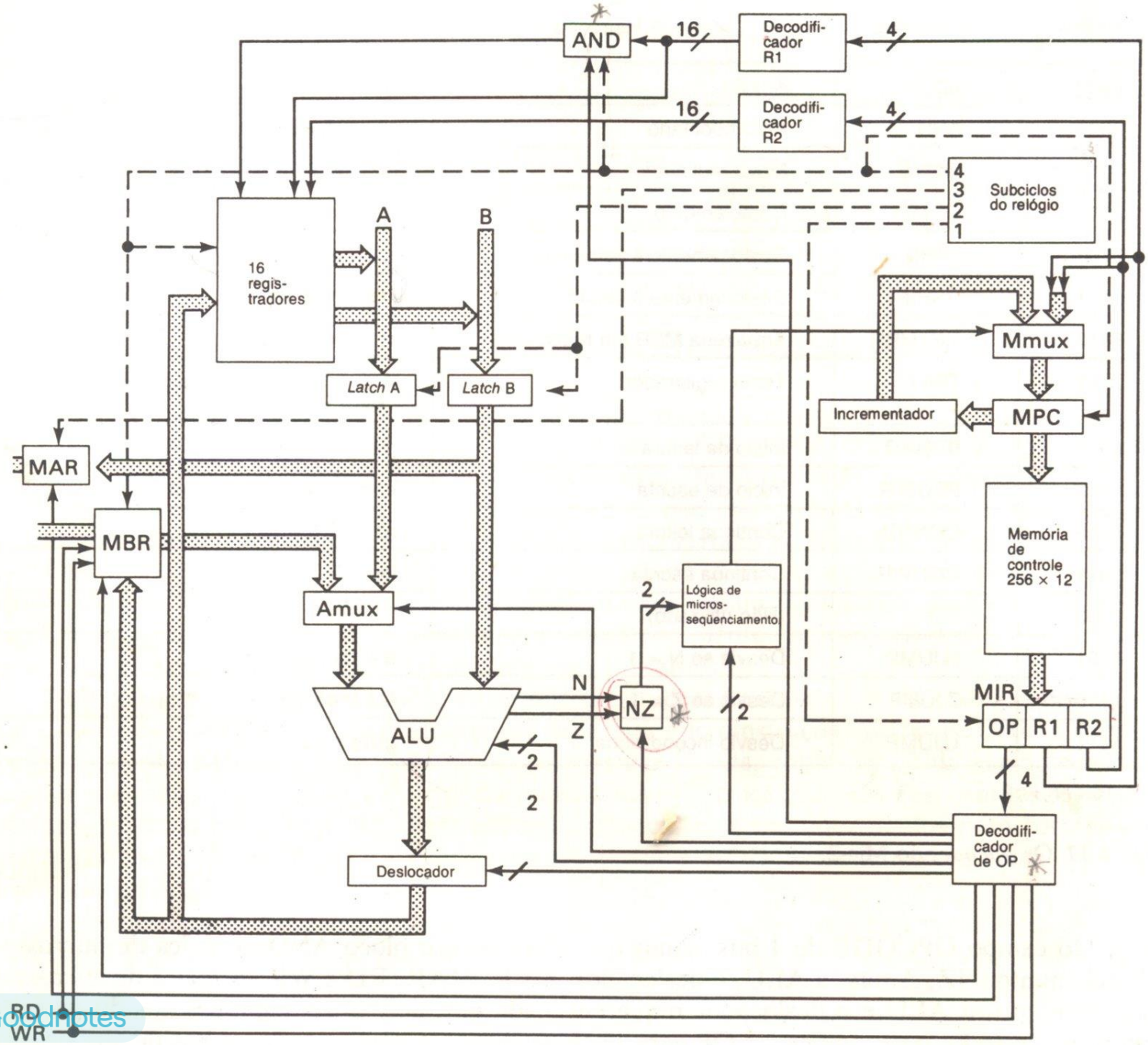
- Um compilador deve gerar uma saída para nível 1 ao invés de nível 2?
  - Na pratica, seria inviável devido às complexidades das linguagens e µarquiteturas e por ser uma solução ineficiente, cara, lento e de difícil implementação!
  - Enquanto sem a interpretação, a execução direto de µinsts poderia aparecer de ser mais rápido. Na verdade ...
    - Vai precisar de mais memória para guardar o programa: um MC grande é muito caro; além disso, deve ser **accessível** e possível **escrever** nela; **[Custo ☒]**
    - Se usa MP, por ser mais lento, a execução fica mais lento **[Desempenho ☒]**
  - Compilador precisa explorar a µarq., µinsts, os registradores, concorrência, etc.
    - Número de µinsts é bem maior do que macroinsts, mais complexidade **[Facilidade ☒]**
  - Cada nova arquitetura tem novas µinsts ⟶ um novo compilador, e mais custos
    - Lembre-se da quantidade compiladores que serão necessários: ***m* × *n*** invés de ***m***

Made with Goodnotes

Microprogramação

- Microprogramação Horizontal x Microprogramação Vertical (MV)
  - MIC-1 é um exemplo de microprogramação horizontal
    - O opcode é composto de sinais de controle, explicitamente, então as µinsts estão largas, com muitos bits
    - Assim, suporta a realização de várias ***micro-operações*** na mesma µinstrução
      - Por ex., **`mar := &m; mbr := lshift(mbr + ac); wr; if n goto 23`**
  - Podemos projetar um MIC-2 que emprega MV com a mesma VD
    - Por exemplo, suas µinstruções terão só 12 bits, com opcodes de só 4 bits
      - Quantos µinsts na µlinguagem de MIC-2 x MIC-1?
    - O microprograma porém teria 161 µinstruções, por ter instruções mais simples
    - Mas o microprograma do MIC-2 só ocupa 1920 bits, 24% a menos dos 2528 bits do microprograma do MIC-1!

Made with Goodnotes

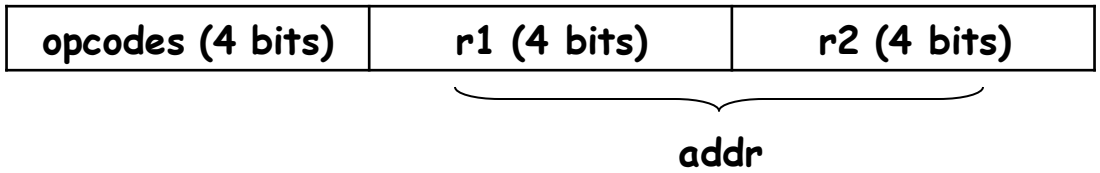


Made with GoodNotes

As Microinstruções de MIC-2

Binário	Mnemônico	Função	Significado
0000	ADD	Adição	r1:= r1 + r2
0001	AND	E booleano	r1:= r1 AND r2
0010	MOVE	Move entre registradores	r1:= r2
0011	COMPL	Complemento	r1:= inv (r2)
0100	LSHIFT	Deslocamento à esquerda	r1:= lshift (r2);
0101	RSHIFT	Deslocamento a direta	r1:= rshift (r2);
0110	GETMBR	Armazena MBR em registrador	r1:= MBR
0111	TEST	Testa registrador	if r2<0 then N=true; if r2=0 then Z=true;
1000	BEGRD	Início de leitura	mar := r1; rd;
1001	BEGWR	Início de escrita	mar := r1; mbr := r2; wr;
1010	CONRD	Continua leitura	rd;
1011	CONWR	Continua escrita	wr;
1100			
1101	NJUMP	Desvia se N	if N then go to addr
1110	ZJUMP	Desvia se Z	if Z then go to addr
1111	UJUMP	Desvio incondicional	go to addr

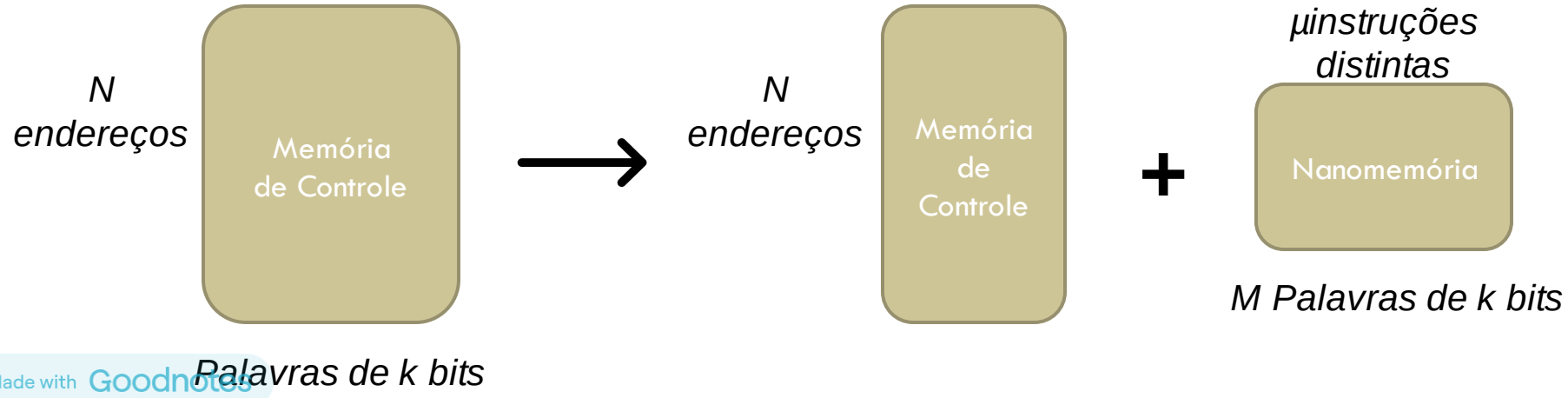
Formato da microinstrução:



Made with Goodnotes

Usando ainda menos memória

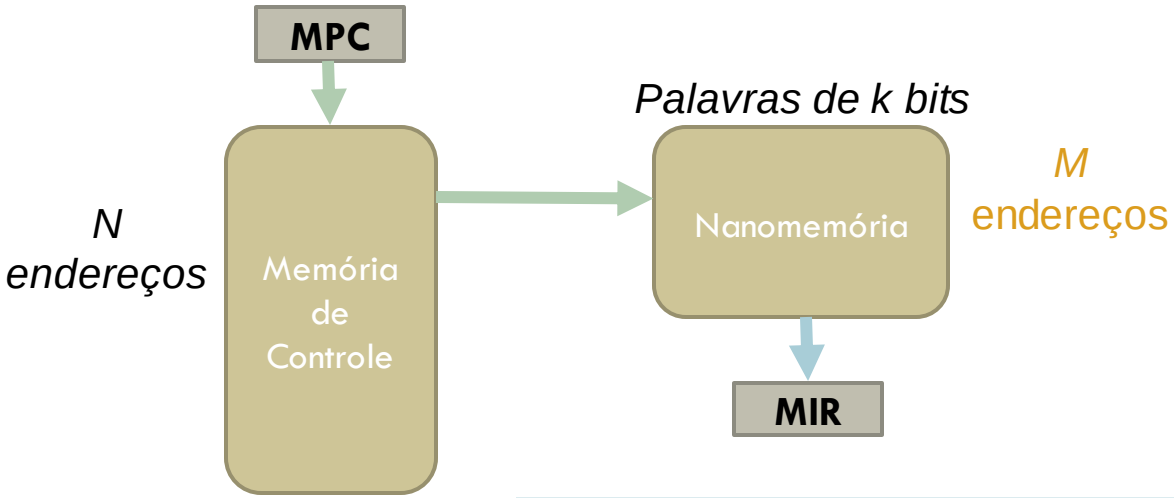
- Que é um programa? Para que serve memória?
- **Nanoprogramação** pode útil quando temos muitas (micro)instruções repetidas
  - Utiliza uma memória adicional (dentro a UC) para guardar μinstruções distintas, chamada uma nanomemória



Made with Goodnotes

Nanoprogramação e Nanomemória

- Um programa é uma sequência de instruções
  - Se a nanomemória guarda as *M* µinstruções (distintas), o que temos na Memória de Controle?
    - A sequência, na forma de ponteiros (endereços da nanomemória)



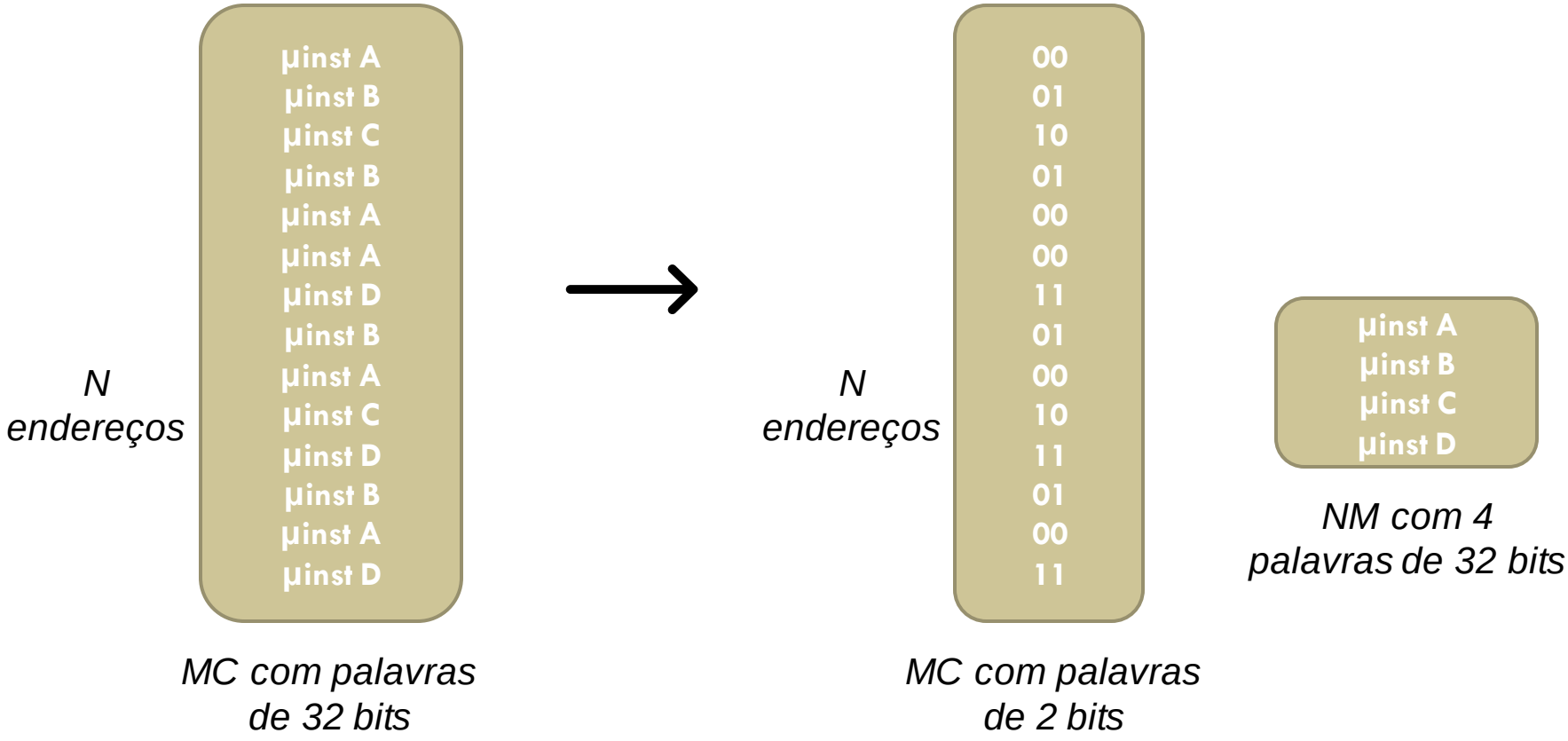
Se *N* é muito maior de *M* então $N.k > N.\log_2 M + M.k$
Isto é, a UC seria mais barato por requer menos espaço

Busca de µinsts requer dois acessos de memória que pode atrasar a execução

Made with Goodnotes

Exemplo de Nanoprogramação

- Nanomemória guarda as instruções distintas



Em termos de espaço, dado $M = 4$, vale a pena quando

$32 \times N > 2 \times N + 128 \Rightarrow N > 128/30$

Made with Goodnotes

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 2 – MICROARQUITETURA (AULA 19)

Made with Goodnotes

Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

O que vem a seguir em Arq. Comp.?

- Processadores executam programas sequências seguindo o modelo Von Neumann
  - A gente projetou um em poucos semanas...
  - O que é que a comunidade/indústria tem feita nos últimos 70 anos?
  - Que mais temos que aprender?
  - Que é um computador?
  - Às vezes você tem que dar um passo para trás para dar dois para frente

Made with Goodnotes

O que vem a seguir em Arq. Comp.?

- Processadores executam programas sequências seguindo o modelo Von Neumann
  - O que é que a comunidade/indústria tem feita nos últimos 70 anos?
  - Que mais temos que aprender?
  - E a Lei de Moore (2x transistores cada 2 anos) e *Dennard's Scaling* (2x desempenho/watt cada 18 meses) ?
    - O consumo de energia dinâmica dos circuitos CMOS é proporcional à frequência.
    - Historicamente, a redução de potência do transistor proporcionada pelo Dennard's scaling permitiu que os fabricantes aumentassem drasticamente as frequências do clock de uma geração para a seguinte, sem aumentar significativamente o consumo geral de energia do circuito.

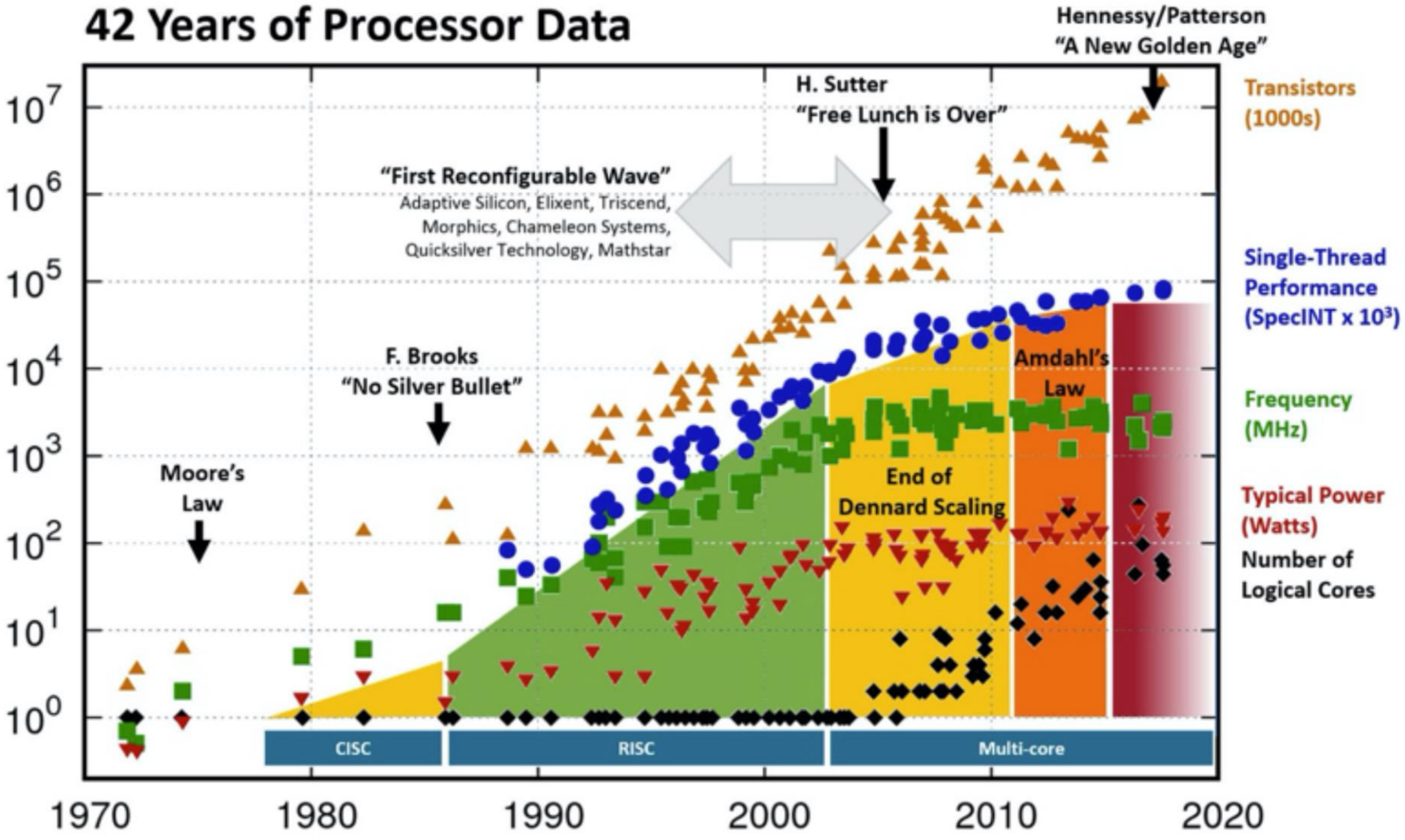
Made with Goodnotes

O que vem a seguir em Arq. Comp.?

- Processadores executam programas sequências seguindo o modelo Von Neumann
  - O que é que a comunidade/indústria tem feita nos últimos 70 anos?
  - Que mais temos que aprender?
  - E a Lei de Moore (2x transistores cada 2 anos) e *Dennard's Scaling* (2x desempenho/watt cada 18 meses) ?
    - O consumo de energia dinâmica dos circuitos CMOS é proporcional à frequência.
    - Historicamente, a redução de potência do transistor proporcionada pelo Dennard's scaling permitiu que os fabricantes aumentassem drasticamente as frequências do clock de uma geração para a seguinte, sem aumentar significativamente o consumo geral de energia do circuito.
  - Mas além disso, podemos otimizar "o modelo de Von Neumann"?

Made with Google Slides

O que vem a seguir? Quem sabe?



Hennessy and Patterson, Turing Lecture 2018, overlaid over "42 Years of Processors Data"
https://www.karlrupp.net/2018/02/42-years-of-microprocessor-trend-data/;"First Wave" added by Les Wilson, Frank Schirrmeister
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

Made with Goodnotes

O que vem a seguir? Quem sabe?

- Duas coisas aconteceram na mesma época em meados dos anos 2000.
- Primeiro, *Dennard's Scaling* realmente chegou ao fim.
  - *Dennard's Scaling permitiu que* um processo de fabricação pode escalar para um outro e aumentar o desempenho sem um aumento significativo do consumo de energia.
  - Antes, as taxas do relógio continuaram aumentar, chegando cerca aos 3 GHz sustentado, mas esta tendência parece que chegou ao fim.
  - O CTO da Intel apontou que se continua assim, as futuras densidades de energia se aproximaram daquelas do núcleo de reatores nucleares ou do centro do sol.
  - A partir desse ponto, em vez de aumentar as taxas de clock, o poder de computação adicional foi fornecido na forma de mais núcleos.

Made with Goodnotes

O que vem a seguir? Quem sabe?

- Duas coisas aconteceram na mesma época em meados dos anos 2000.
- A segunda coisa foi que os arquitetos tinham feito tudo o que podiam pensar para melhorar o desempenho de um processador de uso geral: pipelines, caches multinível, execução fora de ordem, previsão de desvios, multicores, ...
  - Se você precisa obter mais desempenho, não o obterá de graça apenas esperando, como aconteceu naquele período anterior.
  - Você pode ter mais de tudo... mais núcleos de uso geral, mais processadores de uso geral.
  - O que você não pode ter é mais desempenho para uma única tarefa executada em um processador de uso geral.

Made with Goodnotes

A nova *era dourada* de Arq. Comp.

- Se você não consegue obter mais desempenho em um processador de uso geral, a resposta é usar processadores de uso especifico.
- Uma idade de ouro, por quê? Arquiteturas novas e inovadoras serão importantes e não serão superadas apenas alguns anos depois pelas melhorias na computação de uso geral.
- Estamos agora na era da computação de domínios específicos.
  - Se olharmos para o quadro geral, um processador de uso geral começa a parecer ainda menos a solução certa em todas as circunstâncias.
  - Muitos algoritmos são inerentemente paralelos. Na maioria das linguagens de programação, como C++ ou Python, o programador precisa ordenar essas operações em um fluxo sequencial.

Made with Goodnotes

O que vem a seguir para a gente?

- Processadores executam programas sequências seguindo o modelo Von Neumann
  - A gente projetou um processador em poucos semanas...
  - O que é que a comunidade/indústria tem feita nos últimos 70 anos?
  - Que mais temos que aprender?
- Como criar um processador que é mais rápido sem ser mais caro e que continua ser fácil de usar?
  - A facilidade vem de abstração e a realização de traduções de programas escrita em linguagens de programação (sequencias) de alto nível
  - Por ser linguagens sequencias, o fator limitante é que processadores são obrigados a seguir o modelo Von Neumann (A Lei de Programação) ...
  - Como acelera algo que é inerentemente sequencial?
- Bem-vindo ao mundo da ***Microarquitetura!***

Made with Goodnotes

A Nova Era Dourada de Arq. Comp.

- O compilador do código-fonte toma decisões realmente detalhadas sobre exatamente quais instruções, em que ordem, devem operar em quais registradores.
- Mas o processador então pega esta codigo, altamente otimizadas para execução sequencial, e tenta encontrar paralelismo:
  - Ele substitui as decisões tomadas pelo compilador quanto à ordem das instruções, reordenando-as, os registradores a serem usados, renomeando-os e até executa instruções que não deveria enquanto não executa instruções que talvez deveria;
  - Ele adivinha quais acessos à memória serão necessários, usando caches, que funcionam na maioria das vezes;
  - Ele adivinha para que lado os desvios irão e executa instruções ***especulativamente***;
  - E talvez a parte mais incrível de todas, ele faz tudo isso de uma maneira que todas as instruções parecem ser executadas na ordem exata em que o programador e o compilador escreveram em primeiro lugar.
- Mas para algoritmos especializados, com paralelismo e algoritmos bem compreendidos, jogar todo o conhecimento fora e tentar recuperá-lo rapidamente pode não ser o ideal.

Made with Goodnotes

O que vem a seguir para a gente?

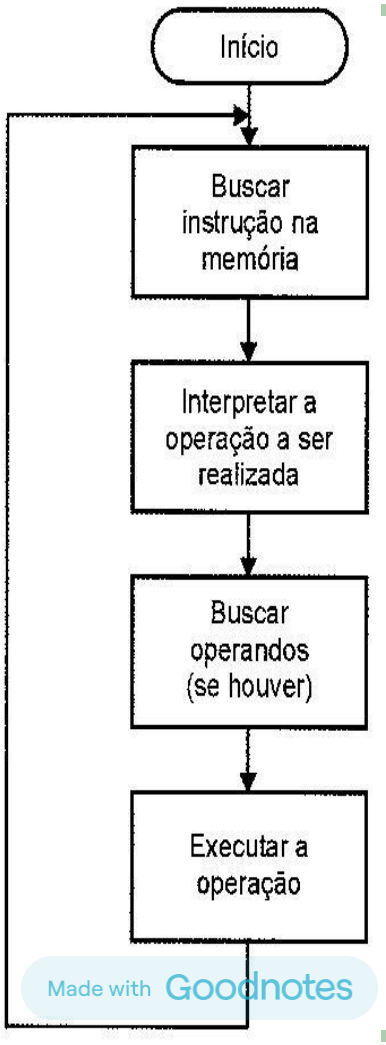
- Processadores executam programas sequências seguindo o modelo Von Neumann
  - A gente projetou um processador em poucos semanas...
  - O que é que a comunidade/indústria tem feita nos últimos 70 anos?
  - Que mais temos que aprender?
- Como criar um processador que é mais rápido sem ser mais caro e que continua ser fácil de usar?
  - A facilidade vem de abstração e a realização de traduções de programas escrita em linguagens de programação (sequencias) de alto nível
  - Por ser linguagens sequencias, o fator limitante é que processadores são obrigados a seguir o modelo Von Neumann (A Lei de Programação) ...
  - Como acelera algo que é inerentemente sequencial?
- Bem-vindo ao mundo da ***Microarquitetura!***



But don't get seen!!

Made with Goodnotes

O que acontece na μarquitetura?



Made with Goodnotes

Nível

6 ???

5 Linguagens de Programação



4 Linguagem de Montagem

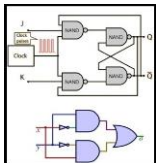
```
Load r1, X
Load r2, Y
Mult r3, r2, r1
Load r4, A
Mult r2, r4, r1
Add r5, r2, r4
Mult r1, r2, r5
Load r3, B
Add r7, r1, r3
```

3 Sistema Operacional

2 Macroarquitetura (ISA)

```
10100010
00100101
11100011
10001001
10101010
00100011
```

1 Microarquitetura



0 Lógica Binária

Desempenho

Compile uma vez, execute muitas vezes

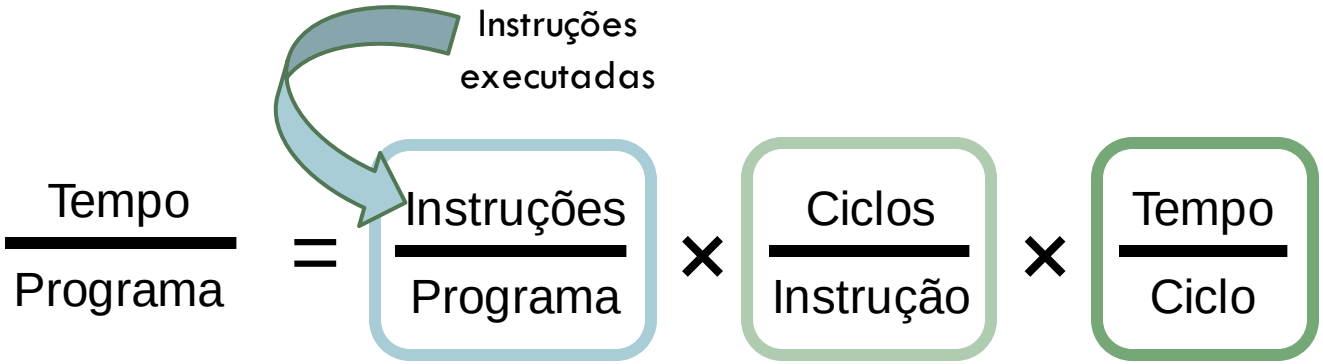
Interpretação Parcial de instruções do programa compilado

Interpretação de instruções da ISA

Execução de instruções da linguagem de nível 1

Desempenho arquitetural

- O tempo de execução depende nas caraterísticas do sistema todo, mas onde podemos achar melhorias?

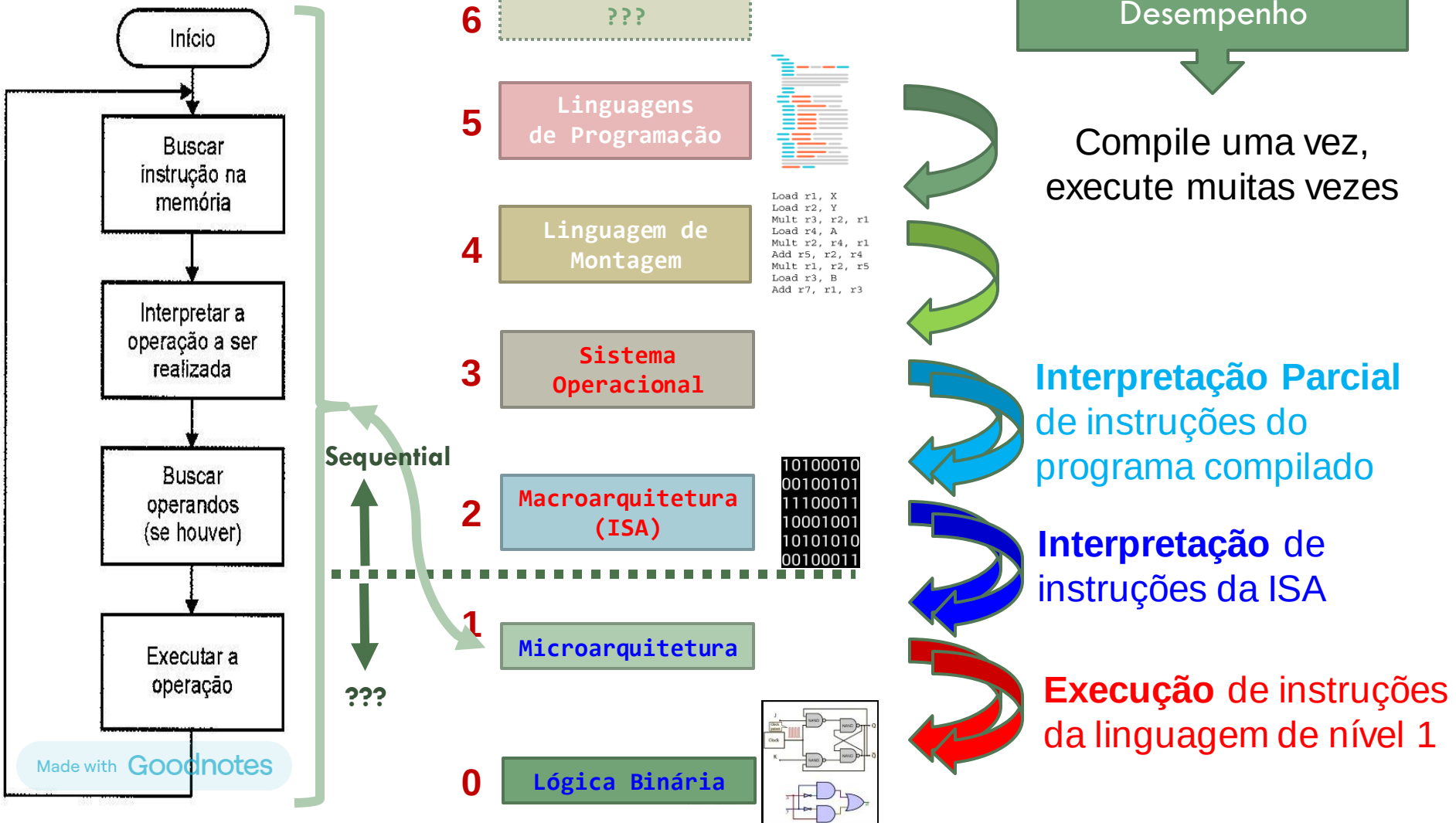


Depende no: *Compilador* ***Microarquitetura*** *Tecnologia*
Isto é na implementação adotada para cada nível

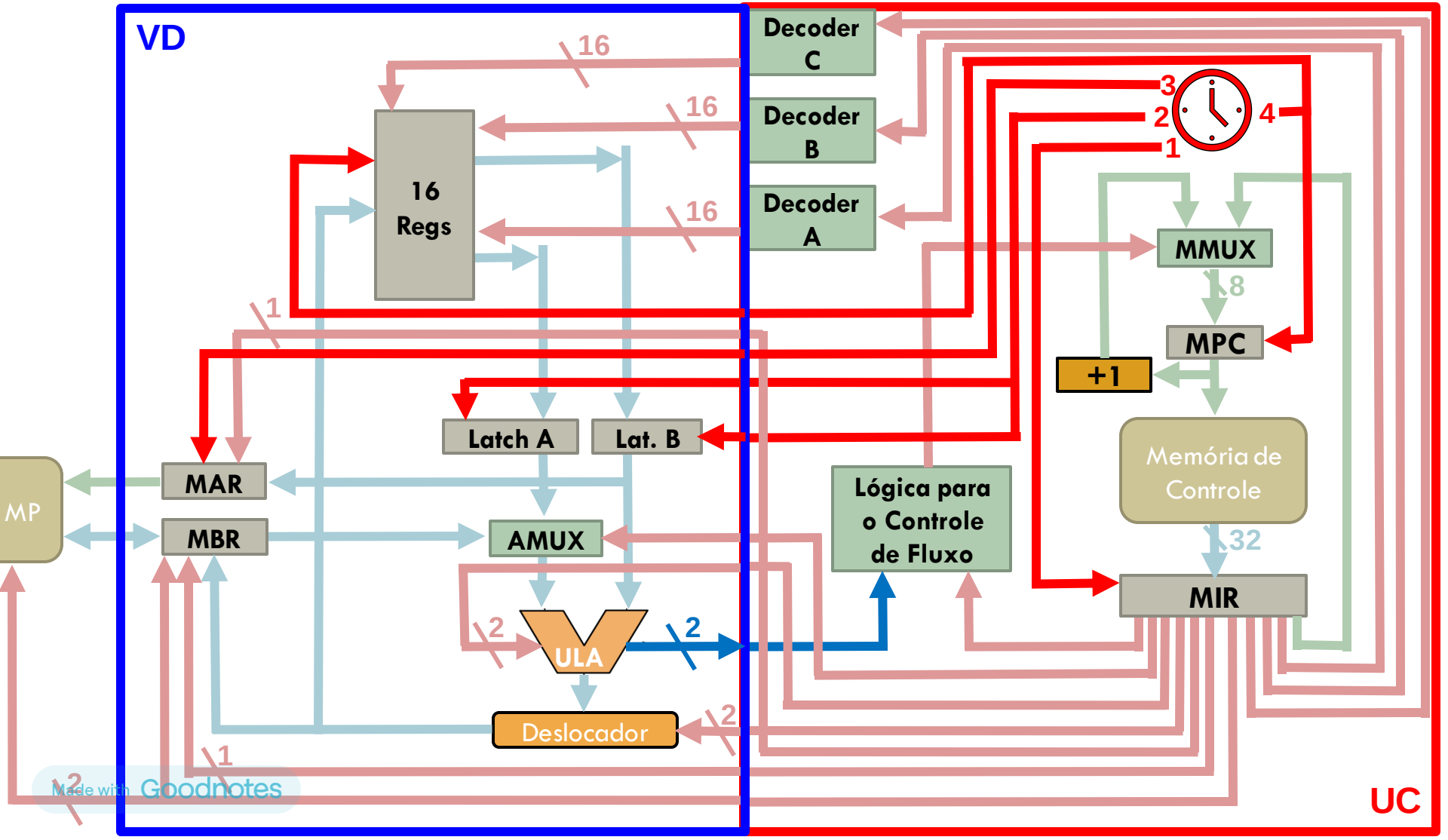
- Seria possível que algum desses elementos da equação pode chegar a ser menor que um?

Made with Goodnotes

Se não é sequencial, como fica?



O projeto do processador MIC-1



Made with Goodnotes

ARQUITETURAS DE COMPUTADORES

TURMA A1

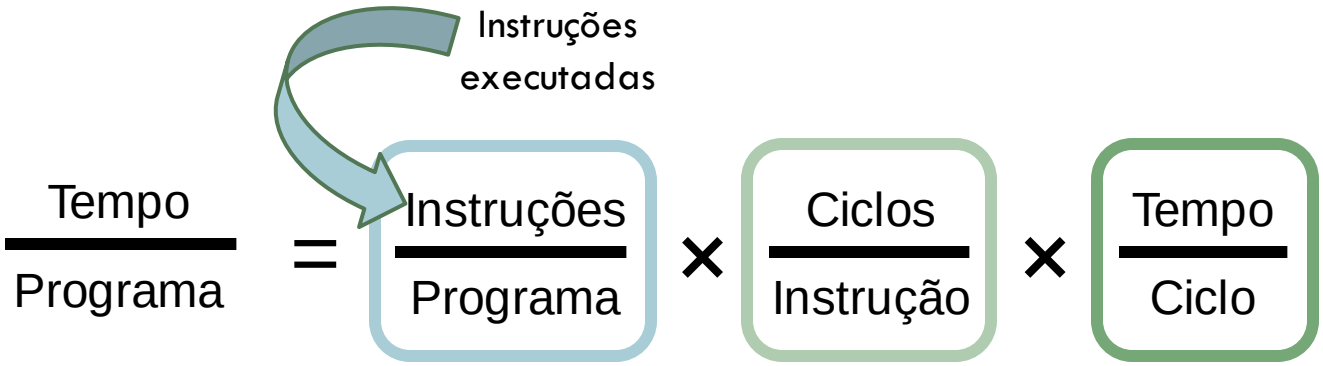
TÓPICO 3 – MEMÓRIA CACHE (AULA 20)

Made with Goodnotes

Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

Desempenho arquitetural

- O tempo de execução depende nas caraterísticas do sistema todo, mas onde podemos achar melhorias?



Depende no: *Compilador* ***Microarquitetura*** *Tecnologia*
Isto é na implementação adotada para cada nível

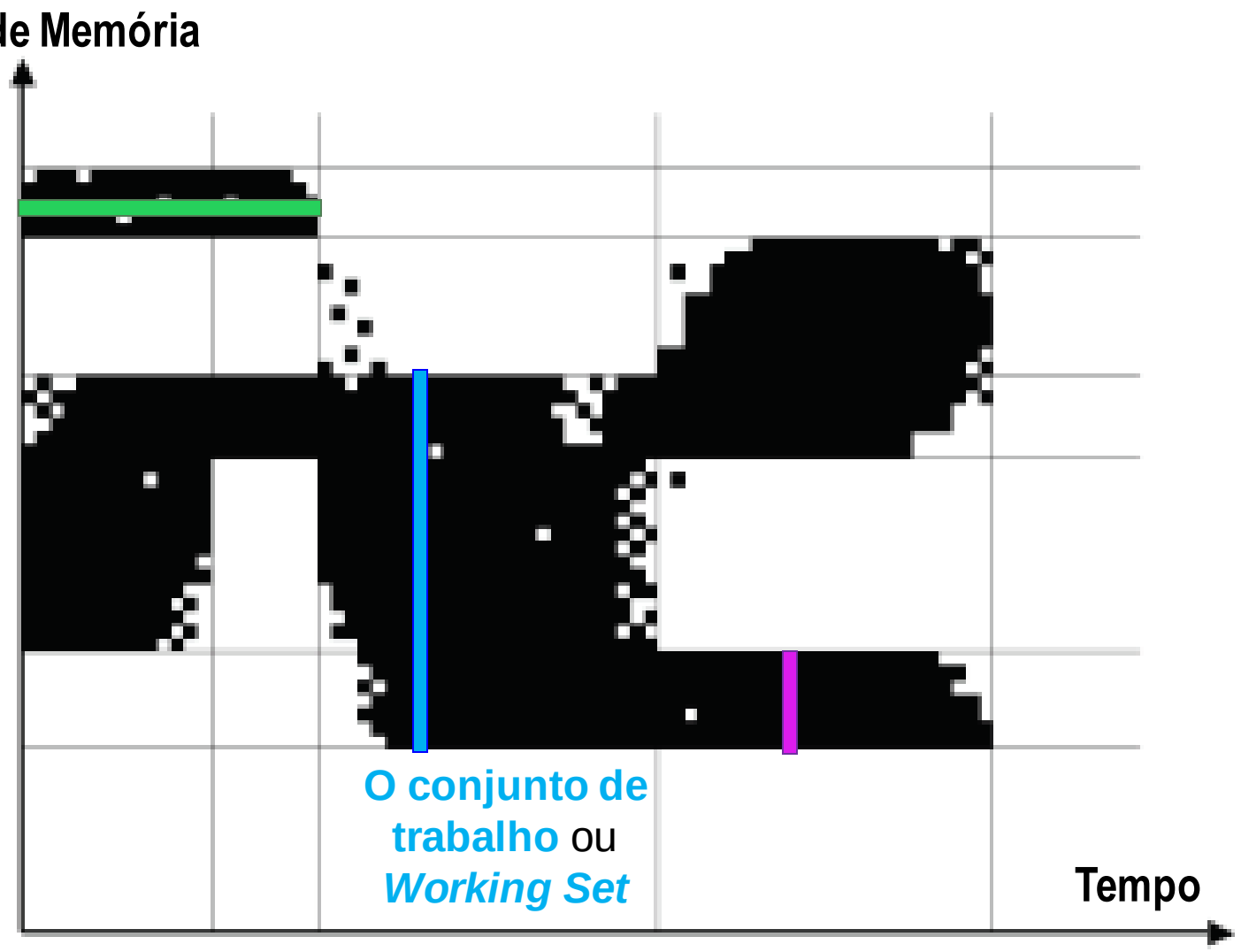
- Como melhorar o desempenho de programas Von Neumann?
  - *"Bend the rules?" = Vergar as regras?*

Made with Goodnotes

Conceito de Localidade

Localidade Temporal:
Os acessos ao mesmo endereço são normalmente agrupados no tempo

Localidade Espacial:
Quando um local é acessado, os locais próximos tendem também a ser acessados

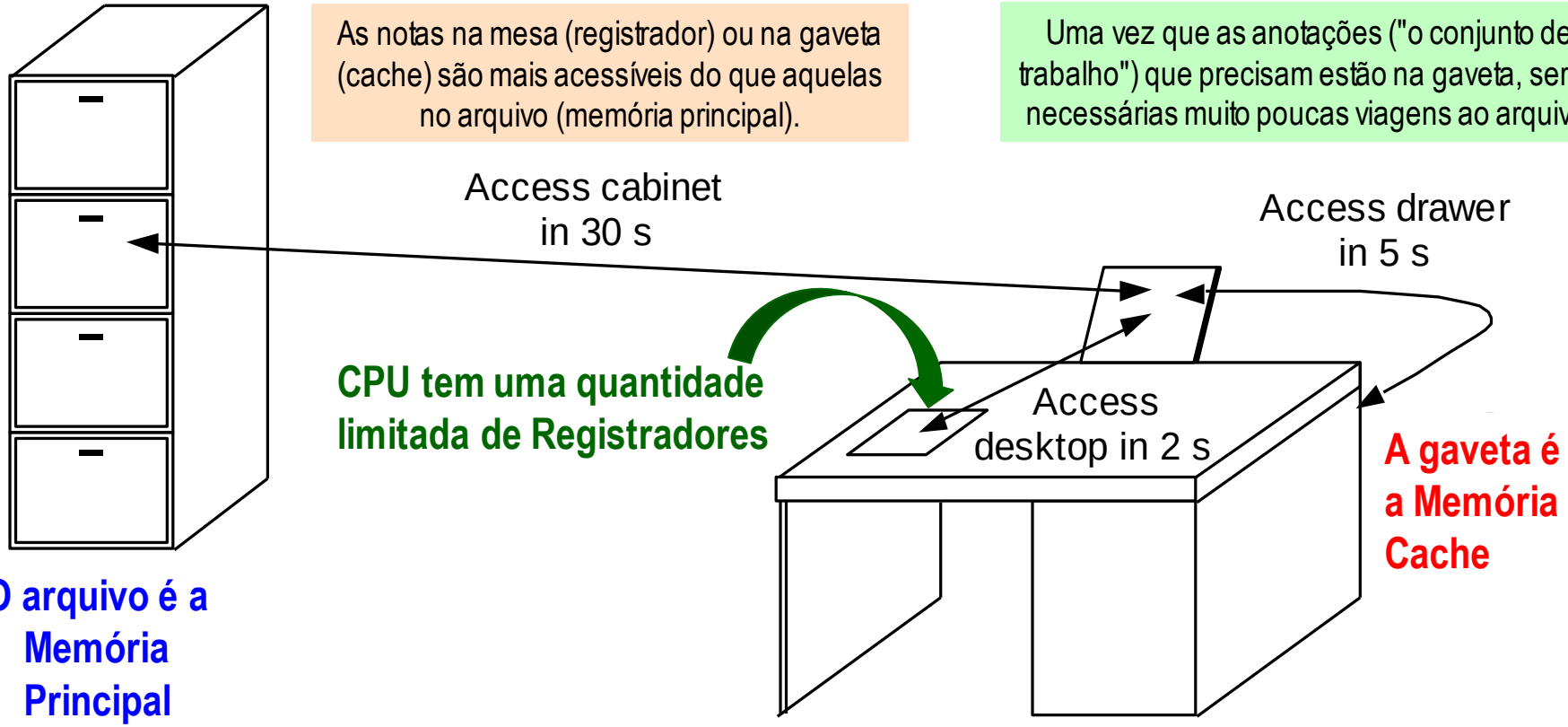


Made with Goodnotes

Memoria Cache – Uma analogia

As notas na mesa (registrador) ou na gaveta (cache) são mais acessíveis do que aquelas no arquivo (memória principal).

Uma vez que as anotações ("o conjunto de trabalho") que precisam estão na gaveta, serão necessárias muito poucas viagens ao arquivo.

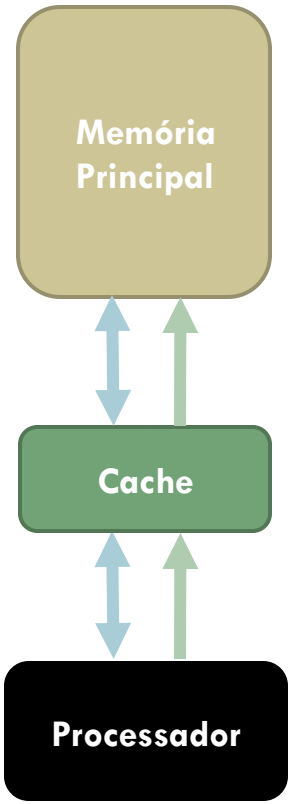


- Você está fazendo uma avaliação, onde quer colocar suas anotações:
  - Na mesa (*desktop*) que representa o CPU onde cabe uma folha por vez;
  - Na gaveta (*drawer*) que representa uma cache, e cabe algumas folhas; ou
  - No armário de arquivo (*cabinet*) que representa a memória principal?

Made with Goodnotes

Aproveitando a Localidade Temporal

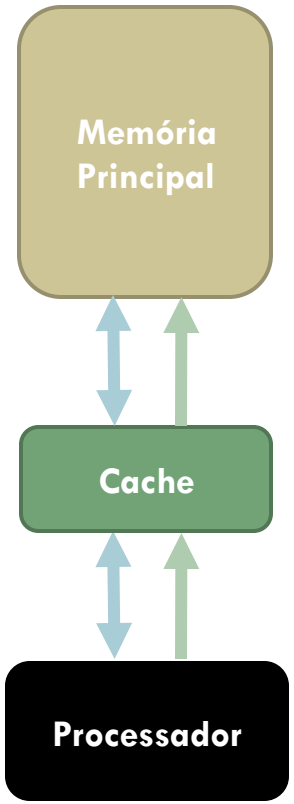
- Na primeira vez que o processador pede para ler de um endereço na memória principal, uma cópia desse dado também é armazenada na cache.
- Na próxima vez que o mesmo endereço for lido, podemos usar a cópia do dado na cache em vez de acessar a MP mais lenta.
- Portanto, a primeira leitura é um pouco mais lenta do que sem cache, pois passa pela memória principal e pelo cache, mas as leituras subsequentes são muito mais rápidas.
- Isso aproveita a localidade temporal - os dados comumente acessados são armazenados na memória cache para ser acessada mais rápidamente no futuro.



Made with Goodnotes

Aproveitando a Localidade Temporal

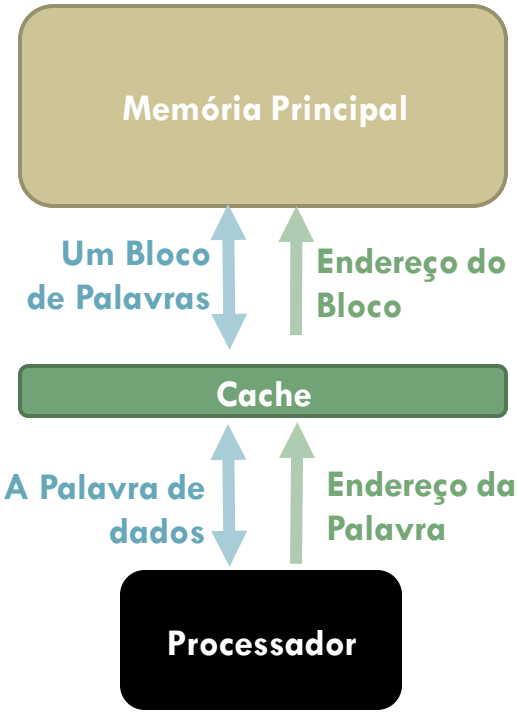
- O tempo total (TT) para busca o mesmo endereço K vezes, sem o uso de um cache
  - $TT_{MP}(K) = K \times TA_{MP}$
- O tempo para a primeira busca usando um cache
  - $TT_C(1) = TA_C + TA_{MP}$
- Como a cache fica com uma cópia do dado recente buscado, requisições subsequentes vão só precisa acessar a cache:
  - $TT_C(K) = TT_C(1) + TT_C(K\text{-}1) = TA_C + TA_{MP} + (K - 1)TA_C$
- Então, qual é o benefício de usar uma cache?
  - *Speedup* = $TT_{MP}(K) \div TT_C(K)$
  - $= (K \times TA_{MP}) \div (TA_{MP} + K \times TA_C)$
  - $= TA_{MP} \div TA_C$ para $K \longrightarrow \infty$
  - Se $TA_{MP} = 100 \times TA_C$, o *Speedup* poderia chegar quase 100



Made with Goodnotes

Aproveitando a Localidade Espacial

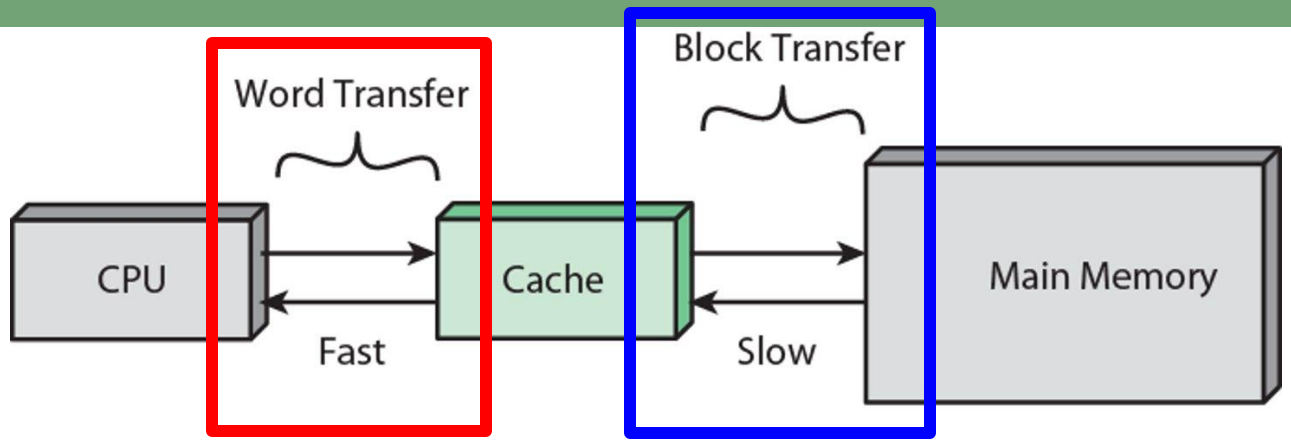
- Quando a CPU lê um endereço, uma cópia da palavra de dados é colocada na cache.
- Mas, em vez de apenas copiar só uma palavra, as palavras de endereços vizinhos na MP também são copiadas juntas para o cache de uma vez só
- Se a CPU posteriormente precisar ler destes dados pela primeira vez, serão acessados da cache e não da MP.
  - Por exemplo, em vez de ler apenas um elemento do array por vez, o cache pode carrega, por exemplo, quatro elementos do array de uma vez.
- Novamente, a carga inicial incorre em uma penalidade de desempenho, mas estamos apostando na localidade espacial e na chance de que a CPU vai precisar dos dados buscados.



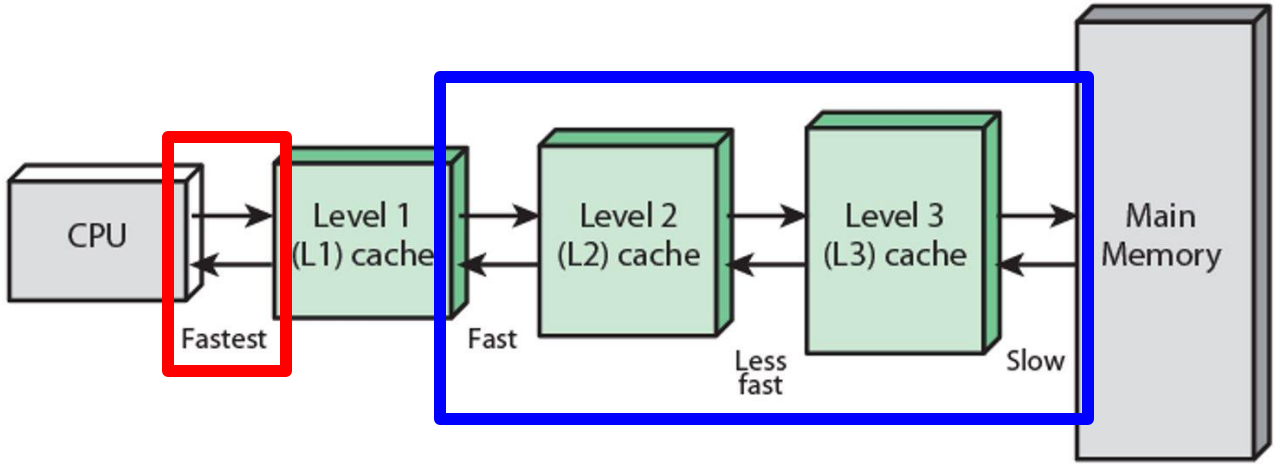
Made with Goodnotes

Funcionamento da Memória Cache

- **Uma palavra** de dados é transferida entre o CPU e a cache
- **Um bloco de palavras** é transferido entre os níveis da cache e da memória



(a) Single cache

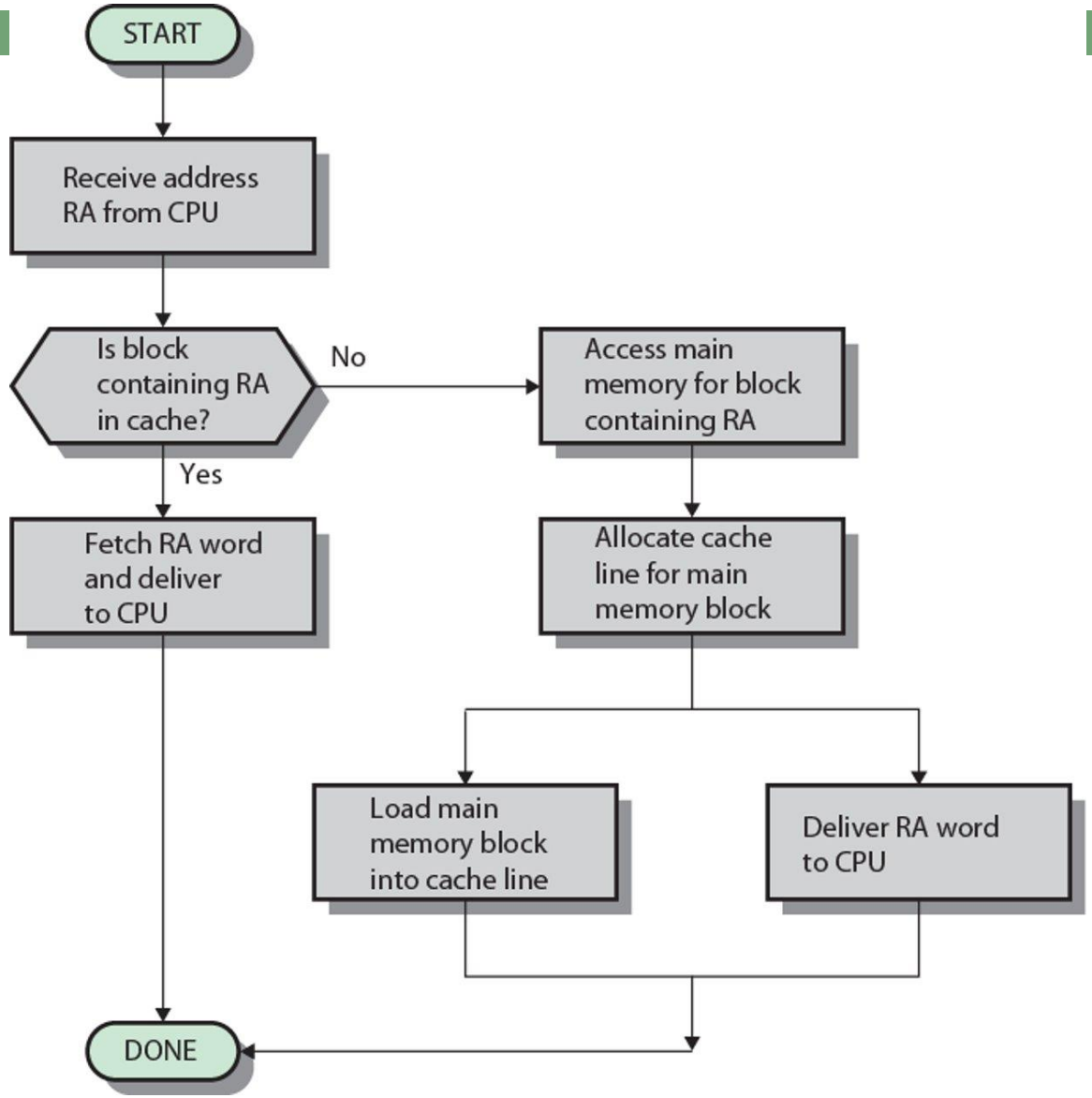


(b) Three-level cache organization

Made with Goodnotes

Funcionamento da Memória Cache

- RA = endereço da **palavra** solicitada
- CPU envia um endereço (RA) para o que ele acha é a memória principal
- O CPU não vai saber se o dado veio do(s) cache(s) ou a MP



Made with Goodnotes

Aproveitando a Localidade Espacial

Soma de Matrizes

```
for(x=0; x<n; x++)
    for(y=0; y<n; y++)
        c[x][y] =  a[x][y] + b[x][y];
```

Trocar a ordem dos dois laços **x** e **y** faria alguma diferença?

Made with Goodnotes

Aproveitando a Localidade Espacial

Soma de Matrizes

```
for(x=0; x<n; x++)
    for(y=0; y<n; y++)
        c[x][y] =  a[x][y] + b[x][y];
```

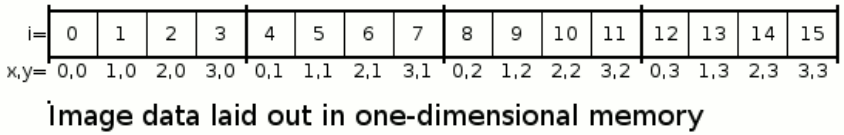


Image data laid out in one-dimensional memory

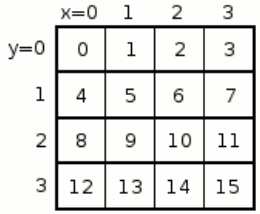
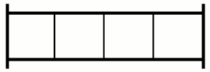
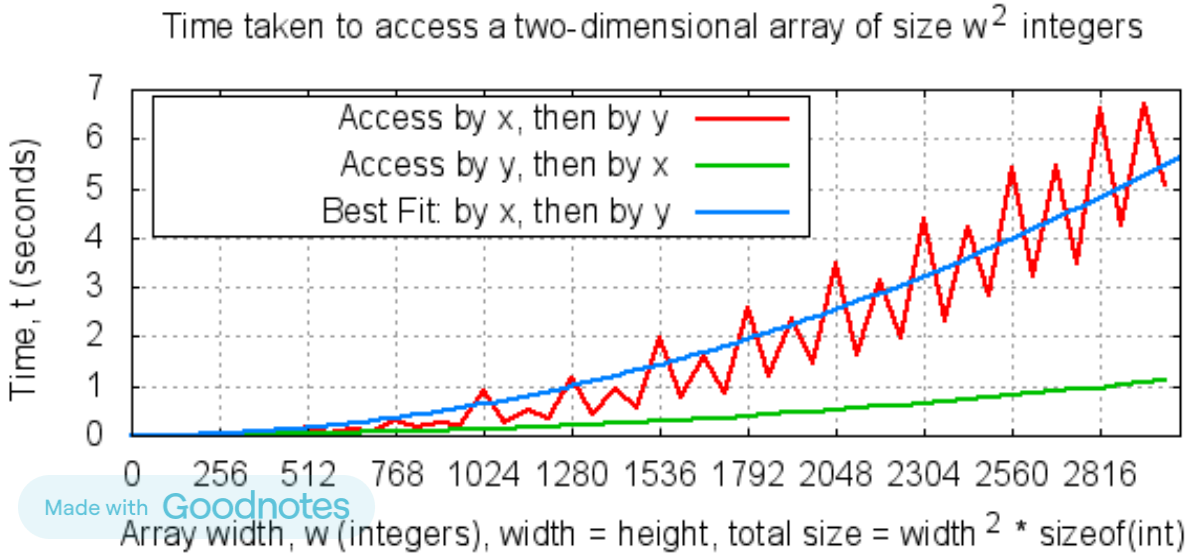


Image data laid out in two dimensions



One byte of pixel data


One row of pixels



Made with Goodnotes

Onde achamos caches?

- A ideia geral por trás das caches é usada em muitas outras situações onde têm comunicação de dados. As redes de comunicação são provavelmente o melhor exemplo.
  - As redes que têm “latência” relativamente alta e “largura de banda” baixa, portanto, transferências de dados repetidas são indesejáveis.
  - Navegadores (*Web browsers*) armazenam suas páginas da Web acessadas mais recentemente em seu disco rígido.
  - Os administradores podem configurar um cache para toda a rede internet, e empresas como a *Akamai* também fornecem serviços de cache.

Made with Goodnotes

Onde achamos caches?

- Alguns outros exemplos:
  - Muitos processadores têm um TLB (*Translation Lookaside Buffer*), que é um cache dedicado ao suporte de memória virtual;
  - Os sistemas operacionais podem armazenar blocos de disco acessados com frequência, como diretórios, na memória principal ... e esses dados podem então ser armazenados no cache da CPU!
  - A técnica de *Memoization* é usado principalmente para acelerar programas, armazenando os resultados de chamadas de funções caras e retornando o resultado previamente colocado na cache quando as mesmas entradas ocorrem novamente.

Made with Goodnotes

Caches: acertos e falhas

- Um acerto de cache (***cache hit***) ocorre se o cache contém o dado que estamos procurando.
  - Os acertos são bons, porque o cache pode retornar o dado muito mais rápido do que a memória principal.
- Ocorre um falha de cache (***cache miss***) se o cache não contém os dados solicitados.
  - Isso é ruim, já que a CPU deve então esperar pela memória principal mais lenta.
- Existem duas medidas básicas de desempenho do cache.
  - A taxa de acertos (***hit rate***) é a porcentagem de acessos à memória que são tratados pelo cache.
  - A taxa de falha (***miss rate* = 1 – *hit rate***) é a porcentagem de acessos que devem ser tratados pela memória principal
- Dado que caches típicamente têm uma taxa de acerto de 95% ou mais, qual é o tempo efetivo de acesso à memoria principal?

Made with Goodnotes

Caches: acertos e falhas

- Um acerto de cache (***cache hit***) ocorre se a cache contém o dado que estamos procurando.
  - Os acertos são bons, porque o cache pode retornar o dado muito mais rápido do que a memória principal.
- Ocorre um falha de cache (***cache miss***) se a cache não contém os dados solicitados.
  - Isso é ruim, já que a CPU deve então esperar pela memória principal mais lenta.
- Existem duas medidas básicas de desempenho da cache.
  - A taxa de acertos (***hit rate***) é a porcentagem de acessos à memória que são tratados pela cache.
  - A taxa de falha (***miss rate*** = **1 – *hit rate***) é a porcentagem de acessos que devem ser tratados pela memória principal
- *Tempo de acesso (TA) médio* = $TA_{cache} + (1 - \textit{hit rate}) \times TA_{memória}$

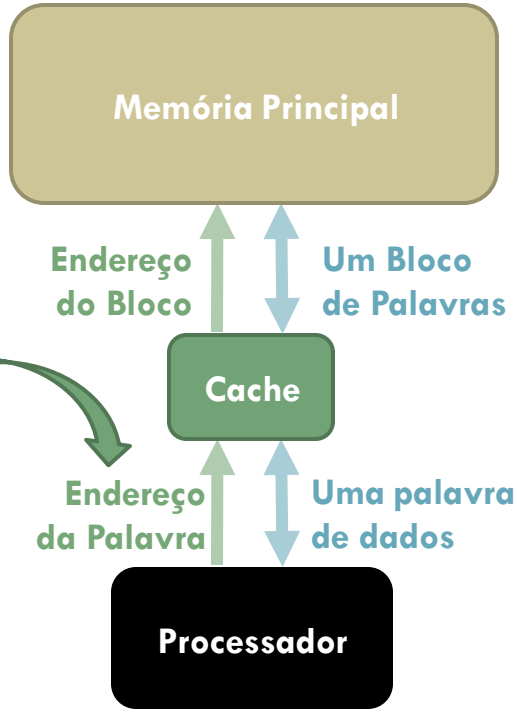
Made with Goodnotes

Acessando Memória Principal

- O processador continua achando que ele está acessando a memória principal, enviando um endereço (com tamanho *M* bits) da palavra solicitada

M bits do endereço da palavra

- Porém a cache, para explorar localidade espacial, precisa acessar um conjunto de palavras – isto é um bloco.



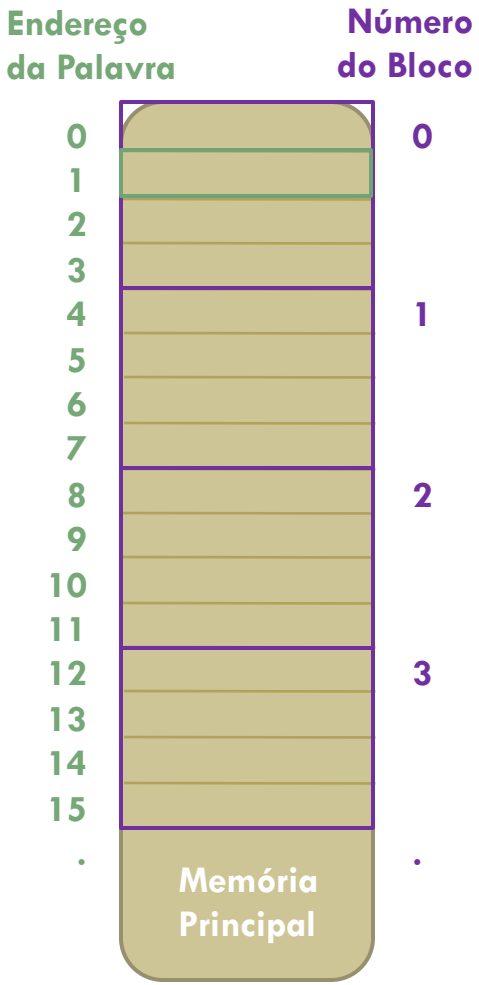
Made with Goodnotes

Acessando Memória Principal

- O processador continua achando que ele está acessando a memória principal, enviando um endereço (com tamanho de *M* bits) da palavra solicitada

M bits do endereço da palavra

- Porém a cache, para explorar localidade espacial, precisa acessar um conjunto de palavras – isto é um bloco.
- Então a MP é dividida em blocos de 2^P palavras de endereços consecutivos
  - **Endereço *i*** fica no **Bloco ($i \div 2^P$)**



Com P = 2

Made with Goodnotes

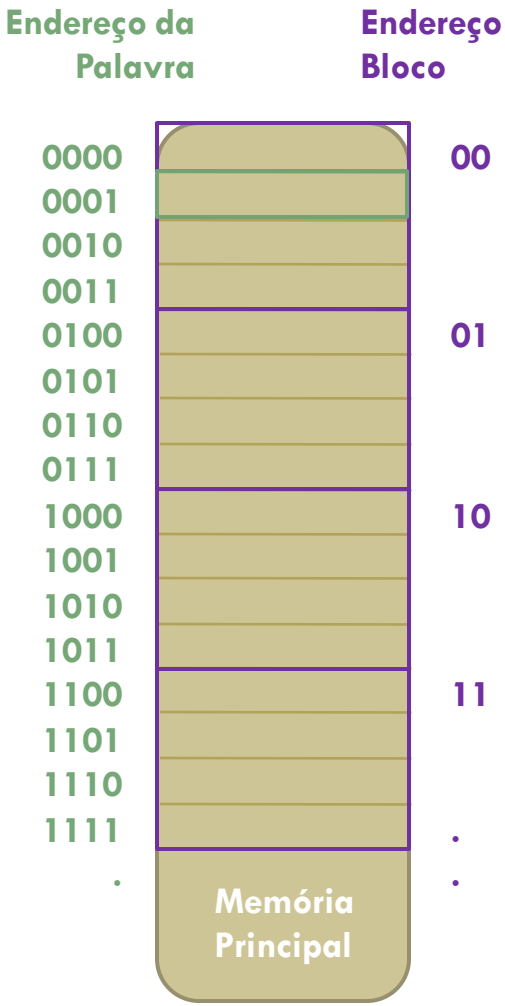
Acessando Memória Principal

- O processador continua achando que ele está acessando a memória principal, enviando um endereço da palavra solicitada

M bits do endereço da palavra

- Porém a cache, para explorar localidade espacial, precisa acessar um conjunto de palavras – um bloco.
- Então a MP é dividida em blocos de 2^P palavras de endereços consecutivos

M - P bits	*P* bits
Endereço do bloco	*Posição da palavra no bloco*



Made with Goodnotes

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 3 – MEMÓRIA CACHE (AULA 21)

Made with Goodnotes

Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

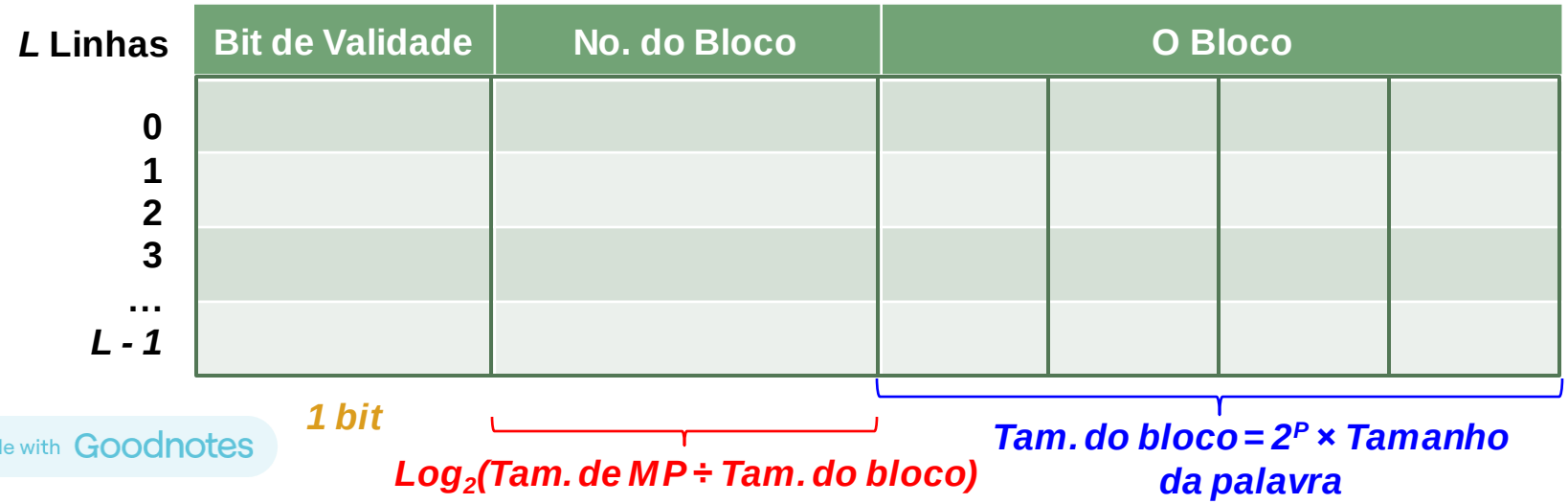
Duas organizações de Cache

- As caches têm uma estrutura diferente de memória principal
  - Além de um bloco de memória, cada entrada ou **linha da cache** precisa guardar alguma informação que identifica qual bloco está sendo armazenado nesta linha
- Existe duas principais organizações (mais uma que é a combinação das duas)
  - Cache associativa
  - Cache com mapeamento direto
  - Cache associativa por conjunto (um híbrido da duas anteriores)
- A diferença entre as duas caches é relacionada à maneira em que como cada uma resolva as seguintes questões:
  - Quando copiamos um bloco de dados da memória principal para a cache, onde exatamente devemos colocá-lo?
  - Como podemos saber se uma palavra já está na cache ou se seu bloco deve ser buscada na memória principal?

Made with Goodnotes

Cache Associativa

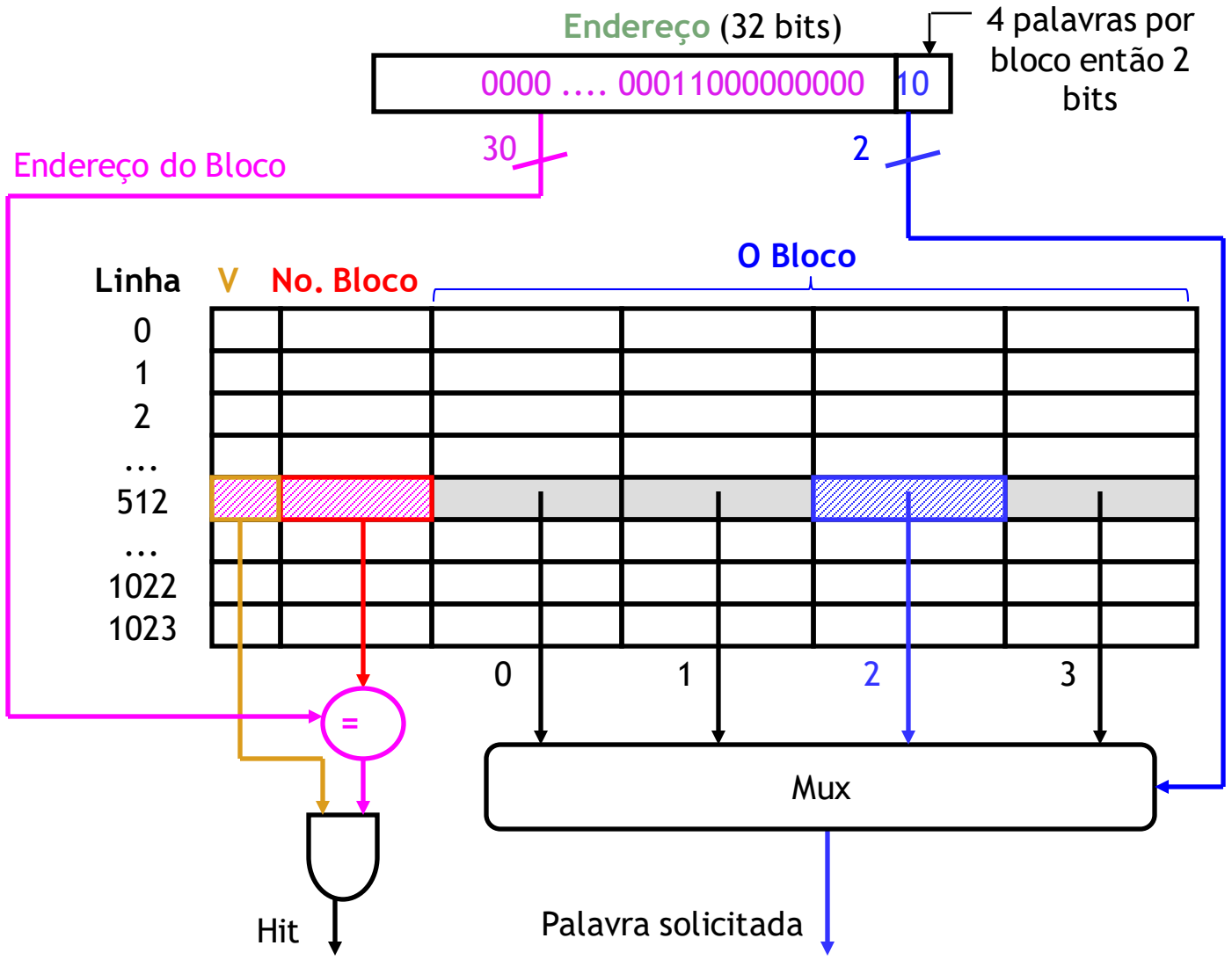
- Neste tipo de cache, um bloco pode ser carregada em qualquer linha (disponível) da cache
- Cada linha tem três campos: um bit de validade que identifica se a linha está guardando um bloco, o número do bloco e o bloco
  - Custo do cache é relacionado seu tamanho =
    - $L \times (1 + log_2(\text{Quantidade de blocos na MP}) + \text{Tamanho do bloco})$ bits



Made with Goodnotes

Cache Associativa

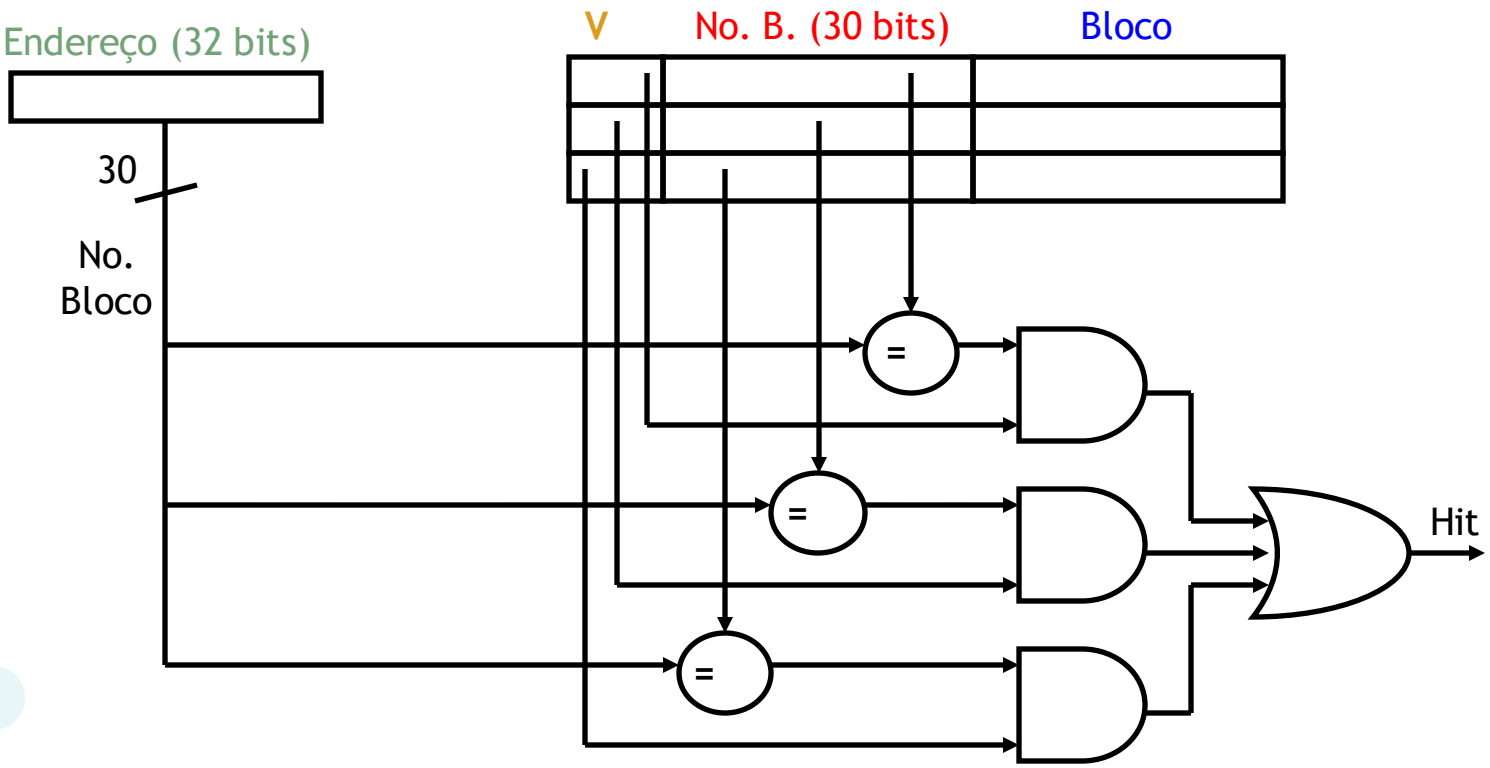
- CPU envia um endereço de onde a cache determina o número do bloco que contém a palavra solicitada
- A cache ainda tem que determinar se o bloco está em alguma linha da cache



Made with Goodnotes

Busca em paralela da Cache Associativa

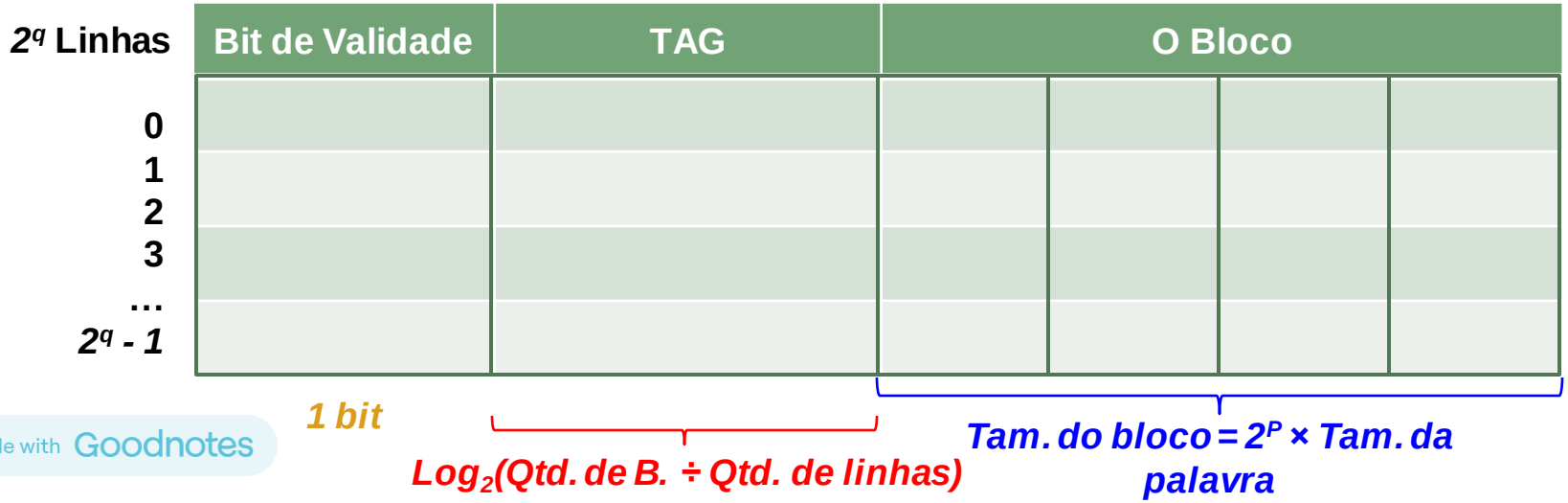
- Nesta cache um bloco pode ser carregada em qualquer linha de cache
  - Então, para achar um dado bloco, vai precisar buscar no cache inteiro
    - Para não demorar, é feita em paralelo, buscando em todas as linhas no mesmo tempo
    - Isso pode ser bastante caro, dependendo na quantidade de linhas (L) do cache



Made with Goodnotes

Cache com Mapeamento Direto

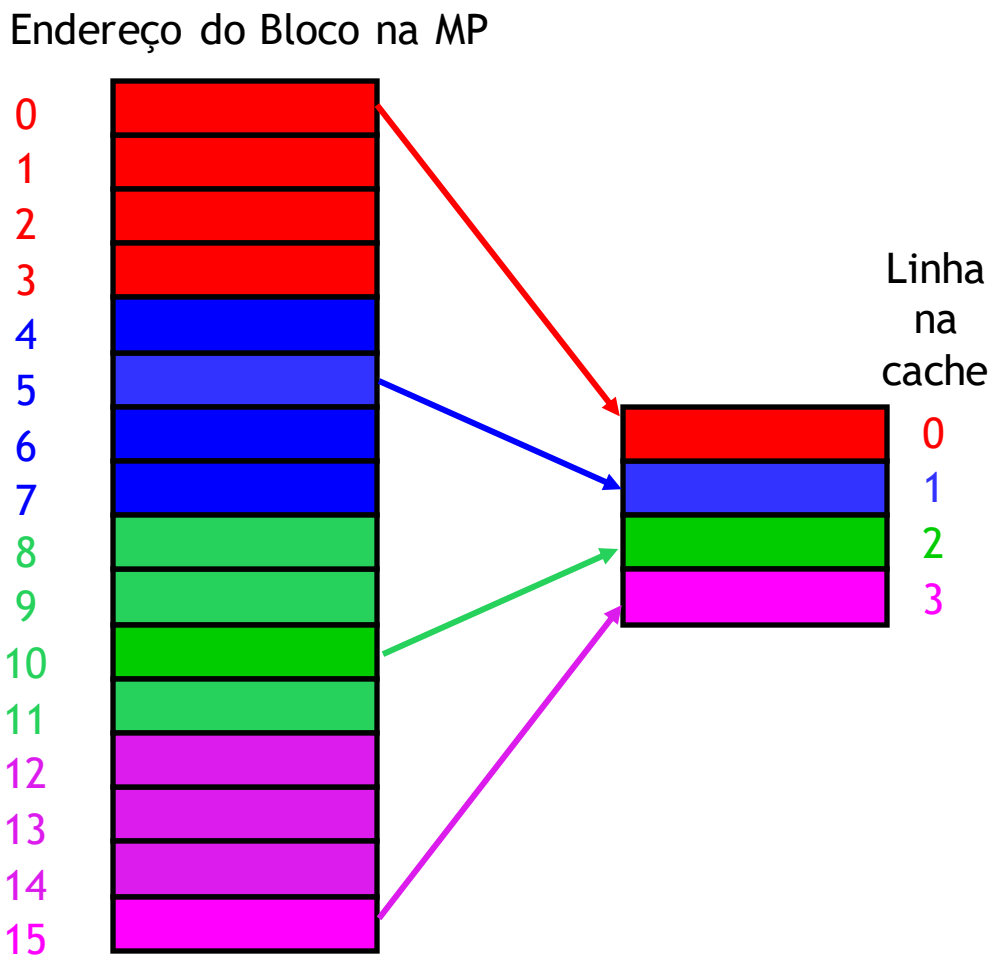
- Neste tipo de cache, cada bloco da memória principal tem **um único lugar (uma linha fixa)** na memória cache
  - Cada linha tem três campos: o bit de validade que identifica se a linha está guardando um bloco, um identificador do bloco (TAG) e o bloco
  - Tamanho da cache = $2^q \times (1 + Log_2(Qtd\ de\ ??) + Tamanho\ do\ bloco)$



Made with Goodnotes

Cache com Mapeamento Direto

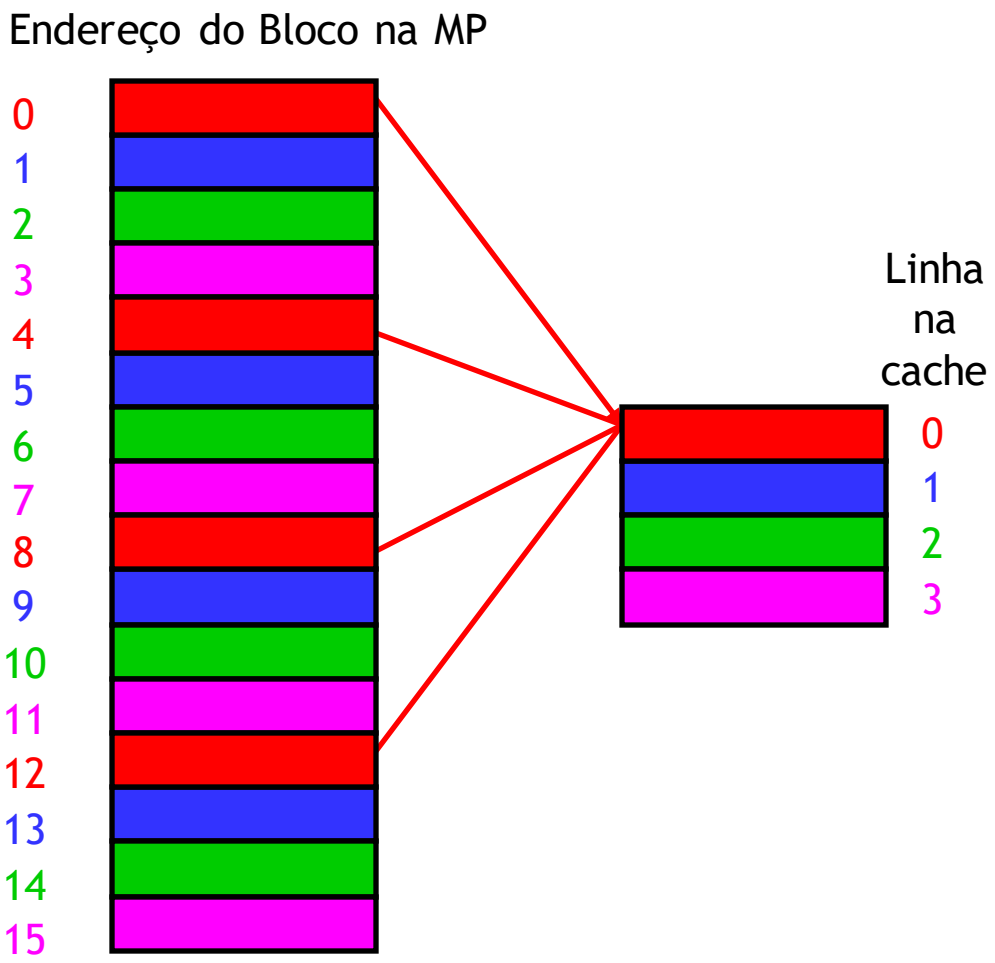
- Cada bloco tem um único lugar na memória cache, mas onde?
- Por exemplo, à direita está uma memória principal de 16 blocos e uma cache de 4 linhas.
  - Os locais de memória 0 a 3 são mapeados para a linha 0; os endereços 4 a 7 mapeiam para a linha 1, etc.



Made with Goodnotes

Cache com Mapeamento Direto

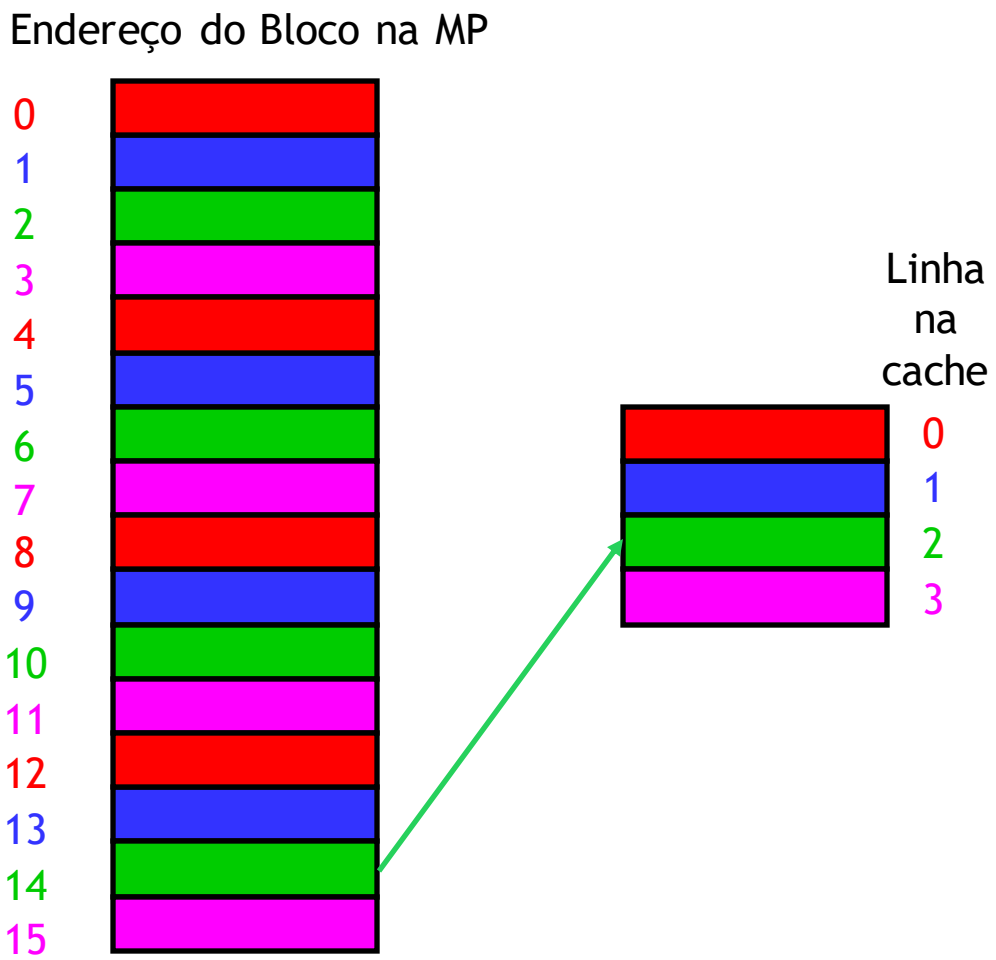
- Com mesma MP de 16 blocos e um cache de 4 linhas, um outro exemplo de alocação pode ser:
  - Os locais de memória 0, 4, 8 e 12 são mapeados para a linha 0; Os endereços 1, 5, 9 e 13 mapeiam para a linha 1, etc.
- Qual é melhor?
  - Lembra de **Localidade**!
- Como podemos calcular este mapeamento?



Made with Slidor.es

Tudo feito com a operação divisão?

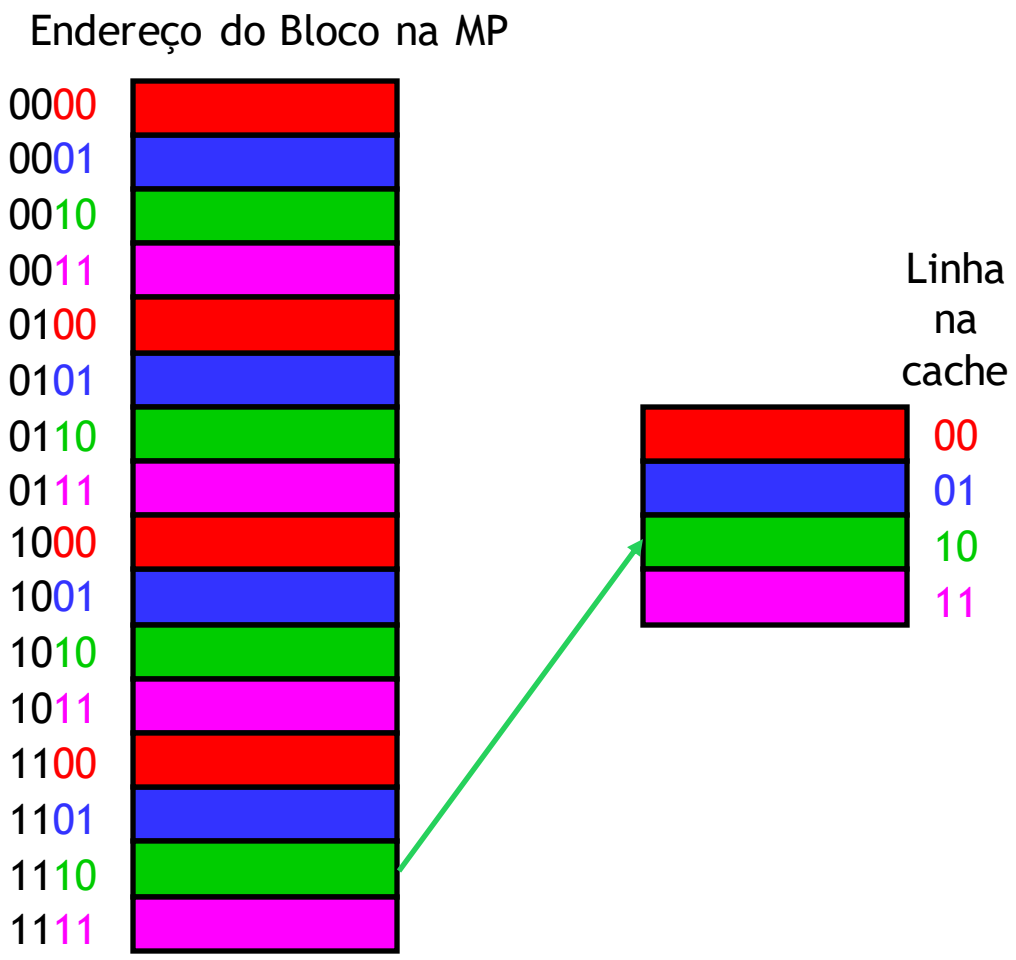
- Uma maneira de descobrir para qual linha da cache um bloco de memória específico deveria ser colocado é usar o operador **mod** (resto).
- Se o cache contém 2^q linhas, então bloco *i* vai para
  - A linha ***i* mod 2^q**
  - Por exemplo, com o cache de quatro linhas aqui, o endereço 14 seria mapeado para a linha 2 do cache deste que
    - 14 mod 4 = 2



Made with Goodnotes

Mas divisões são operações caras ...

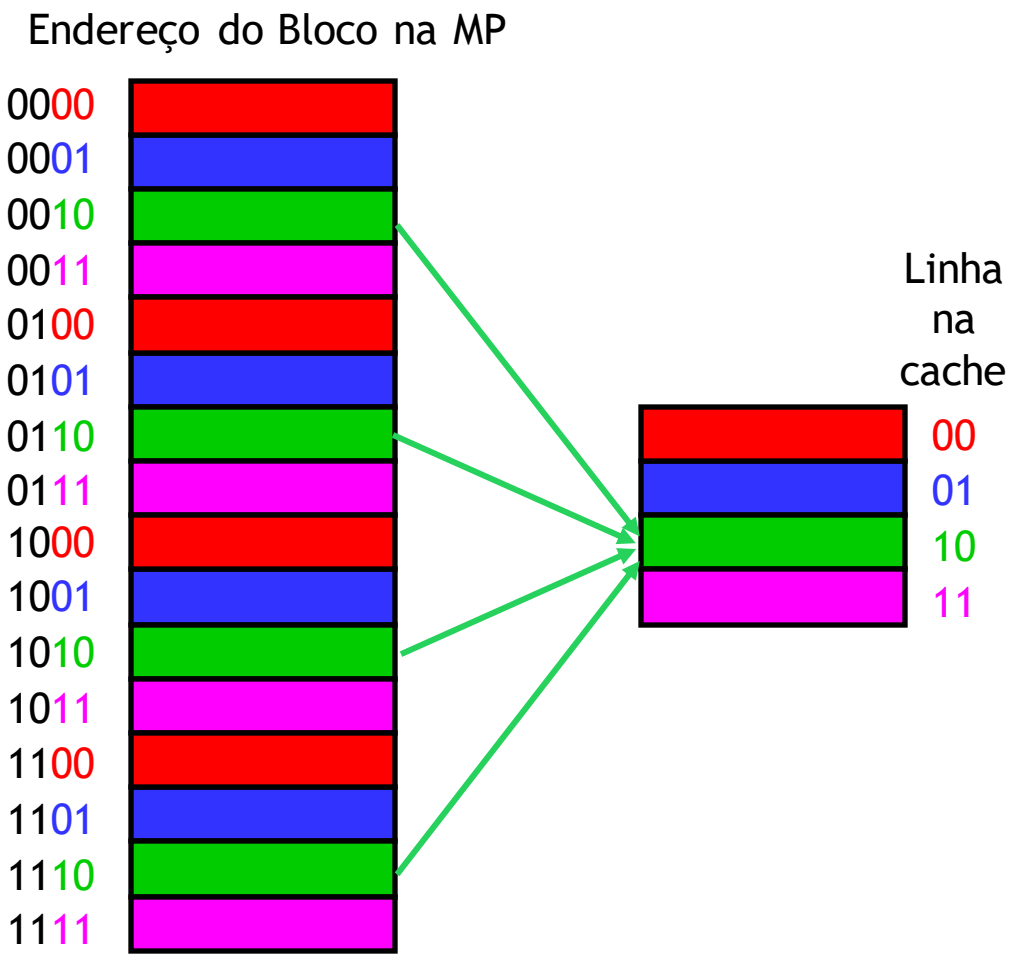
- Uma maneira equivalente de encontrar a localização de um endereço de memória no cache é examinar os *q* bits menos significativos do endereço.
- Com nosso cache de quatro linhas, inspecionaríamos os dois bits menos significativos dos endereços de bloco.
- Novamente, você pode ver que o endereço 14 (1110 em binário) mapeia para a linha 2 (10 em binário).
- Pegar últimos *q* bits de um valor binário é a mesma coisa de
- calcular **mod** 2^q do valor.
- Qual seria o custo desta operação?
  - Porque tempo importa?



Made with Goodnotes

Como isso ajuda a encontrar os dados?

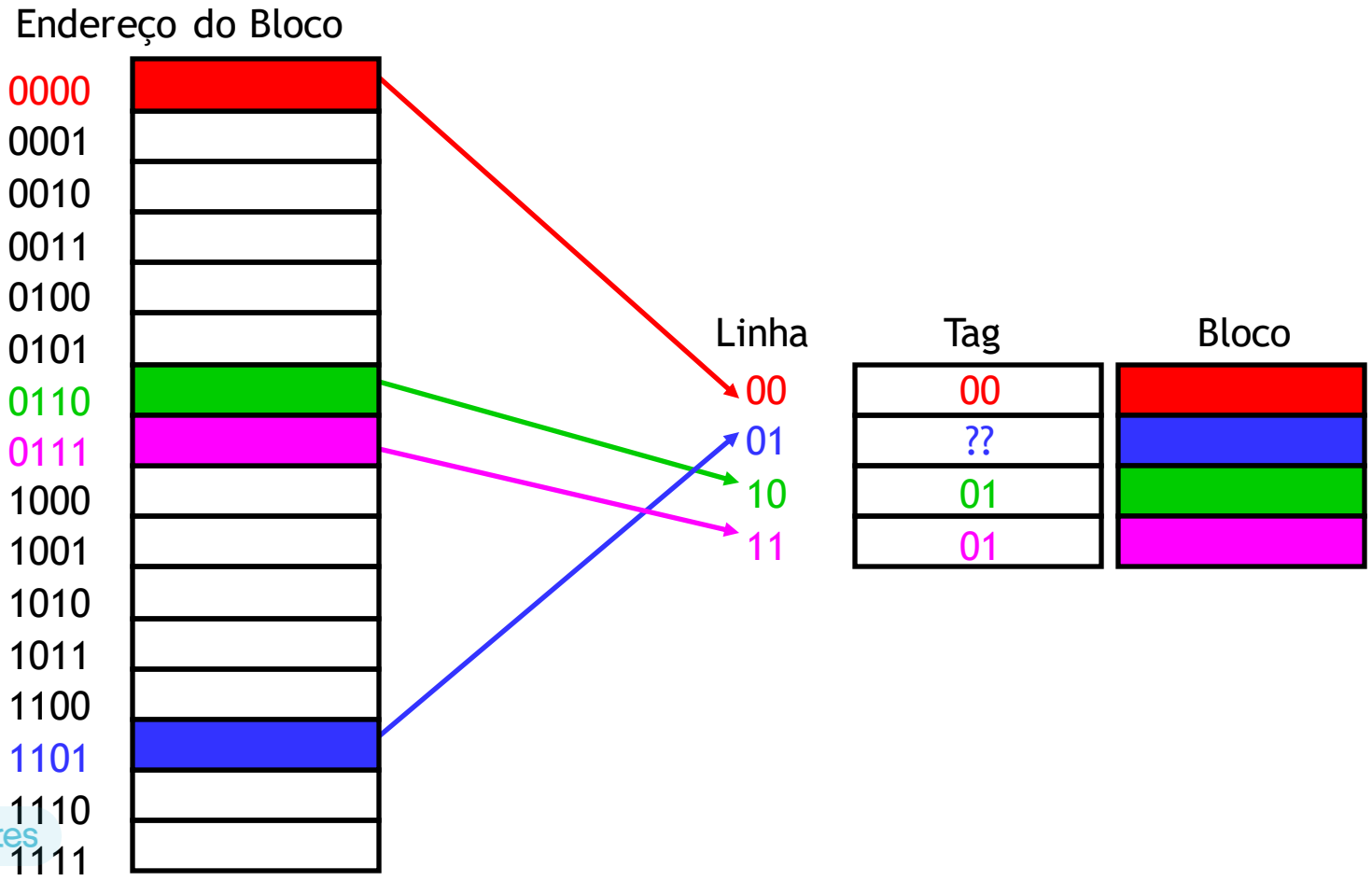
- A segunda pergunta era como determinar se o dado solicitado já está armazenado na cache.
- Se quisermos achar o bloco *i*, podemos usar o truque do **mod** para determinar em qual linha da cache poderia conter *i*.
- Mas lembra que outros endereços podem também mapeiar para a mesma linha. Como podemos distinguir entre estes blocos?
  - Por exemplo, a linha 2 da cache pode conter os blocos 2, 6, 10 ou 14.



Made with Goodnotes

TAG x Endereço do bloco

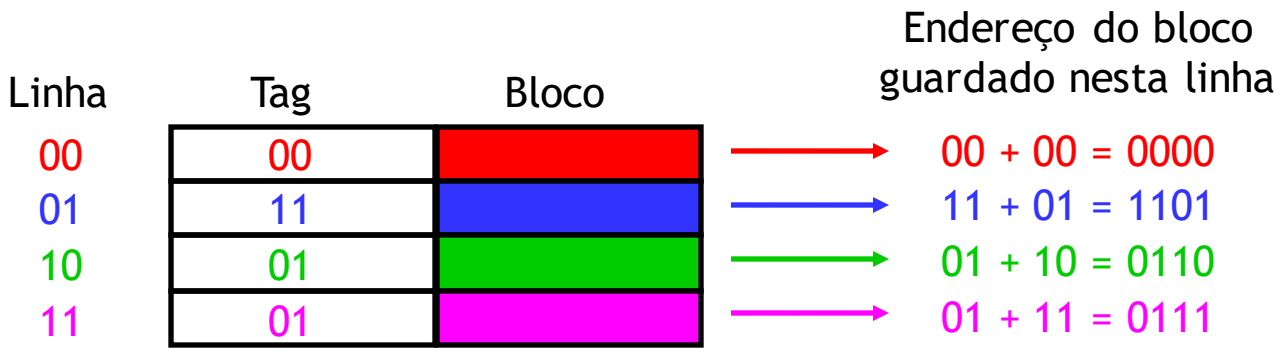
- Usamos **Tags** na linha da cache, que fornecem os bits necessarios para distinguir entre diferentes blocos que mapeiam para a mesma linha do cache.

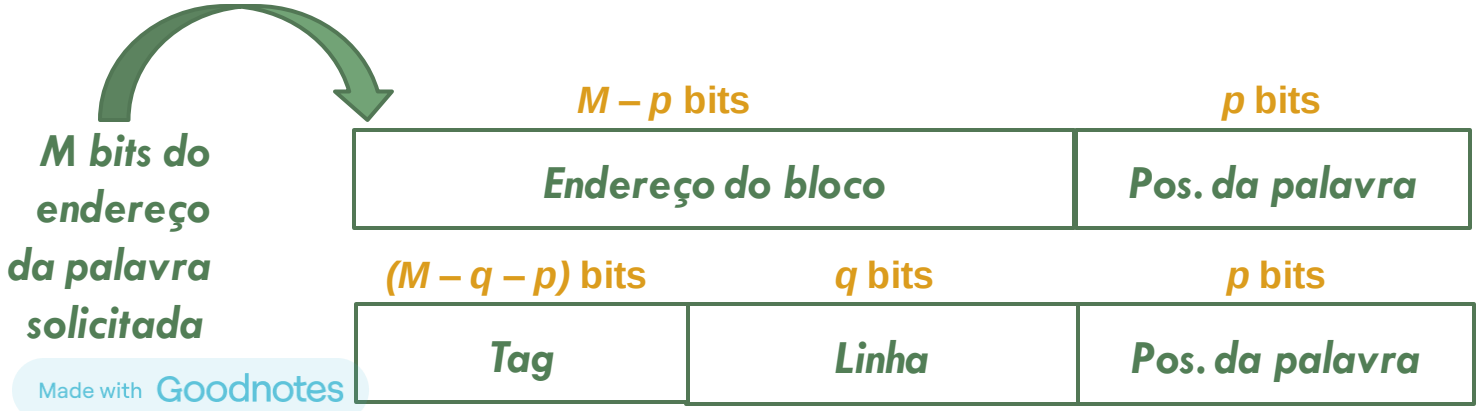


Made with Goodnotes

Tag x Endereço do bloco

- Em vez de armazenar todo o endereço do bloco, na cache com mapeamento direto, podemos descobrir qual bloco da MP está armazenado naquela linha, concatenando o *Tag* do bloco com o número da linha

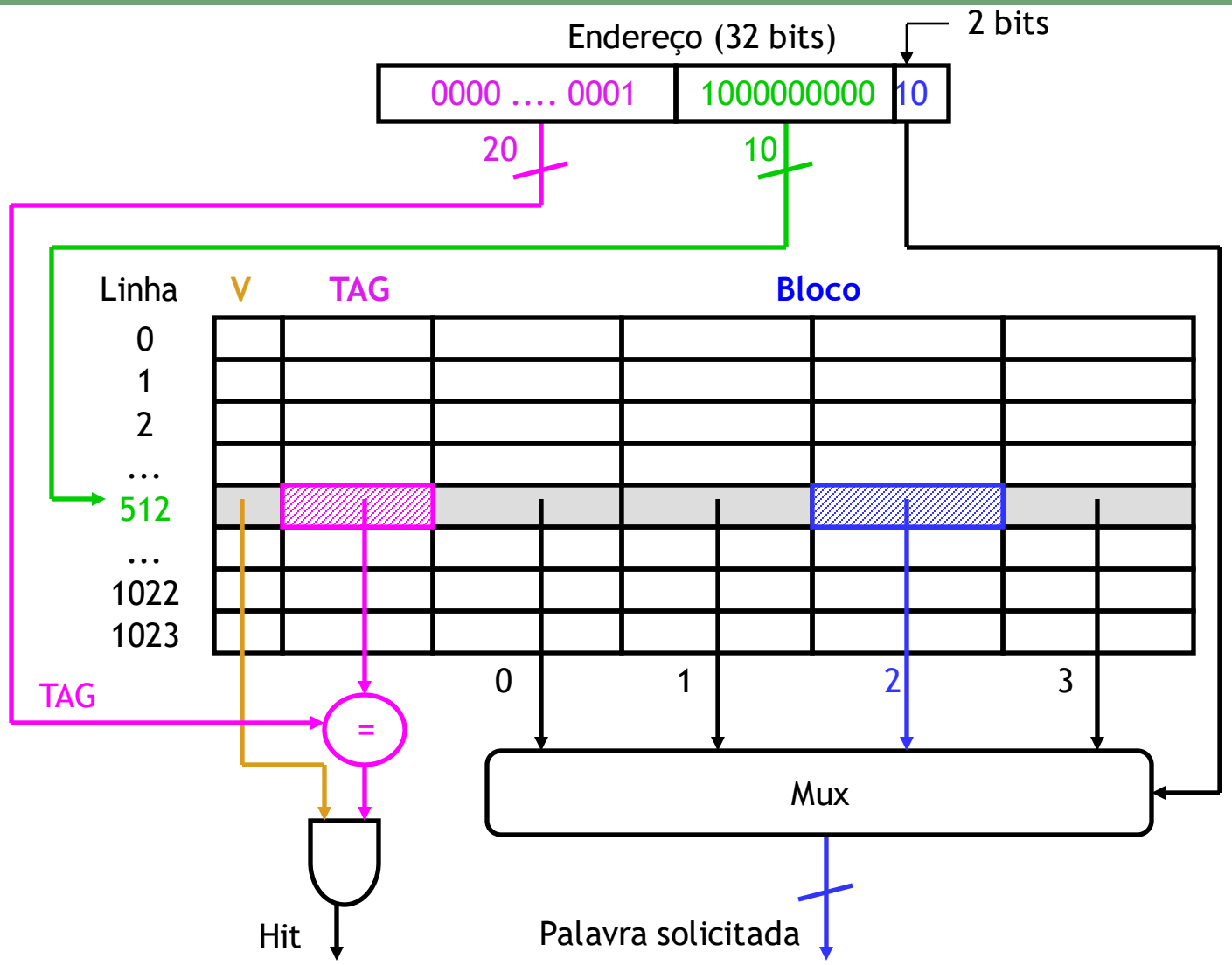




Made with Goodnotes

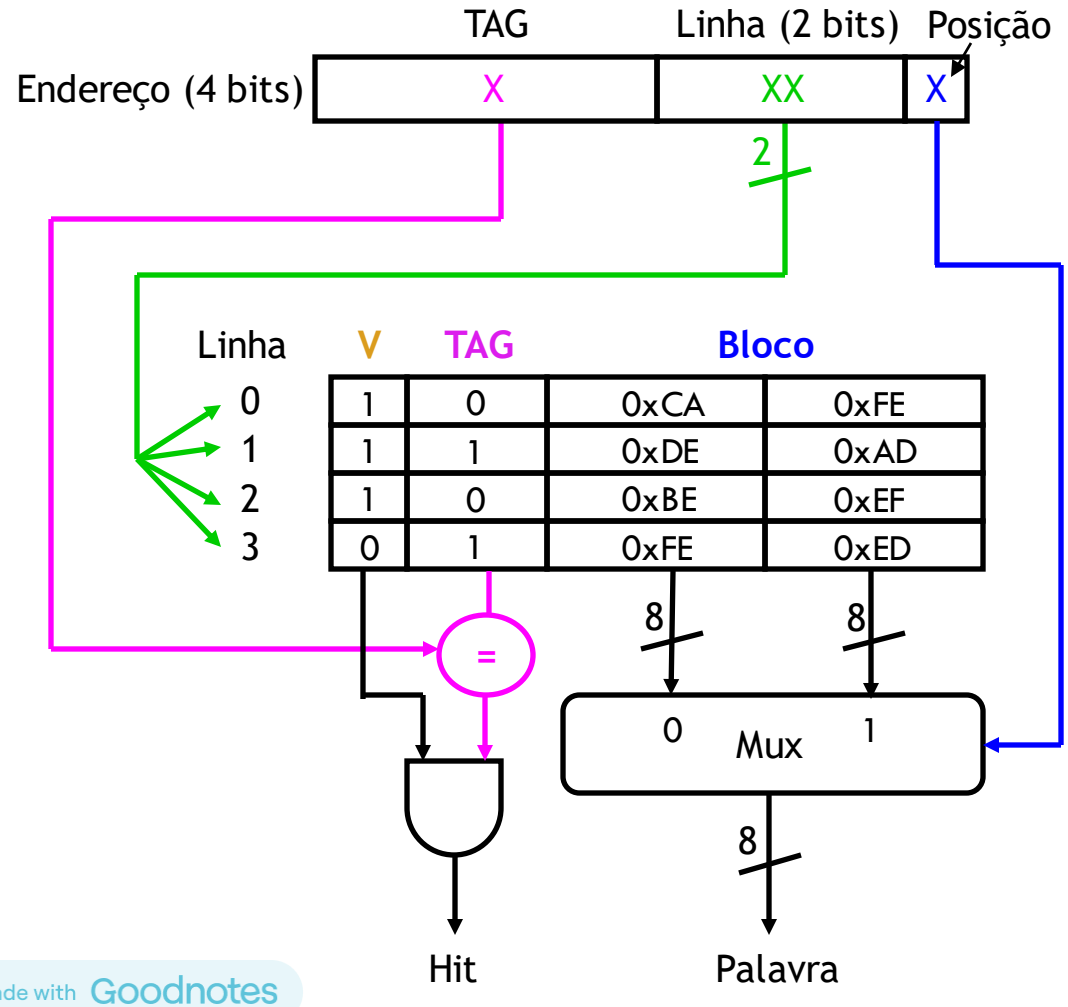
Cache com mapeamento direto

- CPU envia um endereço com que o cache determina o número do bloco que contém a palavra solicitada
- Procura na única linha que pode receber este bloco comparando o TAG do endereço com o TAG guardado



Made with Goodnotes

Um outro exemplo – Exercício



- Para cada endereço em seguir, que este cache vai retornar?
  - Será que uma palavra é lido da cache ou vamos ter um *cache miss*?
  - 1010
  - 1110
  - 0001
  - 1101

Made with Goodnotes

ARQUITETURAS DE COMPUTADORES

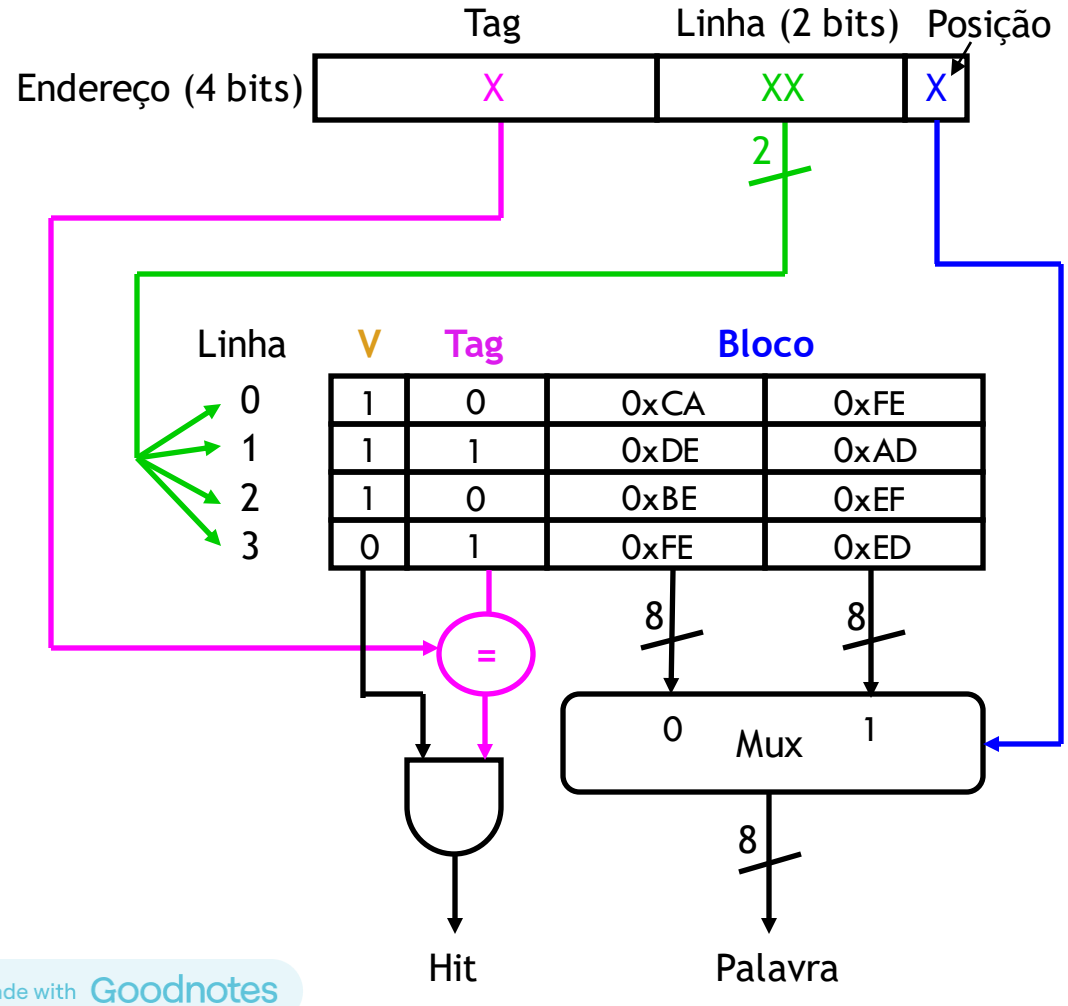
TURMA A1

TÓPICO 3 – MEMÓRIA CACHE (AULA 22)

Made with Goodnotes

Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

Um outro exemplo – Exercício

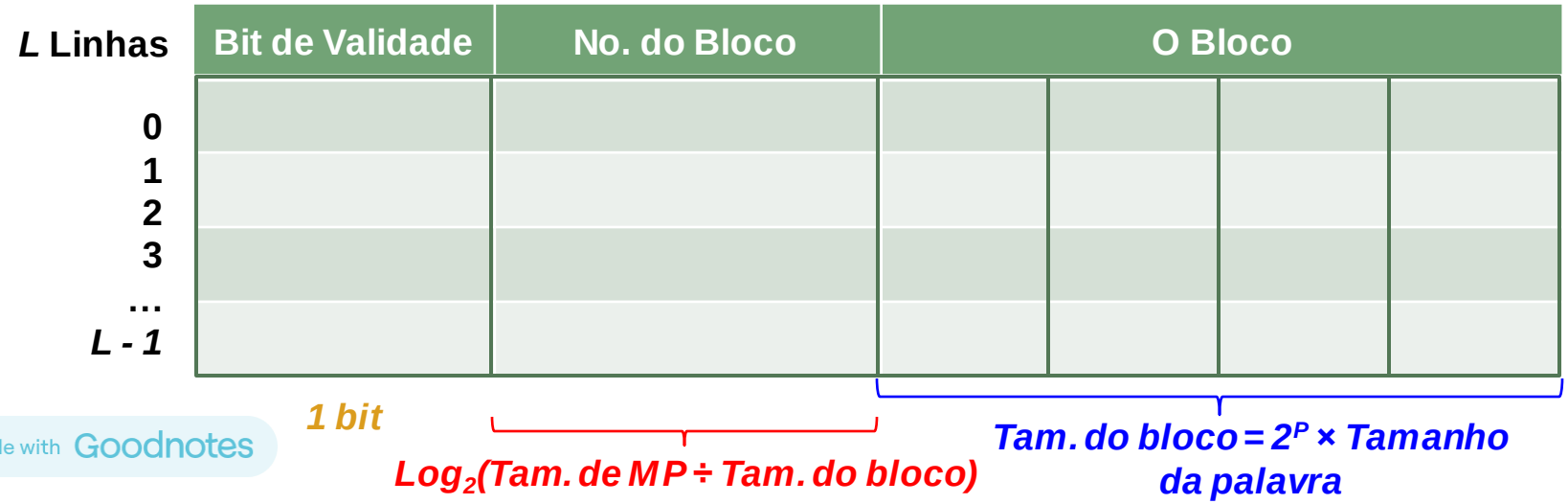


- Para cada endereço em seguir, que este cache vai retornar?
  - Será que uma palavra é lido da cache ou vamos ter um *cache miss*?
  - 1010 (0xDE)
  - 1110 (miss, entrada vazia)
  - 0001 (0xFE)
  - 1101 (miss, os *Tags* estão diferentes então o bloco não está na cache)

Made with Goodnotes

Cache Associativa

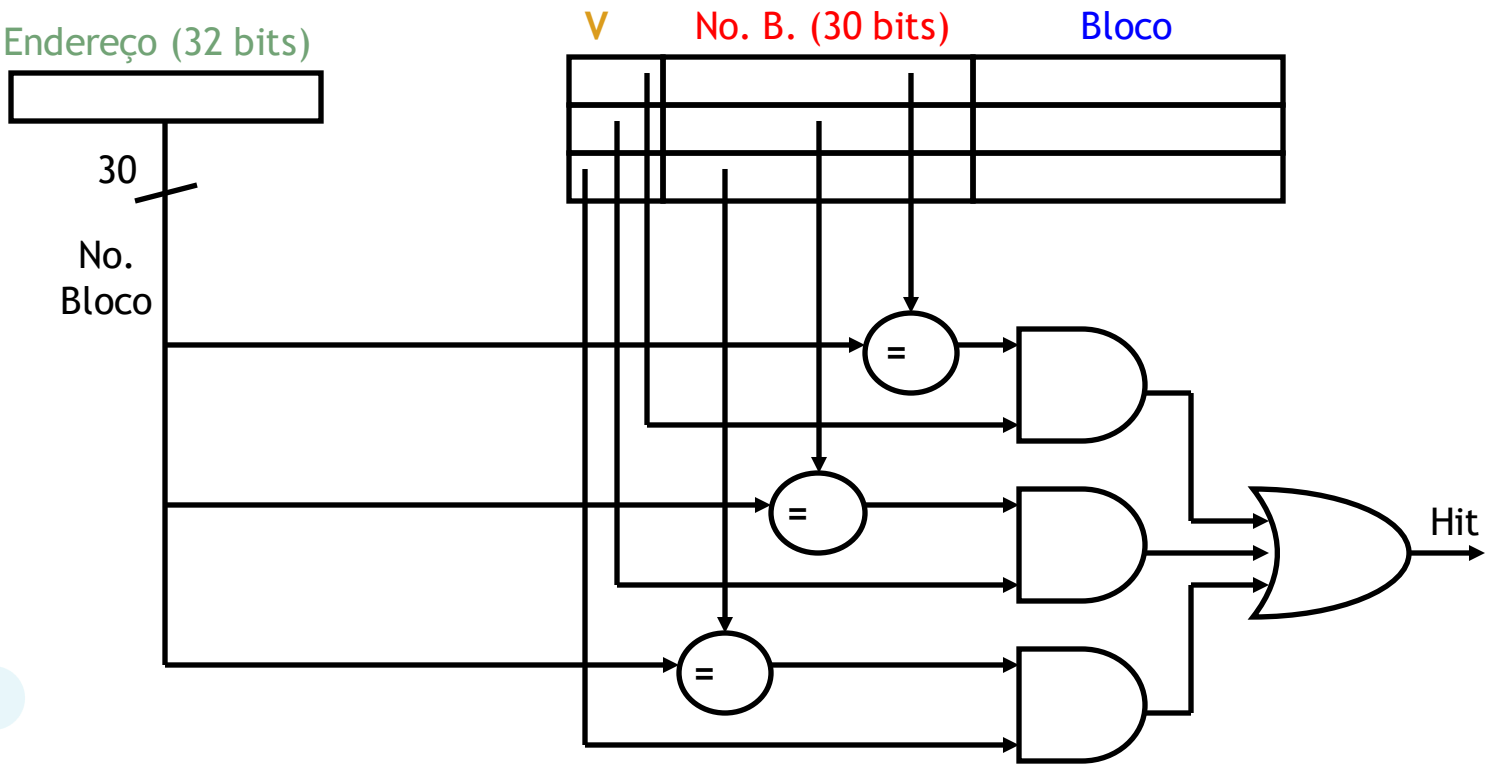
- Neste tipo de cache, um bloco pode ser carregada em qualquer linha (disponível) da cache
- Cada linha tem três campos: um bit de validade que identifica se a linha está guardando um bloco, o número do bloco e o bloco
  - Custo do cache é relacionado seu tamanho =
    - $L \times (1 + log_2(\text{Quantidade de blocos na MP}) + \text{Tamanho do bloco})$ bits



Made with Goodnotes

Busca em paralela da Cache Associativa

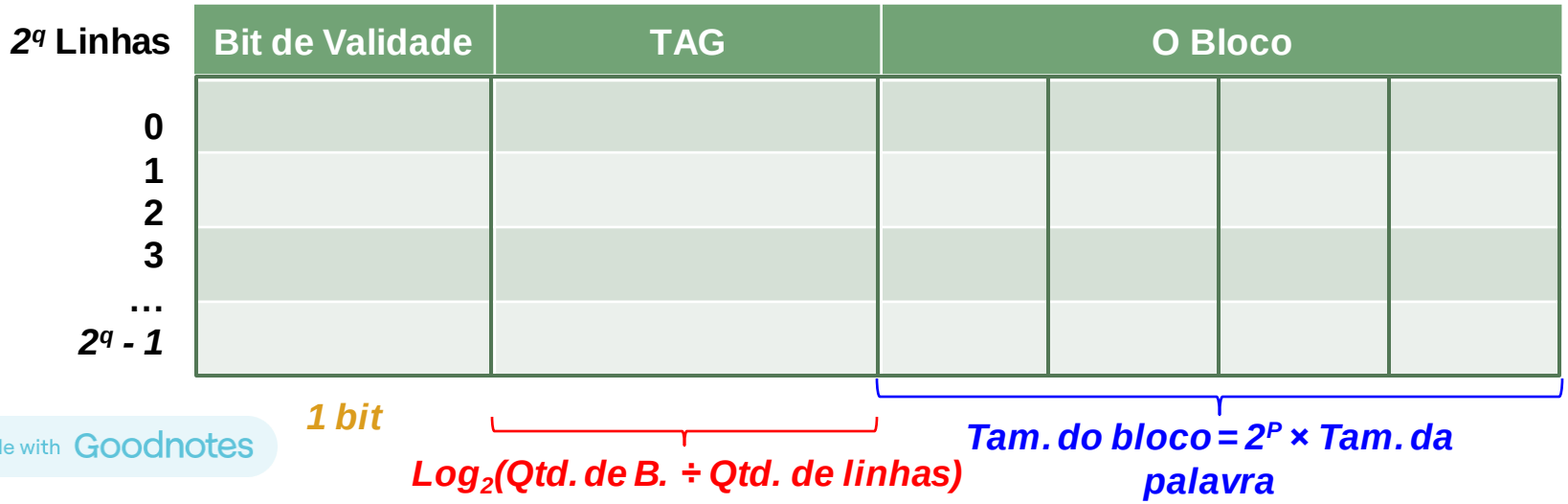
- Nesta cache um bloco pode ser carregada em qualquer linha de cache
  - Então, para achar um dado bloco, vai precisar buscar no cache inteiro
    - Para não demorar, é feita em paralelo, buscando em todas as linhas no mesmo tempo
    - Isso pode ser bastante caro, dependendo na quantidade de linhas (L) do cache



Made with Goodnotes

Cache com Mapeamento Direto

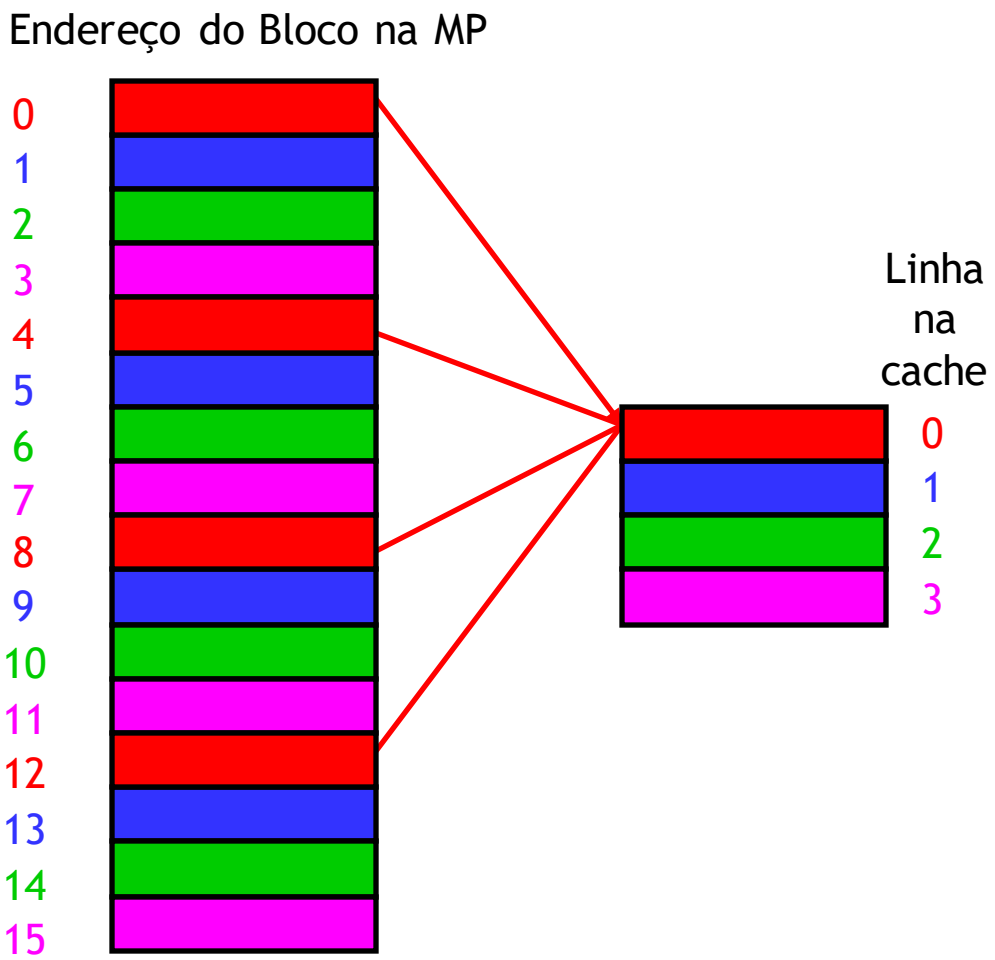
- Neste tipo de cache, **cada bloco** da memória principal **tem um único lugar (uma linha fixa)** na memória cache
  - Cada linha tem três campos: o bit de validade que identifica se a linha está guardando um bloco, um identificador do bloco (TAG) e o bloco
  - Tamanho da cache = $2^q \times (1 + Log_2(Qtd\ de\ ??) + Tamanho\ do\ bloco)$



Made with Goodnotes

Cache com Mapeamento Direto

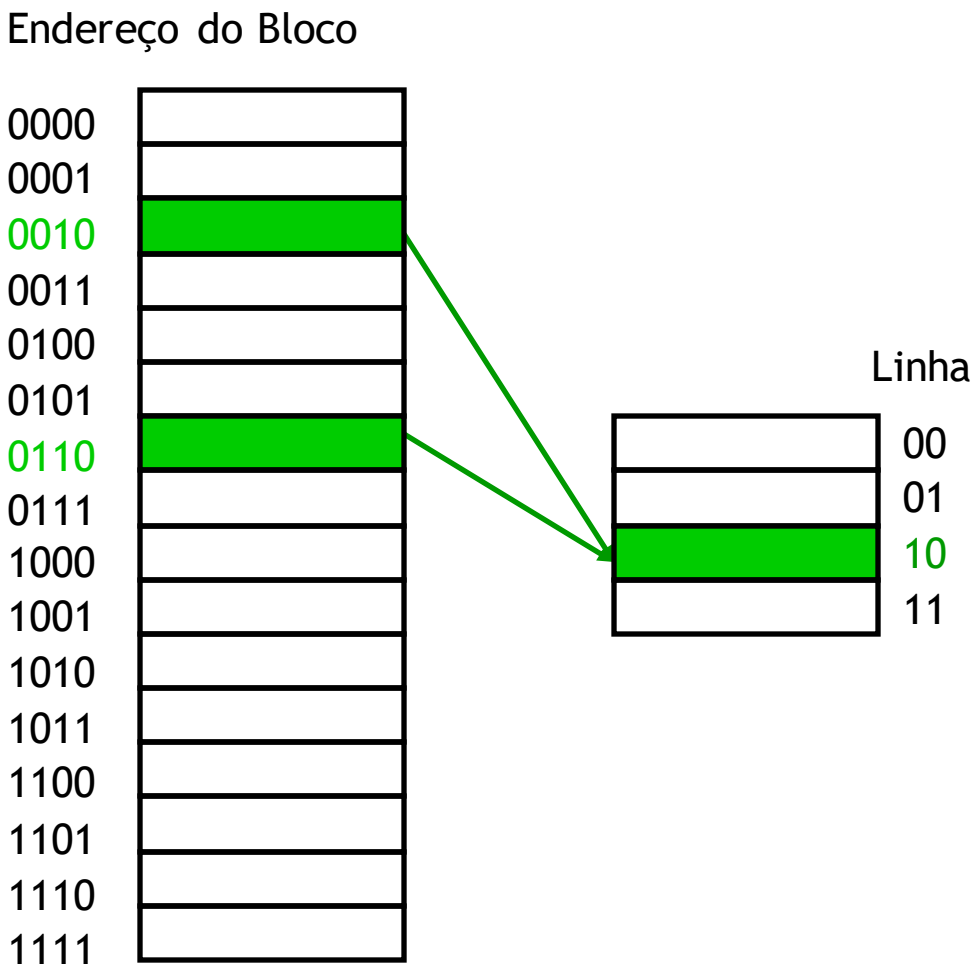
- Cada bloco tem um **único** lugar na memória cache onde pode ser guardado, mas onde?
- O mapeamento é equivalente a uma função *hash*



Made with Goodnotes

Cache com Mapeamento Direto

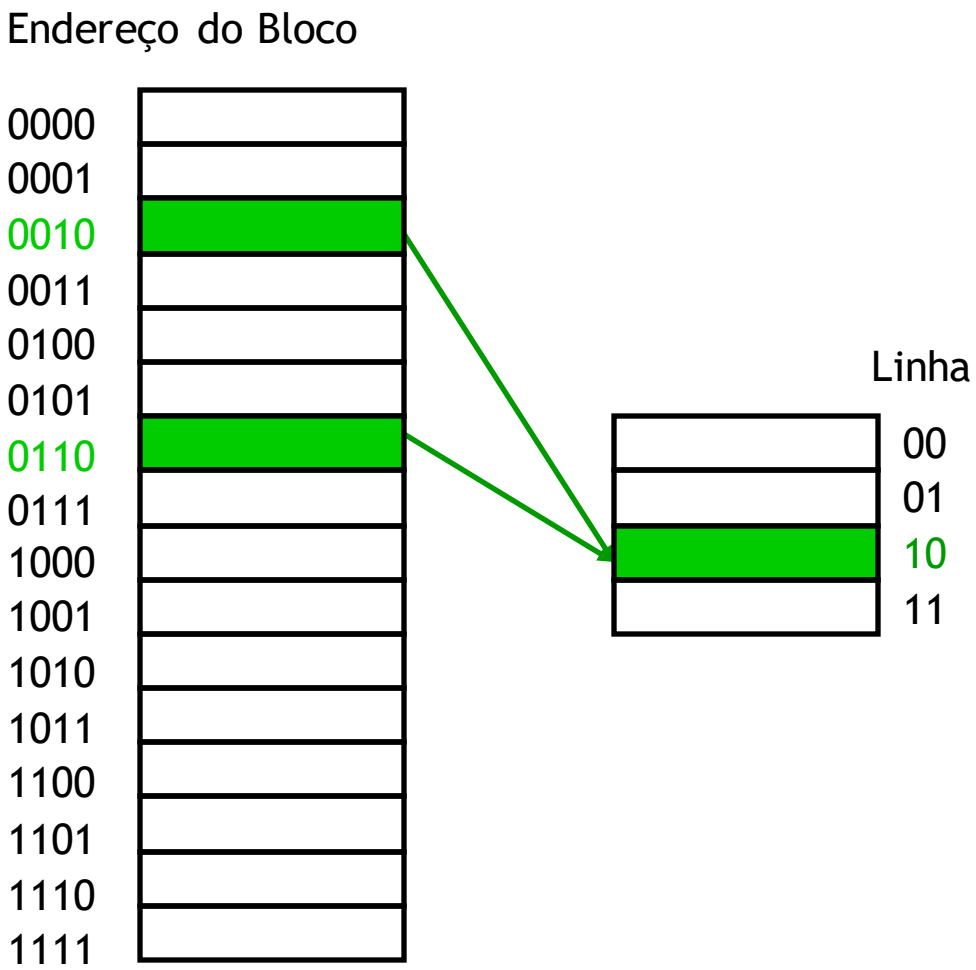
- Esta cache é mais fácil para implementar
  - As índices e deslocamentos não precisam ser calculados porque:
    - A cache tem 2^q linhas
    - Cada endereço de memória pertence a exatamente um bloco de tamanho 2^P.
- Mas, o que acontece se um programa usa endereços em blocos 2 e 6, 2 e 6, com frequencia?



Made with Goodnotes

Cache com Mapeamento Direto

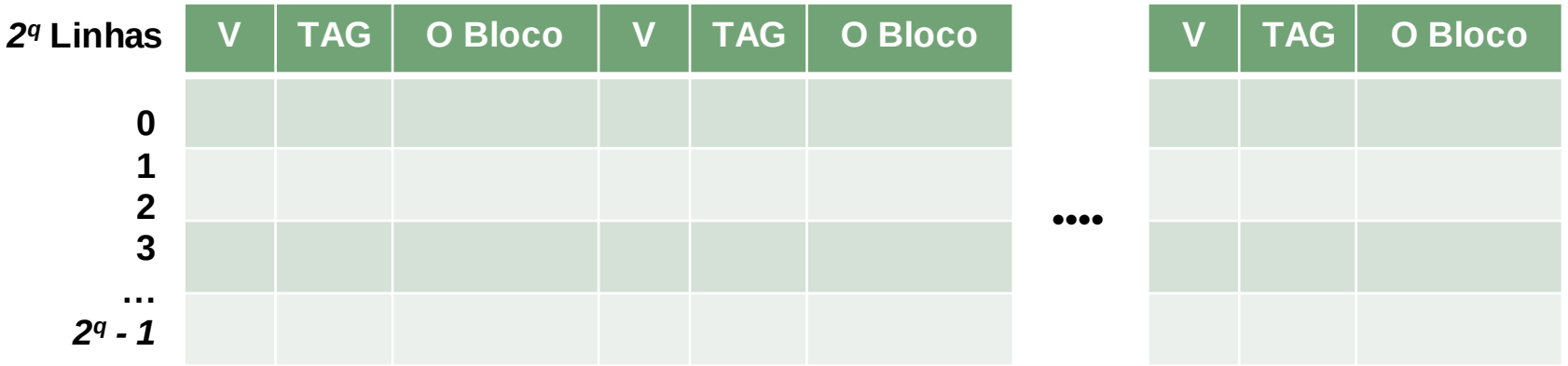
- Cada endereço de MP é mapeado à uma unica linha da cache
- No entanto, isso não é realmente flexível.
  - Se um programa usa endereços
  - em blocos 2, depois 6, depois 2 de novo, etc., cada acesso resultará em uma falha de cache e a recarregamento da linha 2
  - Este cache tem quatro entradas, mas o mapeamento direto pode não deixar usar todos elas.
  - Isso pode resultar em mais *cache misses* do que gostaríamos.



Made with Goodnotes

Cache Associativa por Conjunto (CAC)

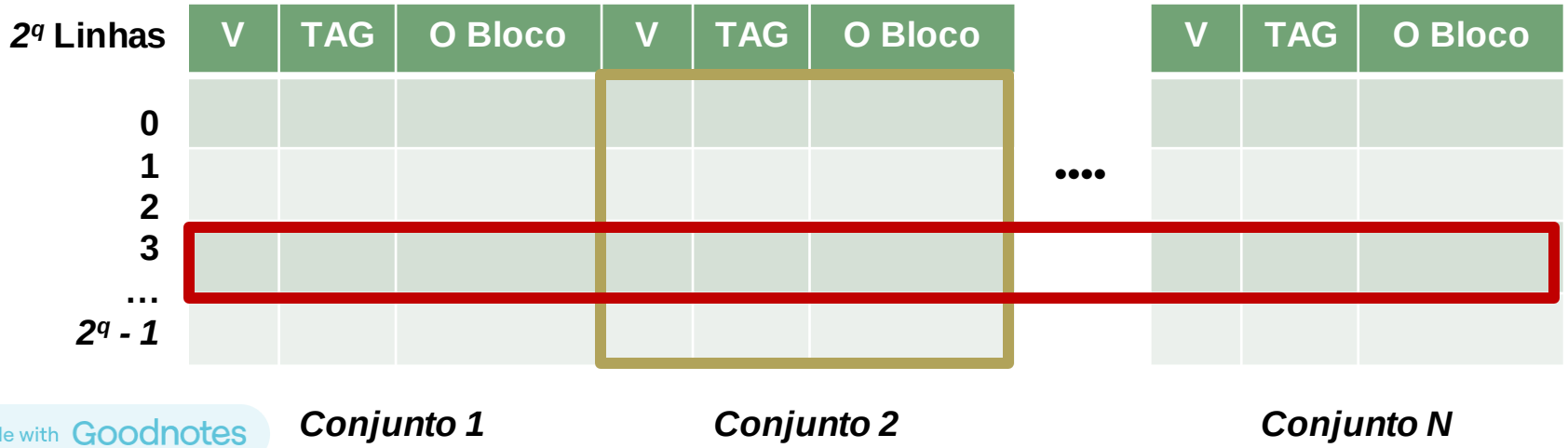
- Este tipo de organização intercala as vantagens dos dois caches anteriores acabando com
  - a comparação exaustiva dos endereços de bloco nas linhas do cache totalmente associativa, e
  - o problema de conflito de endereços por uma mesma linha na cache com mapeamento direto.
- *N-way set associative cache*:



Made with Goodnotes

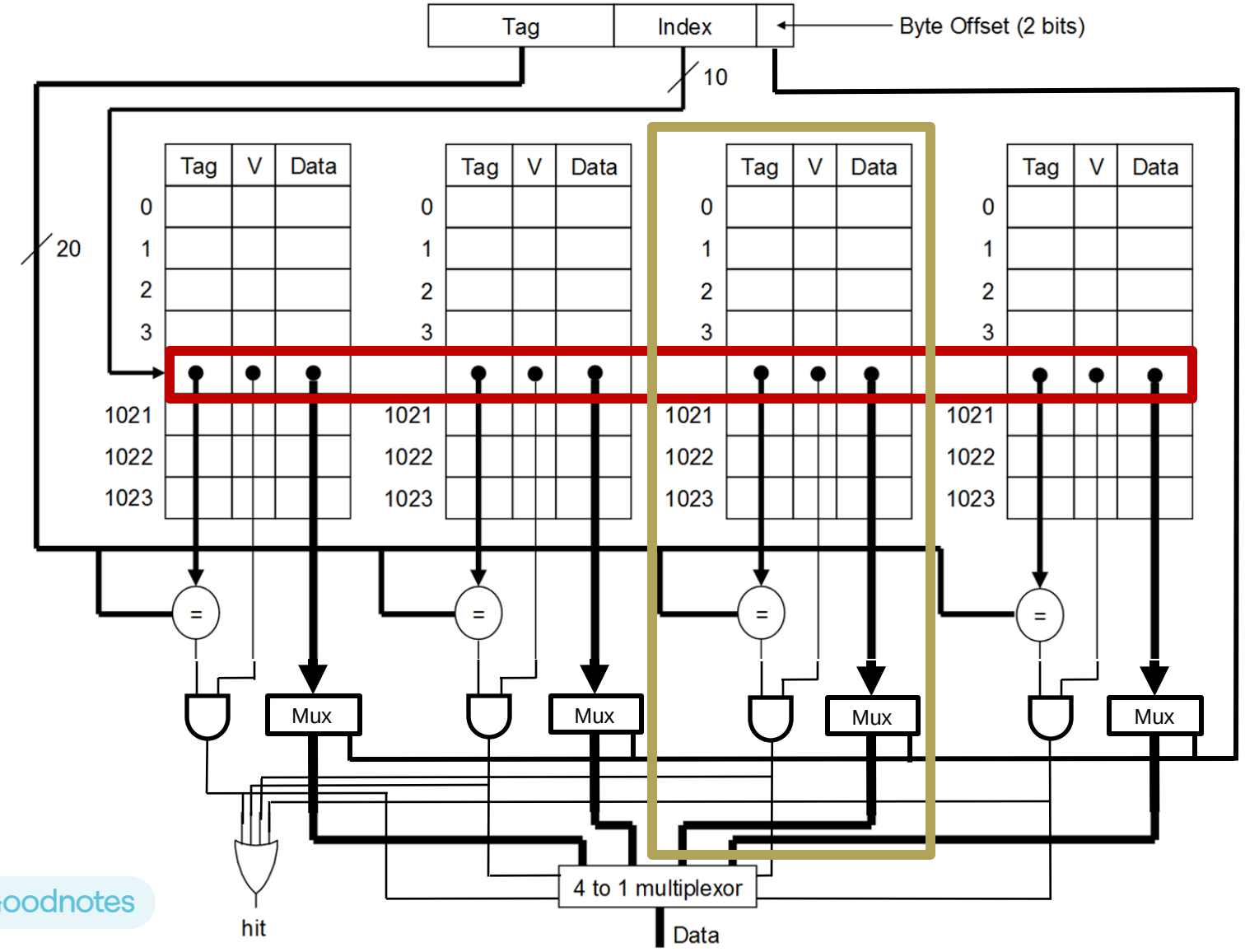
Cache Associativo por Conjunto

- A cache é composta de *N* **caches com mapeamento direto** coladas juntos
- Linha ***i*** de cada cache de mapeamento direto forma **um cache associativo** de tamanho *N* entradas
- Cada endereço de memória é mapeado para exatamente uma linha da cache (como no caso do cache com mapeamento direto), mas o bloco pode ser colocados em qualquer conjunto (ou qualquer entrada do cache associativo) daquela linha.



Made with Goodnotes

Um exemplo de CAC



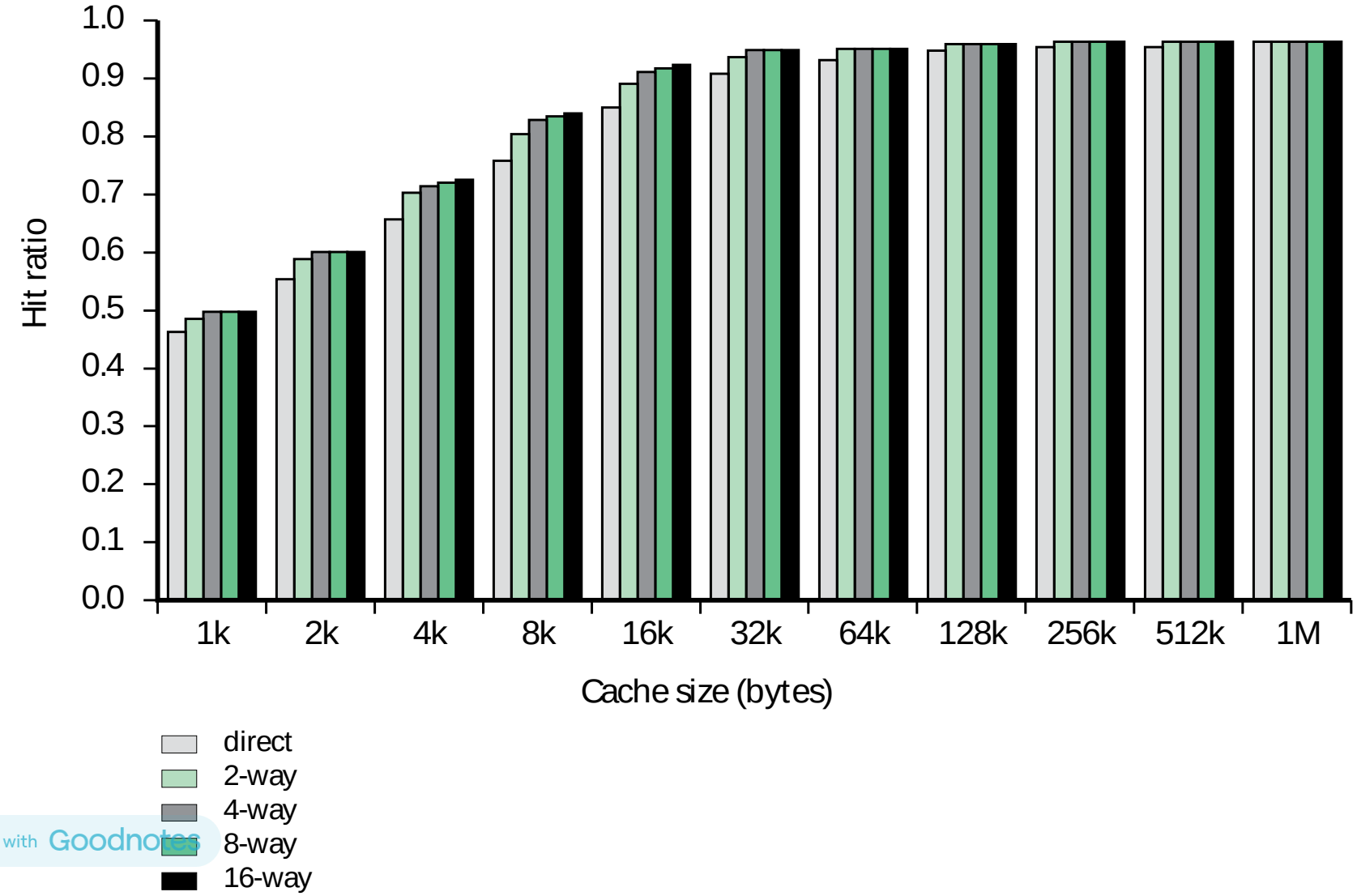
Made with Goodnotes

Cache Associativa por Conjunto

- Tamanhos maiores de **bloco** podem tirar vantagem da **localidade espacial** carregando no cache dados não apenas de um endereço, mas também de endereços próximos
- As caches associativas mapeia cada bloco de memória a um determinado conjunto dentro da cache, mas não para uma posição específica dentro desse conjunto.
- Os tamanhos dos conjuntos podem variar de 1 (torna-se uma cache com mapeamento direto) a *L* (geralmente 2^k), a quantidade total de blocos na cache (esta seria uma cache totalmente associativa).
- Quantidades de conjuntos maiores e uma maior associatividade levam a menos conflitos de cache e então talvez taxas de falha mais baixas, porém também aumentam o custo de hardware.
- Na prática, caches associativas por conjunto de 2 a 16 atingem um bom equilíbrio entre baixas taxas de falha e custos.

Made with Goodnotes

Associatividade x Tamanho da Cache



Made with Goodnotes

Política de Substituição/Despejo

- No caso de uma cache com mapeamento direto, quando uma entrada/linha está ocupada porém não com o bloco necessário, é fácil sabe que este bloco teria que ser substituído.
  - O TAG do endereço e TAG da entrada serão diferentes
- No caso de caches associativas (quando está cheia) ou caches associativas por conjunto (quando a linha está cheia) e bloco não se encontra na cache, algum bloco teria que retirado da cache para criar espaço para o novo bloco que precisar ser trazido da memória principal.
- A questão é qual bloco deve ser escolhido para ser removido?

Made with Goodnotes

Política de Substituição

- O mais simples em termos de custo, seria despejar a entrada mais velha, a entrada que entrou antes as outras
  - A cache pode ser implementada como um buffer circular, sem aumentar o tamanho de cada linha
- Alternativamente, a escolha poderia ser feita numa forma randômica
  - Surpreendentemente, funciona muito bem na prática
- A melhor escolha talvez seria despejar o bloco que ficou sem uso para o maior tempo, o *Least Recently Used* (LRU)
  - Mas como implementa esta política?
    - Uma maneira necessita uma contador para cada entrada. Cada vez a cache recebe uma solicitação, todos os contadores são incrementados e se tem um hit em uma linha, o contador dela é zerado.
    - O bloco LRU é aquele que tem a maior valor no contador
    - Porém está implementação fica muito caro por necessitar um contador por linha além de ter decidir quantos bits o contador deve usar.
  - Na pratica, é usado uma forma aproximada que se chama *Pseudo LRU*
    - Dividindo a cache em grupos de entradas, o bloco mais velho do grupo LRU seria despojado.

Made with Goodnotes

Operações de escrita no cache

- Caches funcionam muito bem quando o processador precisa ler dados da memória
- Mas quando é necessário escrever?
  - O objetivo é alterar um valor em uma determina posição (endereço) de memória.
    - Mas lembra que os acessos agora ao MP são feitos por blocos
    - Observe também que 15% das referências de memória são gravações
- A operação de escrita traz o seguinte dilema:
  - Devemos sempre atualizar os valores na MP para manter a consistência dos dados? [Técnica chamada *Write Through*]
  - Ou devemos fazer atualizações só na cache enquanto o bloco estiver lá? Quer dizer, só quando um bloco modificado é removido do cache a memória principal seria atualizada. [*Copy Back* ou *Write Back*]

Made with Goodnotes

Escrita: *Write Through*

- Sempre que o processador atualiza uma posição de memória, o dado é atualizado tanto na cache, quanto na MP
- Mantém a consistência dos dados da cache e da MP.
  - MP não tem dados com valores diferentes daqueles na cache.
- Isso é bom porque:
  - Simplifica substituição de bloco na cache;
  - Alguns dispositivos de E/S podem ler diretamente da MP, sem passar pela cache.
- Por outro lado, escritas feita pelo processador vão sempre ser lentas
  - Sempre precisam fazer acessos à MP.

Made with Goodnotes

Escrita: *Write Through* com *buffer*

- Caches que utilizam *write through* podem sofrer de escritas lentas, portanto, os processadores normalmente incluem um buffer, que enfileira escritas pendentes para a memória principal e permite que a CPU continue sua execução
- Buffers são comumente usados quando dois dispositivos são executados em velocidades diferentes.
  - Se um produtor gerar dados muito rapidamente para um consumidor manipular, o dados extras são armazenados em um buffer e o produtor pode continuar com outras tarefas, sem esperar pelo consumidor.
  - Por outro lado, se o produtor desacelerar, o consumidor pode continuar
  - rodando em velocidade máxima enquanto houver dados em excesso no buffer.
  - No caso de escrita, o produtor é a CPU e o consumidor é a memória principal.

Made with Goodnotes

Escrita: *Copy/Write Back*

- Nesta técnica, operações de escrita sempre só envolve a cache
- Apenas quando bloco é removido do cache, o conteúdo é atualizado na MP.
  - Normalmente, há um bit em cada linha da cache, chamado *dirty bit*, que acusa se o bloco foi modificado e precisa ou não ser atualizado na MP.
- Assim como as leituras, escritas também são tipicamente rápidas por acessar apenas a cache.
  - Se um único endereço é escrito com frequência, então não vale a pena continualmente re-escrever esses dados na memória principal.
  - Se várias palavras dentro do mesmo bloco de cache forem modificados, ainda vai acontecer apenas uma operação de escrita de memória no momento do *write-back*.

Made with Goodnotes

Escrita: *Copy/Write Back*

- Por outro lado:
  - Agora a cache e a memória contêm dados diferentes que quer dizer estão em um estado inconsistente!
  - Dispositivos de E/S que acessariam diretamente a memória precisam passar pela cache.
    - Isso pode sobrecarregá-la.
  - Há ainda a questão de como lidar com vários processadores/núcleos, cada um com sua própria cache.
    - Problema de Coerência de Cache

Made with Goodnotes

Write around ou *Allocate on write*

- Uma das duas políticas de escrita anteriores pode ser usado quando o bloco da palavra a ser escrita já está no cache
- Um segundo cenário seria quando o processador quer escrever uma palavra num bloco que não está na cache?
  - Com uma política de *write around* ou *write-no-allocate*, a operação de escrita vai diretamente para o principal memória sem afetar a cache.
  - Alternativamente, uma estratégia de *allocate on write* carregaria os dados recém-gravados na cache.

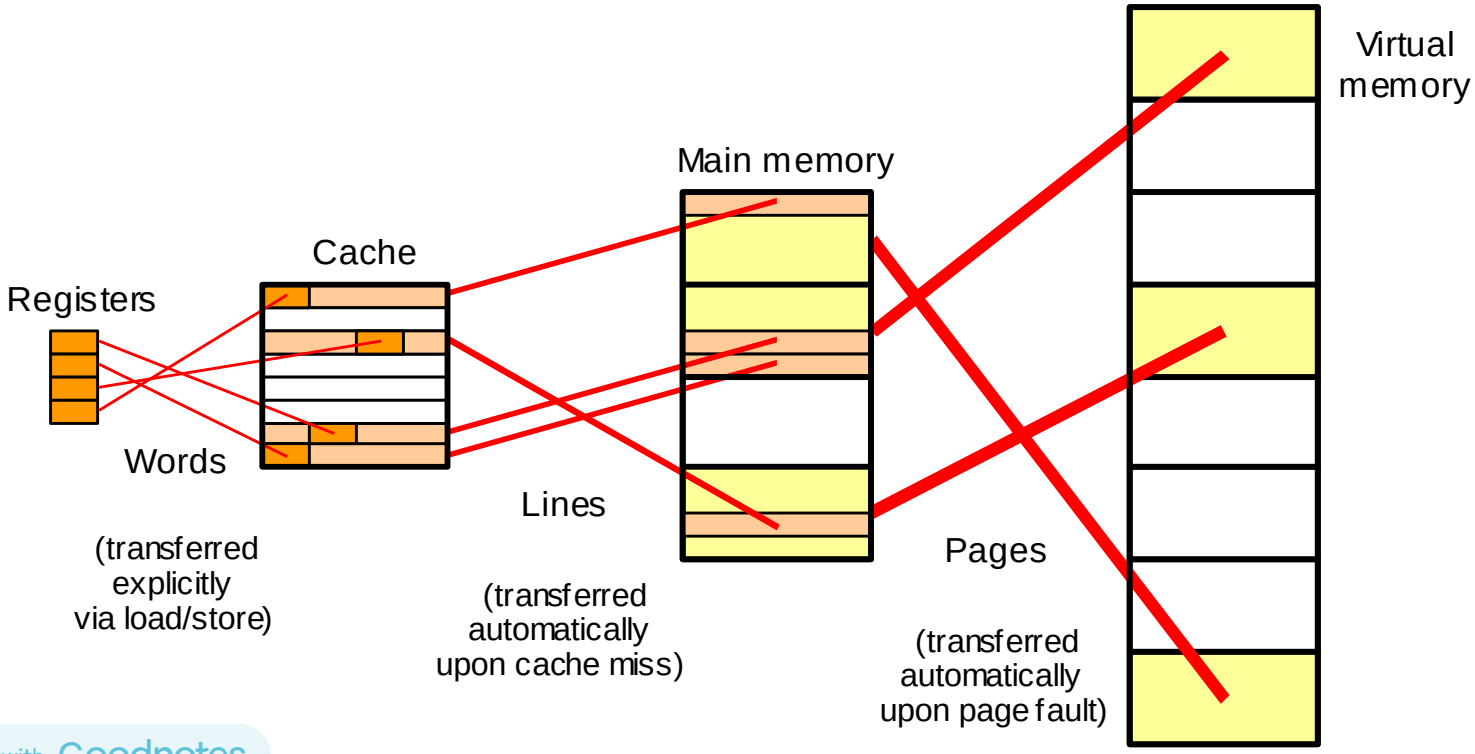
Made with Goodnotes

Escrita: Um resumo

- Escrever em um cache apresenta alguns problemas interessantes
  - As políticas de *write-through* e *write-back* mantêm o cache consistente com memória principal de maneiras diferentes para acertos de escrita (*write hits*)
  - *Write-around* e *alocate-on-write* são duas estratégias para lidar com *write misses*, diferindo em se os dados atualizados são carregados na cache ou não
- O desempenho do sistema de memória depende do tempo de acerto do cache, taxa de falha e penalidade da falha, bem como o programa sendo executado.
  - Podemos usar esses custos para encontrar o tempo médio de acesso à memória.
    - AMAT = tempo de acerto + (taxa de falha x penalidade de falha)
- A organização de um sistema de memória afeta seu desempenho.
  - O tamanho do cache, o tamanho do bloco e a associatividade afetam a taxa de falha.
  - Podemos organizar a memória principal para ajudar a reduzir as penalidades por falhas.
    - Por exemplo, a memória intercalada oferece suporte a acessos de dados em pipeline.

Made with Goodnotes

Localidade na hierarquia de memória



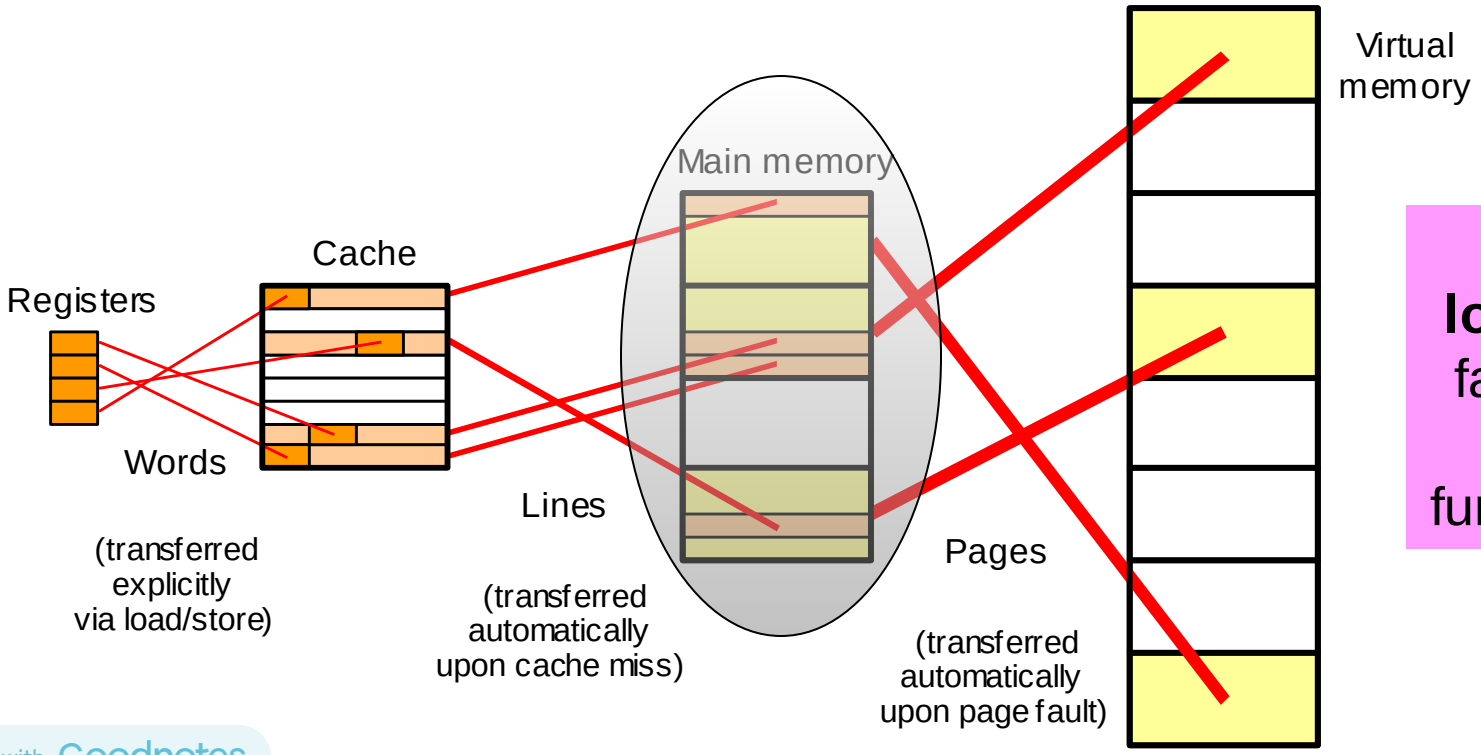
Made with Goodnotes

Localidade na hierarquia de memória

Memory Cache: fornece a ilusão de alta velocidade

Memoria Principal: custo razoável, mas é lento e pequena

Memory Virtual: fornece a ilusão de um tamanho muito grande



A **localidade** faz que as ilusões funcionarem

Made with Goodnotes

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 4 – PARALELISMO NO NÍVEL DE INSTRUÇÃO (AULAS 23 E 24)

Made with Goodnotes

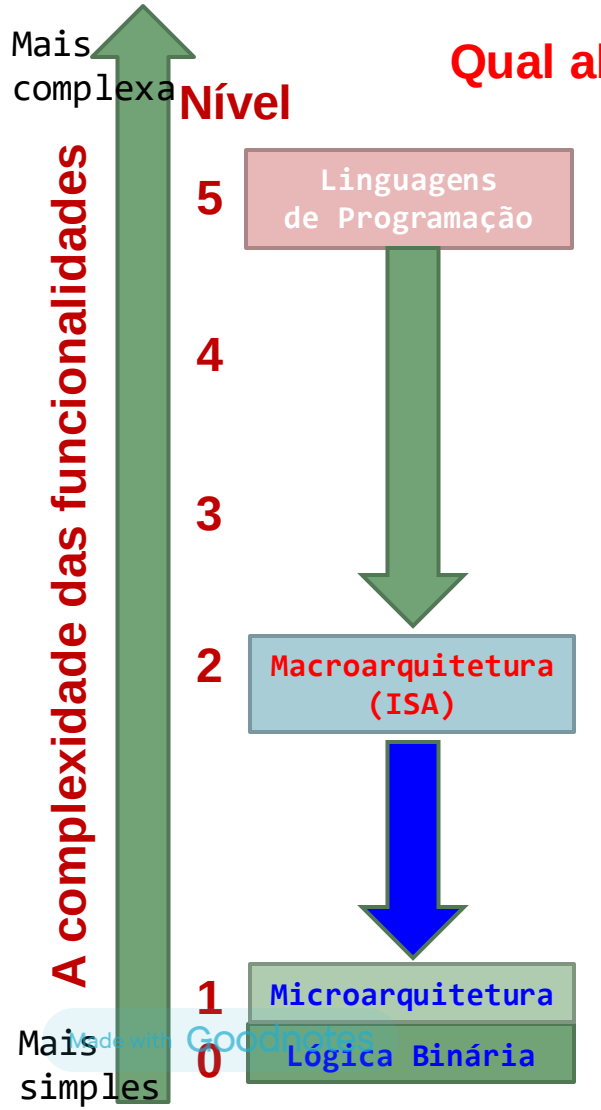
Prof. Vinod Rebello, vinod@ic.uff.br, Sala 413

A Nova Era Dourada de Arq. Comp.

- O compilador do código-fonte toma decisões realmente detalhadas sobre exatamente quais instruções, em que ordem, devem operar em quais registradores.
- Mas o processador então pega esta codigo, altamente otimizadas para execução sequencial, e tenta encontrar paralelismo.
  - Ele substitui as decisões tomadas pelo compilador quanto à ordem das instruções, reordenando-as, e os registradores a serem usados, renomeando-os.
  - Ele adivinha quais acessos à memória serão necessários, usando caches, que funcionam na maioria das vezes.
  - Ele adivinha para que lado os desvios irão e executa instruções ***especulativamente***.
  - E então, talvez a parte mais incrível de todas, ele faz tudo isso de uma maneira que todas as instruções parecem ser executadas na ordem exata em que o programador e o compilador escreveram em primeiro lugar.
- Mas para algoritmos especializados, com paralelismo e algoritmos bem compreendidos, jogar todo o conhecimento fora e tentar recuperá-lo rapidamente pode não ser o ideal.

Made with Goodnotes

Tópico 2: A grande pergunta



Qual abordagem executa programas mais rápido?

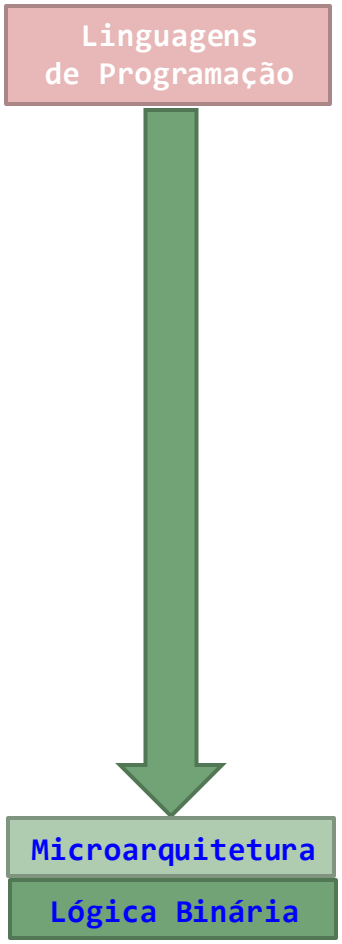
Opção A: Tradução (Compilador + Montador) e Interpretação (no Processador)

Fazendo interpretação não está atrasando a execução?

Opção B: Só Tradução, (Com um Compilador gerando instruções de nível 1 diretamente)

Quão fácil será para o compilador fazer isso?

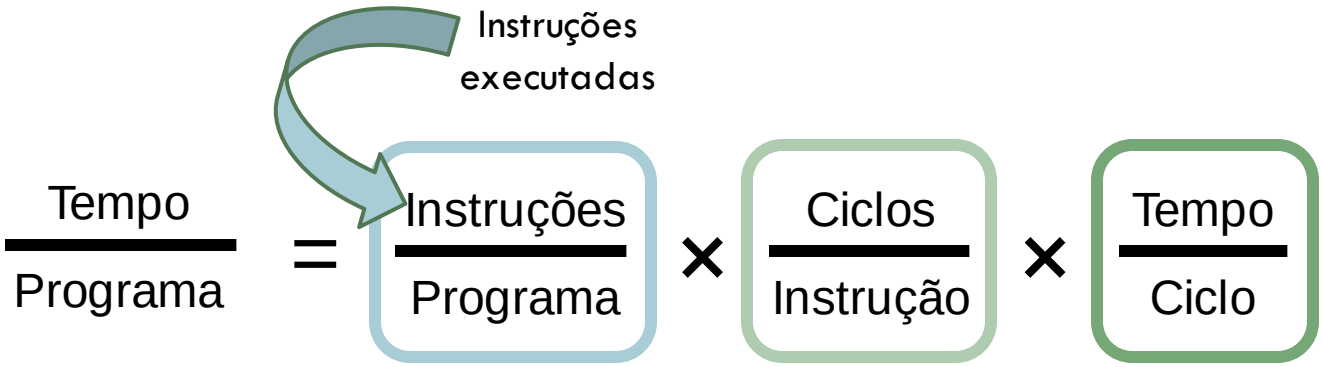
Haverá um impacto adverso no desempenho?



Made with Goodnotes

Desempenho arquitetural

- O tempo de execução depende nas caraterísticas do sistema todo, mas onde podemos achar melhorias?

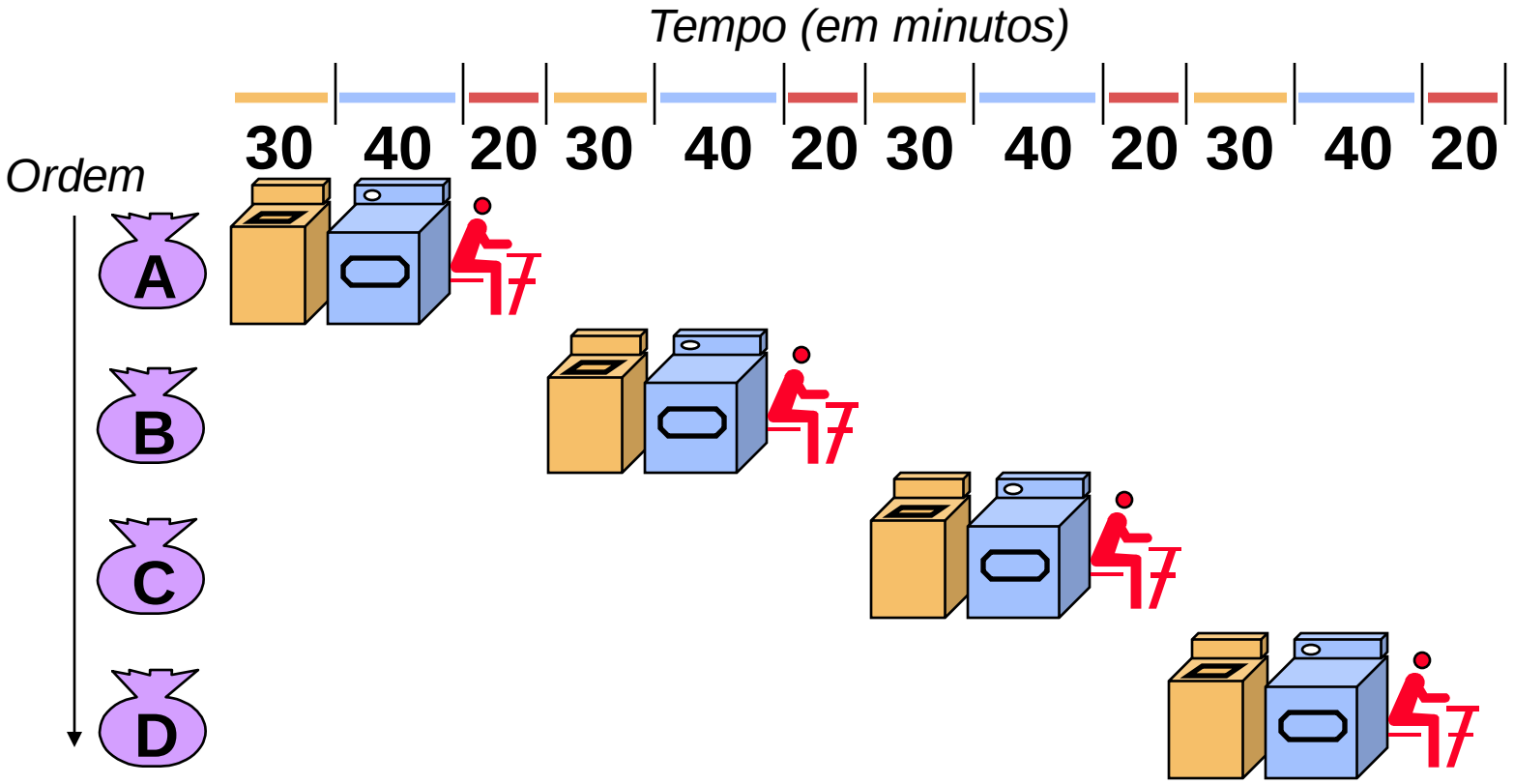


Depende no: *Compilador* ***Microarquitetura*** *Tecnologia*

- Como melhorar o desempenho de programas Von Neumann?
  - *"Bend the rules?" = Vergar as regras?*
  - Seria possível que algum desses elementos da equação pode chegar a ser menor que um? Por exemplo, o CPI < 1

Made with Goodnotes

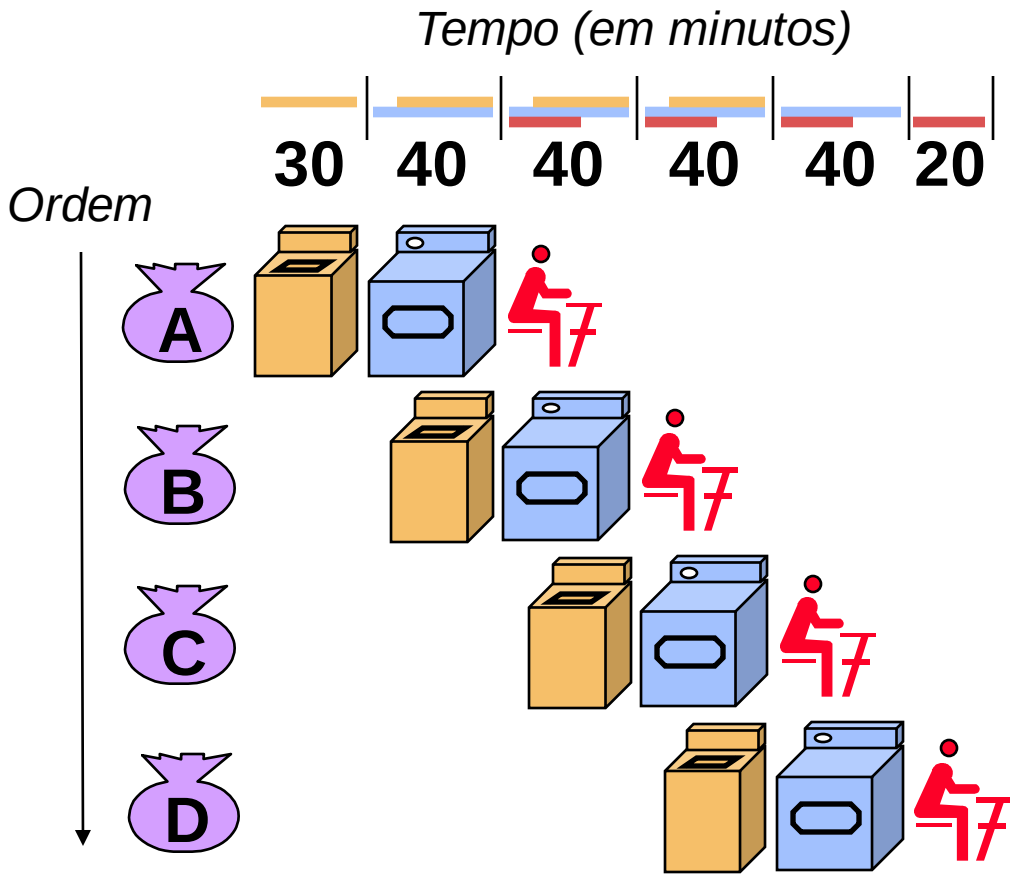
Execução sequencial



- Comparando os passos para a executar uma instrução com a tarefa de lavanderia: lavar, secar e passar roupa
  - Anote que as etapas podem levar tempos diferentes

Made with Goodnotes

Pipelining

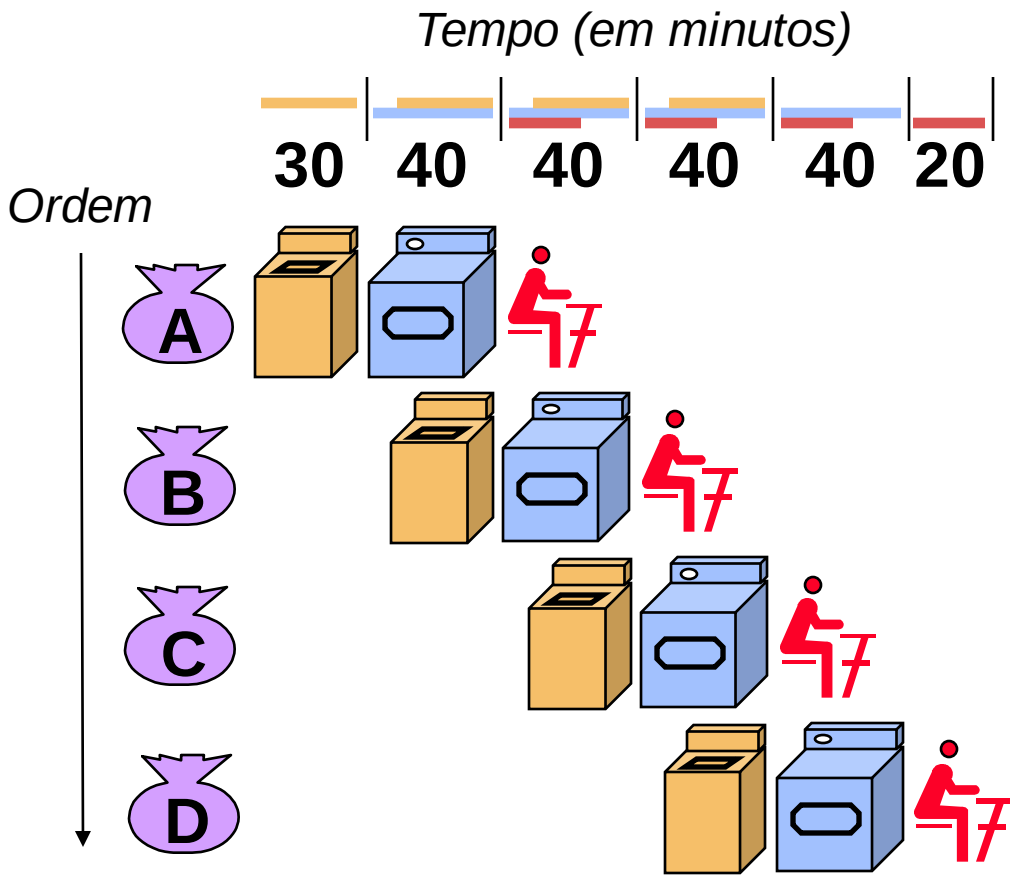


- Linha de montagem
- Não espera terminar a tarefa atual (por ex., A) antes de começar a próxima tarefa (B)

- Qual é a máximo ***Speedup*** de um pipeline?

Made with Goodnotes

Pipelining



- Linha de montagem
- Não espera terminar a tarefa atual (por ex., A) antes de começar a próxima tarefa (B)

- Qual é a máxima ***Speedup*** de um pipeline?
- Um pipeline pode atingir uma aceleração potencial equivalente ao número de estágios do pipeline

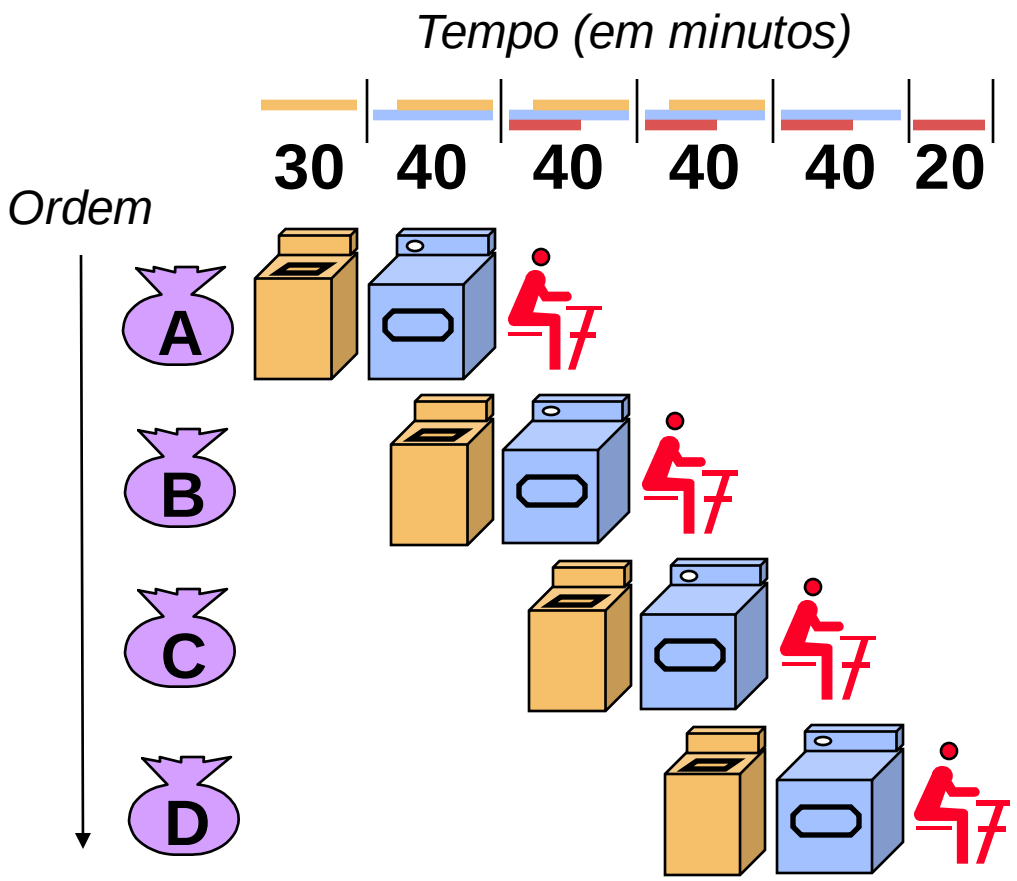
Made with Goodnotes

Pipelining

- Vamos supor que:
  - Cada tarefa  (macroinstrução) seria realizada por um sequências de **e** estágios ou etapas (cada uma equivalente a uma ou mais microinstruções);
  - Cada microinstrução leva **c** ciclos;
  - Programas contem **N** macroinstruções (tarefas).
- Tempo de Execução (TE):
  - Sequencial = **N.e.c**
  - Pipeline = **e.c + (N - 1).c**
- Maior Speedup, $S_{max} = TE_{sequencial} / TE_{pipeline}$
- => $S_{max} = [N * e * c] / [(e + N - 1) * c]$
- $S_{max} = [N * e] / [e + N - 1]$
- Mas observe que geralmente **N >> e** ! Então, S aproxima **e**, a quantidade de estágios no pipeline.

Made with Goodnotes

Pipelining



- O pipeline **não melhora a latência de uma única tarefa**, mas **a vazão da toda a carga de trabalho (o programa)**
  - Quanto tempo precisa esperar entre a terminação de tarefas?
- Taxa do pipeline é limitada pelo estágio do pipeline mais lento
- **Múltiplas tarefas estão operando simultaneamente**
- Comprimentos desequilibrados dos estágios do pipeline reduzirão as acelerações
- O tempo para "preencher" o pipeline e "drená-lo" reduz a aceleração

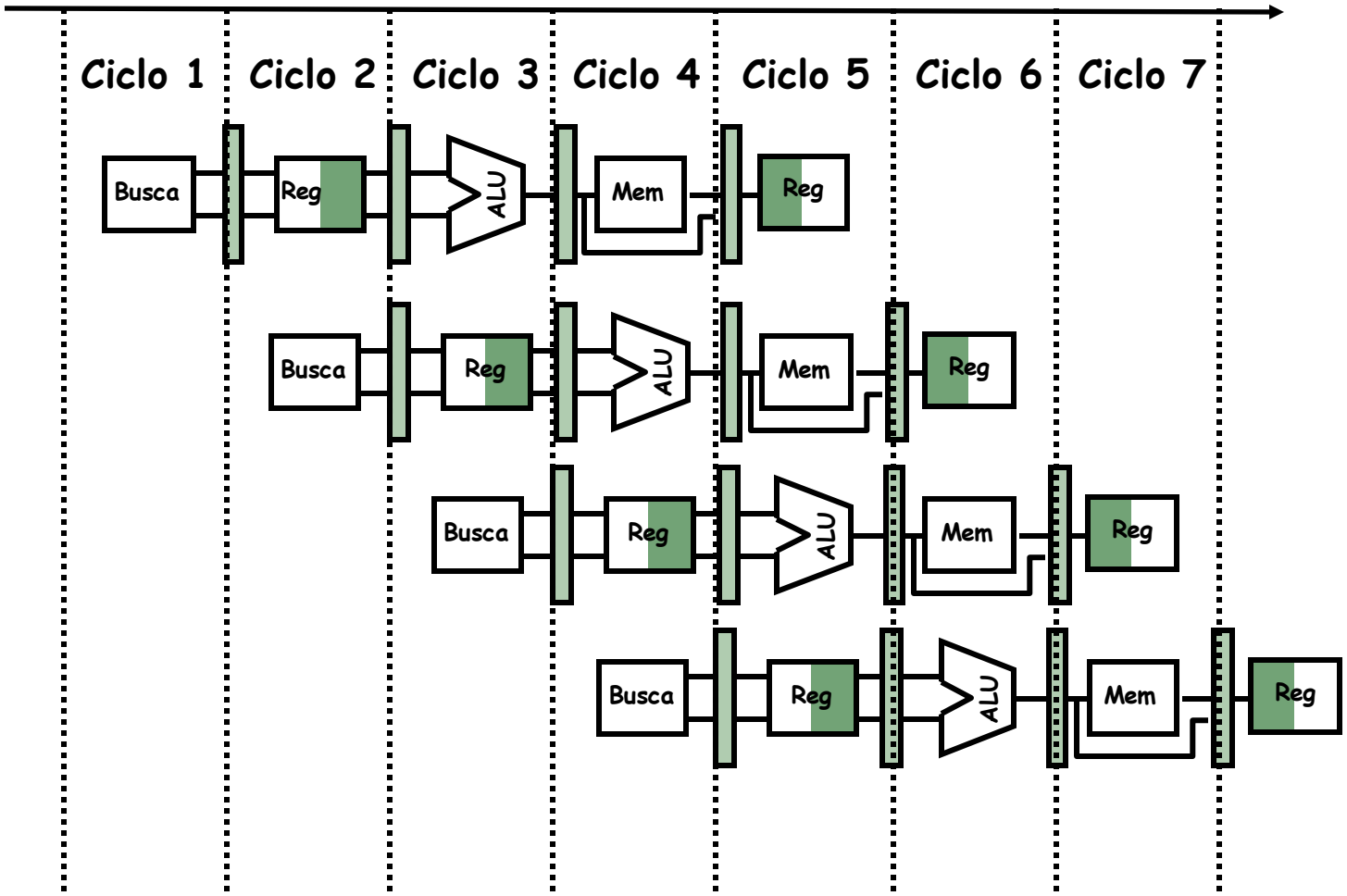
Made with Goodnotes

Pipeline RISC clássico

- Tem cinco estágios:
  - **Estágio 1 (Instruction Fetch - Busca de instrução):** Neste estágio, a CPU lê as instruções do endereço na memória cujo valor está presente no contador de programa (registrador PC).
  - **Estágio 2 (Instruction Decode):** Nesta etapa, a instrução é decodificada e o conjunto de registradores é acessado para obter os valores dos registradores utilizados na instrução.
  - **Estágio 3 (Execução da Instrução):** Nesta etapa são realizadas as operações da ALU.
  - **Estágio 4 (Acesso à Memória):** Nesta etapa, os operandos de memória são lidos e escritos de/para o endereço da memória que está presente na instrução.
  - **Estágio 5 (Write Back):** Nesta etapa, o valor calculado/buscado é reescrito no registrador presente na instrução atual.

Made with Goodnotes

Pipeline RISC clássico



Made with Goodnotes

Limitações

- Dependências são propriedades de um programa que têm o potencial de ser perigosas (causa *Hazards* ou erros) se não impedir que a próxima instrução seja executada durante o seu ciclo de relógio designado
  - Dependências estruturais
    - Limitações de hardware não permite a execução de todas as combinações de instruções
  - Dependências de dados
    - Se instruções no pipeline não são independentes, uma pode ter que esperar o resultado de uma outra anterior
  - Dependências de controle
    - Quando a próxima instrução é buscada e quando é o fluxo/sequencia determinado? A execução de alguns instruções determinam a instrução seguinte
- Para evitar erros nos programas, a solução barata é parar pipeline até que a dependência seja resolvida, inserindo uma ou mais **"bolhas"**
- O impacto no **tempo médio de execução da instrução** (CPI) em relação o valor ideal (uma por ciclo) define **a eficiência do pipeline**

Made with GoodNotes

Dependências

- Dependências estruturais
  - Esse tipo surge devido ao conflito de recursos no pipeline. Um conflito de recurso é uma situação em que mais de uma instrução tenta acessar o mesmo recurso no mesmo ciclo. Um recurso pode ser um registrador, a memória, ou elemento da via de dados.
  - Solução: replicação do recurso
- Dependências de dados
  - Três tipos de dependências de dados existem, só um afeta um pipeline clássico
  - Solução: **Reordenação** das instruções no programa e/ou a técnica de hardware ***Data forwarding***. Arquiteturas superescalares também emprega **Renomeação de registradores**
- Dependências de controle
  - Esse tipo de dependência ocorre quando a sequência das instruções é alterada (quando tem mudanças no fluxo do controle) através instruções do tipo BRANCH, CALL, JUMP, etc. Em muitas arquiteturas, o processador não saberá o endereço de destino dessas instruções quando precisar inserir a nova instrução no pipeline. Devido a isso, instruções indesejadas podem alimentar o pipeline.
  - Solução parcial: **Previsão** ou **Predição de desvios**

Made with Goodnotes

A eficiência de um pipeline

- A **Eficiência** = S_{obtido} / S_{max}
  - Sabemos que S_{max} = e
- A **Vazão (*Throughput*)** é o número de instruções executadas dividido pelo tempo para executa-las = N / TE
  - Então, a vazão = N / ((e + N – 1) * c)
- O inverso da vazão (TE / N) é o tempo médio por instrução
  - Comumente indicado em termos de ciclos: **Ciclos Por Instrução (CPI)**
  - O CPI do pipeline ideal é 1
- A eficiência do pipeline também = $CPI_{ideal} / CPI_{obtido}$

Made with Goodnotes

O CPI de um pipeline

- Instruções com dependências podem gastar mais de um ciclo devido os atrasos (as bolhas), precisamos identificar quais são:
  - Deixa P_{DC}, P_{DL}, e P_{DD} sejam as percentagens dos N instruções que são do tipo desvios condicionais, desvios de laços e instruções com dependências de dados.
  - As respetivas penalidades (o número de ciclos de atraso para cada instrução com a dependência): b_{DC}, b_{DL}, e b_{DD}
  - Técnicas de predição de desvios conseguem prever alguns dos desvios corretamente e, então, nem sempre paga a penalidade. Deixa TA_{DC} e TA_{DL} sejam a taxa de acerto da técnica empregada.
- $$CPI_{obtido} = (1 - P_{DC} - P_{DL} - P_{DD})(1) + P_{DC}(TA_{DC}(1) + (1 - TA_{DC})(1 + b_{DC}) + P_{DL}(TA_{DL} + (1 - TA_{DL})(1 + b_{DL}) + P_{DD}(1 + b_{DD})$$

Made with Goodnotes

Previsão de Desvios (vídeos curtos em inglês)

- Branch in a Pipeline - https://www.youtube.com/watch?v=ckQAlp7WeHo
- Branch Prediction Requirements - https://www.youtube.com/watch?v=C_pQScxiISA
- Branch Prediction Accuracy - https://www.youtube.com/watch?v=wse0huBFfZA
- Why We Need Better Prediction? - https://www.youtube.com/watch?v=oFZEu_yLRuA
- Better Prediction How? - https://www.youtube.com/watch?v=Jxk7ZPxVKw4
- BTB Branch Target Buffer - https://www.youtube.com/watch?v=0b7T_035pRs
- Realistic BTB - https://www.youtube.com/watch?v=YQEq04nNm_o
- Direction Predictor - https://www.youtube.com/watch?v=s4jOyD51dSo
- Problems with 1bit Predictions - https://www.youtube.com/watch?v=malQIOtaAuU
- 2 Bit Predictor - https://www.youtube.com/watch?v=JTXIhNQwhyw
- History Based Predictors - https://www.youtube.com/watch?v=qUfNeC9VGgg
- 1 Bit History With 2 Bit Count - https://www.youtube.com/watch?v=zhyEYt2uylQ
- 2 Bit History Predictor - https://www.youtube.com/watch?v=XvJDif_YIgY
- 2 Bit History Predictor 2 - https://www.youtube.com/watch?v=Q2W_m6YTp50
- Predication - https://www.youtube.com/watch?v=f4Nx3-chbqA

Made with Goodnotes